

软件著作权源程序鉴别材料

飞鸟快验网络系统 V1.0.449

说明：本材料按 60 页、每页 50 行生成。前 10 页为程序前部结构与 WebApi 入口代码，后续 50 页为连续选取的代表性核心业务代码。

文件路径	起始行	结束行	行数
main.go	1	87	87
core/core.go	1	155	155
core/InitRouter.go	1	150	150
new/app/router/Route.go	1	37	37
new/app/router/webApi2/webApi.go	1	71	71
new/app/controller/admin/KuaiYanFull.go	1	530	530
new/app/logic/common/VMP/VMP.go	1	186	186
new/app/logic/common/rmbPay/rmbPay.go	1	947	947
new/app/logic/common/ka/ka.go	1	837	837

```
main.go:1      package main
main.go:2
main.go:3      import (
main.go:4          "context"
main.go:5          "fmt"
main.go:6          "github.com/gin-gonic/gin"
main.go:7          "github.com/songzhibin97/gkit/cache/local_cache"
main.go:8          "go.uber.org/zap"
main.go:9          "server/new/app/logic/webSocket"
main.go:10
main.go:11         //"net/http"
main.go:12         //_ "net/http/pprof" // 开启 pprof
main.go:13         "os"
main.go:14         "runtime/debug"
main.go:15         "server/Service/Ser_Init"
main.go:16         "server/api/middleware"
main.go:17         "server/core"
main.go:18         "server/global"
main.go:19         "server/new"
main.go:20         _ "server/new/app/logic"
main.go:21         "server/new/app/logic/common/setting"
main.go:22         "time"
main.go:23     )
```

```

main.go:24
main.go:25      /*设置包自动管理*/
main.go:26      //go:generate go env -w G0111MODULE=on
main.go:27
main.go:28      /*设置包管理下载地址*/
main.go:29      //go:generate go env -w GOPROXY=https://goproxy.cn,direct
main.go:30
main.go:31      /*设置包自动整理信息到 go.mod 内*/
main.go:32      //go:generate go mod tidy
main.go:33
main.go:34      /*设置包自动根据 go.mod 下载*/
main.go:35      //go:generate go mod download
main.go:36
main.go:37      func main() {
main.go:38          //go log.Fatal(http.ListenAndServe("127.0.0.1:6060", nil))
main.go:39          defer func() {
main.go:40              if err := recover(); err != nil {
main.go:41                  局_上报错误 := fmt.Sprintf("全局捕获错误:\n", err, "\n 堆
栈信息:\n", string(debug.Stack()))

main.go:42                  fmt.Println("发生致命错误:", 局_上报错误)
main.go:43                  global.Q 快验.Z 置新用户消息(2, 局_上报错误)

main.go:44              }
main.go:45          }()
main.go:46
main.go:47
main.go:48          global.GVA_Viper = core.InitViper() //初始化配置读写器 和全局配置结
构变量 GVA_config

main.go:49
main.go:50          global.GVA_LOG = core.InitZap()    // 初始化 zap 日志记录器
第 1 页

main.go:51          zap.ReplaceGlobals(global.GVA_LOG) //替换系统的 log 记录器 为 zap 的
全局日志记录器 方便统一管理

main.go:52          global.H 缓存 = local_cache.NewCache( //需要比数据库先初始化,因为读
取系统配置就用到缓存了

main.go:53              //设置缓存默认超时时间 为 24 小时
main.go:54              local_cache.SetDefaultExpire(time.Hour*24),
main.go:55              local_cache.SetCapture(func(k string, v interface{}) { //设
置缓存删除捕获函数,缓存到期删除时会触发

main.go:56                  //fmt.Printf("delete k:%s v:%v\n", k, v)
main.go:57              }),
main.go:58          )
main.go:59          global.GVA_DB, _ = Ser_Init.InitGormMysql() // gorm 连接数据库
Gorm 参考资料 https://www.cnblogs.com/davis12/p/16365213.html

main.go:60
main.go:61          if global.GVA_DB != nil { //如果数据库不为空
main.go:62              c := gin.Context{}

```

```

main.go:63          Ser_Init.InitDbTables(&c) // 如果数据库连接成功就初始化表 //不
                  在这里了,只能由 InitMysql 初始化

main.go:64
main.go:65          // 程序结束前关闭数据库链接
main.go:66          db, _ := global.GVA_DB.DB()
main.go:67          defer db.Close()
                  //延迟关闭程序结束前关闭表

main.go:68          middleware.J 集_UserAPI 路由_加密.G 更新 md5API 名称(setting.Q 系
                  统设置().Y 用户 API 加密盐) //只有数据库成功才可以操作 不然或报错

main.go:69          websocket.L_webSocket.D 断开所有连接()
main.go:70          } else {
main.go:71              global.GVA_LOG.Info(fmt.Sprintf("数据库连接失败,等待输入数据库
                  信息"))
main.go:72          }
main.go:73          core.InitCron 定时任务()
main.go:74          new.Main()
main.go:75
main.go:76          core.RunWindowsServer() //启动 web 服务器 先启动 不然无法自验证

main.go:77
main.go:78          }
main.go:79
main.go:80          func STOP() {
main.go:81              global.Cron 定时任务.Cron.Stop()
main.go:82              //先关闭端口 解除占用
main.go:83              global.GVA_Gin.Shutdown(context.Background()) //这句话可以停止侦听
                  关闭端口

main.go:84              // 退出当前进程
main.go:85              os.Exit(0)
main.go:86
main.go:87          }
core.go:1          package core
core.go:2
core.go:3          import (
core.go:4              "EFunc/utils"
core.go:5              "fmt"
core.go:6              "github.com/gin-gonic/gin"
core.go:7              "github.com/spf13/viper"
core.go:8              "go.uber.org/zap"
core.go:9              "go.uber.org/zap/zapcore"
core.go:10             "os"
core.go:11             "runtime"
core.go:12
core.go:13             "server/core/internal"

                  第 2 页

core.go:14             "server/global"
core.go:15             "server/new/app/logic/common/cron"
core.go:16             "server/new/app/logic/common/cron/functions"
core.go:17             kuaiyanctrl "server/new/app/controller/admin"

```

```

core.go:18
core.go:19         utils2 "server/utils"
core.go:20     )
core.go:21
core.go:22     // InitZap 日志系统初始化
core.go:23     // Author [SliverHorn](https://github.com/SliverHorn)
core.go:24     func InitZap() (logger *zap.Logger) {
core.go:25         // 判断是否有 Director 文件夹 没有就创建
core.go:26         if ok, _ :=
core.go:27             utils2.PathExists(global.GVA_CONFIG.Zap.Director); !ok {
core.go:28             fmt.Printf("create %v directory\n",
core.go:29                 global.GVA_CONFIG.Zap.Director)
core.go:30             _ = os.Mkdir(global.GVA_CONFIG.Zap.Director, os.ModePerm)
core.go:31         }
core.go:32         //获取日志配置信息
core.go:33         cores := internal.Zap.GetZapCores()
core.go:34         //创建日志记录器 logger
core.go:35         logger = zap.New(zapcore.NewTee(cores...))
core.go:36         //判断全局日志配饰 是否显示文件方法行号信息
core.go:37         if global.GVA_CONFIG.Zap.ShowLine {
core.go:38             logger = logger.WithOptions(zap.AddCaller())
core.go:39         }
core.go:40         //返回创建的日志记录器
core.go:41         return logger
core.go:42     }
core.go:43
core.go:44     // InitViper //
core.go:45     // 新建并初始化配置读写器赋值给全局变量 GVA_Viper 并把配置信息反序列化到全
core.go:46     // 局文件
core.go:47     func InitViper() *viper.Viper {
core.go:48
core.go:49         //新建一个 viper 配置读写器
core.go:50
core.go:51         v := viper.New()
core.go:52         //写入默认值
core.go:53         v.SetDefault("管理入口", "Admin")
core.go:54         v.SetDefault("代理入口", "Agent")
core.go:55         v.SetDefault("Port", 18888)
core.go:56         v.SetDefault("系统模式", 0)
core.go:57         //=====验证码默认配置
core.go:58         v.SetDefault("captcha.open-captcha", 1) //设置验证码默
core.go:59         //认 ip 防暴次数
core.go:60         v.SetDefault("captcha.open-captcha-timeout", 3600) //防暴时间 被
core.go:61         //爆破后开启验证秒数
core.go:62         v.SetDefault("captcha.img-height", 80) //设置验证码高
core.go:63         //度

```

```

core.go:58          v.SetDefault("captcha.img-width", 240)          //设置验证码宽
                        度
core.go:59          v.SetDefault("captcha.Key-long", 4)          //设置验证码长
core.go:60          v.SetDefault("captcha.Type", 1)              //设置验证码
                        类型    mark 后期类型拓展滑动验证码
core.go:61          //=====日志默认配置
core.go:62          v.SetDefault("zap.director", "log")
//设置日志文件目录
core.go:63          v.SetDefault("zap.encode-level", "LowercaseColorLevelEncoder")
//设置日志文件编码

                                第 3 页
core.go:64          v.SetDefault("zap.format", "console")
//设置是否替换系统日志
core.go:65          v.SetDefault("zap.level", "info")
//设置是否替换系统日志
core.go:66          v.SetDefault("zap.log-in-console", "true")
//设置是否输出到控制台
core.go:67          v.SetDefault("zap.max-age", 0)
//设置日志最大数量
core.go:68          v.SetDefault("zap.show-line", true)
//设置显示代码行号
core.go:69          v.SetDefault("zap.stacktrace-key", "stacktrace")
//设置显示栈名
core.go:70          //=====数据库默认配置
core.go:71          v.SetDefault("mysql.Config", "")
core.go:72          v.SetDefault("mysql.Dbname", "")
core.go:73          v.SetDefault("mysql.LogMode", "error")
core.go:74          v.SetDefault("mysql.LogZap", true)
core.go:75          v.SetDefault("mysql.MaxIdleConns", 10)
core.go:76          v.SetDefault("mysql.MaxOpenConns", 100)
core.go:77          v.SetDefault("mysql.Path", "")
core.go:78          v.SetDefault("mysql.Port", "3306")
core.go:79          v.SetDefault("mysql.Prefix", "")
core.go:80          v.SetDefault("mysql.Singular", false)
core.go:81          v.SetDefault("mysql.Username", "")
core.go:82          global.GVA_CONFIG.Q 取运行目录 = utils.C 程序_取运行目录()
core.go:83          if runtime.GOOS == "windows" {
core.go:84              global.GVA_CONFIG.Q 取运行目录 = "."
core.go:85          }
core.go:86          path := global.GVA_CONFIG.Q 取运行目录 + "/config.json" //设置文件
                        目录    //注意设置 ./config.json 宝塔写文件不会写运行目录 文件会在
                        /www/server/panel 文件夹

core.go:87
core.go:88          //设置路径
core.go:89          v.SetConfigFile(path)
core.go:90          v.SetConfigType("json")
core.go:91          var err error
core.go:92          //读入配置信息

```

```

core.go:93         if utils.W 文件_是否存在(path) {
core.go:94             err = v.ReadInConfig()
core.go:95             //如果 err 不动于空 就说明有问题,输出错误信息
core.go:96             if err != nil {
core.go:97                 panic(fmt.Errorf("读入配置文件失败: %s \n", err))
core.go:98             }
core.go:99         } else {
core.go:100             err = v.WriteConfig()
core.go:101         }
core.go:102         //viper 支持监听配置文件, 并会在配置文件发生变化, 重新读取配置文件和回
//避免每次配置变化时, 都需要重启启动应用的麻烦。

core.go:103         // 监听配置文件 发生手动操作的变化自动读取
core.go:104         //v.WatchConfig()
core.go:105
core.go:106         ////设置配置更新时处理函数 2023/4/23 发现自动回连续读取两次配置,
//导致数据不正确, 停止自动读取,改为手动读取

core.go:107         //v.OnConfigChange(func(e fsnotify.Event) {
core.go:108             // fmt.Println("配置文件已更改:", e.Name)
core.go:109             // //重新读取配置反序列化到全局配置结构里 global.GVA_CONFIG
config.Server 失败输出错误

core.go:110             // if err = v.Unmarshal(&global.GVA_CONFIG); err != nil {
core.go:111                 // fmt.Println("配置文件反序列化失败 2:", err)
core.go:112             // }
core.go:113             //})

```

第 4 页

```

core.go:114
core.go:115         //读取配置反序列化到全局配置结构里 global.GVA_CONFIG
config.Server 失败输出错误

core.go:116         if err = v.Unmarshal(&global.GVA_CONFIG); err != nil {
core.go:117             fmt.Println("配置文件反序列化失败:", err)
core.go:118         }
core.go:119
core.go:120         if global.GVA_CONFIG.Port == 0 {
core.go:121             global.GVA_CONFIG.Port = 18888
core.go:122         }
core.go:123
core.go:124         return v
core.go:125     }
core.go:126
core.go:127     // InitCron 定时任务 //
core.go:128     // 新建 Cron 定时任务,并插入需要初始化的任务
core.go:129     func InitCron 定时任务() {
core.go:130
core.go:131         global.Cron 定时任务 = cron.D 定时任务{} //这里设置的 6 位表达式 秒级
core.go:132         global.Cron 定时任务.Init()
core.go:133

```

```

core.go:134 //1 "0 0 0 * * *" 每天的 0 点启动 * 通配符可以匹配任何数字
core.go:135 //2 "*/5 * * * * *" 表示每隔 5 秒钟执行一次
core.go:136 //3 "*/1 * * * * *" 表示每隔 1 分钟执行一次, 比秒级别解析器少了个*
core.go:137 //4 "30 * * * * *" 分钟域为 30, 其他域都是*表示任意。每 30 分触发
core.go:138 //5 "30 3-6,20-23 * * *": 分钟域为 30, 小时域的 3-6,20-23 表示 3 点
    到 6 点和 20 点到 23 点。每小时的 30 分钟触发。
core.go:139 //6 "0 0 0 * * ?" 表示每天 0 点执行一次
core.go:140 //7 "0 0 1 1 * ?" 表示每月 1 号凌晨 1 点执行一次
core.go:141 //8 "0 1,2,3 * * * ?" 表示在 1 分, 2 分, 3 分执行一次
core.go:142 //9 "0 0 0,1,2 * * ?" 表示每天的 0 点, 1 点, 2 点执行一次
core.go:143 err := global.Cron 定时任务.T 添加本机任务("快验心跳", "0 */5 * * *
    ", kuaiyanctrl.K 快验心跳)
core.go:144 if err != nil {
core.go:145     global.GVA_LOG.Error("T 添加任务定时任务快验心跳失败:" +
err.Error())
core.go:146 } //5 分钟心跳执行一次
core.go:147
core.go:148 err = global.Cron 定时任务.T 添加本机任务("定时刷新数据库定时任务 2",
    "0 */1 * * * *", functions.S 刷新数据库定时任务 2)
core.go:149 if err != nil {
core.go:150     global.GVA_LOG.Error("定时刷新数据库定时任务 2 失败:" +
err.Error())
core.go:151 }
core.go:152 _ = functions.S 刷新数据库定时任务(true)
core.go:153 functions.D 定时任务_统计初始化日活月活(&gin.Context{})
core.go:154 global.Cron 定时任务.Cron.Start()
core.go:155 }
InitRouter.go:1 package core
InitRouter.go:2
InitRouter.go:3 import (
InitRouter.go:4     "fmt"
InitRouter.go:5     "github.com/gin-gonic/gin"
InitRouter.go:6     "github.com/gin-gonic/gin/binding"
InitRouter.go:7     "github.com/go-playground/locales/zh"
InitRouter.go:8     ut "github.com/go-playground/universal-translator"

    第 5 页
InitRouter.go:9     "github.com/go-playground/validator/v10"
InitRouter.go:10     enTranslations "github.com/go-
playground/validator/v10/translations/en"
InitRouter.go:11     zhTranslations "github.com/go-
playground/validator/v10/translations/zh"
InitRouter.go:12     "io"
InitRouter.go:13     "log"
InitRouter.go:14     "reflect"
InitRouter.go:15     "runtime/debug"
InitRouter.go:16     "server/api/UserApi"
InitRouter.go:17     // "server/api/Admin" // [已迁移到新架构]

```

```

InitRouter.go:18      // "server/api/WebApi" // [已迁移到新架构]
InitRouter.go:19      "server/api/middleware"
InitRouter.go:20      "server/global"
InitRouter.go:21      "server/new/app/router"
InitRouter.go:22      "server/structs/Http/response"
InitRouter.go:23  )
InitRouter.go:24
InitRouter.go:25      // InitRouters 初始化总路由
InitRouter.go:26      func InitRouters() *gin.Engine {
InitRouter.go:27
InitRouter.go:28          if !(global.GVA_Viper.GetInt("系统模式") == 1056795985) {
InitRouter.go:29              gin.DefaultWriter = io.Discard //禁止控制台输出
InitRouter.go:30              gin.SetMode(gin.ReleaseMode)    //设置为生产模式
InitRouter.go:31          }
InitRouter.go:32
InitRouter.go:33          Router := gin.Default() //返回路由实例
InitRouter.go:34          _ = InitTrans("ZH")
InitRouter.go:35          // Router.Use(middleware.LoadTls()) // 如果需要使用 https 请打开此
          中间件 然后前往 core/server.go 将启动模式 更变为 Router.RunTLS("端口", "
          你的 cre/pem 文件", "你的 key 文件")

InitRouter.go:36
InitRouter.go:37          // 跨域, 如需跨域可以打开下面的注释
InitRouter.go:38          Router.Use(middleware.Cors()) // 直接放行全部跨域请求
InitRouter.go:39          Router.Use(T 统一恐慌恢复()) // 直接放行全部跨域请求
InitRouter.go:40          //公共路由器 无需鉴权
InitRouter.go:41          PublicGroup := Router.Group("")
InitRouter.go:42          RouterUserApi(PublicGroup) //先注册用户路由,因为管理员应用设置需要验
          证码接口需要获取用户 api 列表

InitRouter.go:43          //RouterAdmin(PublicGroup) // [已迁移到新架构] Admin 路由由
          new/app/router/admin/admin.go 注册

InitRouter.go:44          //RouterWebApi(PublicGroup) // [已迁移到新架构] WebApi 路由由
          new/app/router/webApi2/webApi.go 注册

InitRouter.go:45          router.RouterInit(PublicGroup)
InitRouter.go:46          if global.GVA_Viper.GetInt("系统模式") == 1 {
InitRouter.go:47              Router.NoRoute(func(c *gin.Context) {
InitRouter.go:48                  response.FailWithMessage("演示模式不可操作,请部署到自己服
          务器深度体验", c)

InitRouter.go:49                  return
InitRouter.go:50              })
InitRouter.go:51          }
InitRouter.go:52
InitRouter.go:53          //global.GVA_LOG.Info("router register success(路由注册成功)")
InitRouter.go:54          return Router
InitRouter.go:55      }
InitRouter.go:56
InitRouter.go:57      // InitTrans 初始化控制器翻译器

```



```

InitRouter.go:58 func InitTrans(locale string) (err error) {
InitRouter.go:59     // 修改 gin 框架中的 Validator 引擎属性, 实现自定义
InitRouter.go:60     if v, ok := binding.Validator.Engine().(*validator.Validate); ok
{
InitRouter.go:61
InitRouter.go:62         // 注册一个获取 json tag 的自定义方法 将字段名改为中文,
InitRouter.go:63         v.RegisterTagNameFunc(func(fld reflect.StructField) string {
InitRouter.go:64             name := fld.Tag.Get("zh") //如果有这个信息,就是用这个
InitRouter.go:65             if name == "" {
InitRouter.go:66                 name = fld.Tag.Get("json") //没有就用 json
InitRouter.go:67             }
InitRouter.go:68             return name
InitRouter.go:69         })
InitRouter.go:70
InitRouter.go:71         zhT := zh.New() // 中文翻译器
InitRouter.go:72         //enT := en.New() // 英文翻译器
InitRouter.go:73         // 第一个参数是备用 (fallback) 的语言环境
InitRouter.go:74         // 后面的参数是应该支持的语言环境 (支持多个)
InitRouter.go:75         uni := ut.New(zhT, zhT) //也是可以的
InitRouter.go:76         //uni := ut.New(enT, zhT, enT)
InitRouter.go:77
InitRouter.go:78         // locale 通常取决于 http 请求头的 'Accept-Language'
InitRouter.go:79         // 也可以使用 uni.FindTranslator(...) 传入多个 locale 进行查找
InitRouter.go:80         global.Trans, ok = uni.GetTranslator(locale)
InitRouter.go:81         if !ok {
InitRouter.go:82             return fmt.Errorf("uni.GetTranslator(%s) failed",
locale)
InitRouter.go:83         }
InitRouter.go:84
InitRouter.go:85         // 注册翻译器
InitRouter.go:86         switch locale {
InitRouter.go:87             case "en":
InitRouter.go:88                 err = enTranslations.RegisterDefaultTranslations(v,
global.Trans)
InitRouter.go:89             case "zh":
InitRouter.go:90                 err = zhTranslations.RegisterDefaultTranslations(v,
global.Trans)
InitRouter.go:91             default:
InitRouter.go:92                 err = zhTranslations.RegisterDefaultTranslations(v,
global.Trans)
InitRouter.go:93             }
InitRouter.go:94             return
InitRouter.go:95         }
InitRouter.go:96         return
InitRouter.go:97     }
InitRouter.go:98     func T 统一恐慌恢复() gin.HandlerFunc {
InitRouter.go:99         return func(c *gin.Context) {
InitRouter.go:100             defer func() {

```

```

InitRouter.go:101         if err := recover(); err != nil {
InitRouter.go:102             局_上报错误 := fmt.Sprintf("全局捕获错误:\n", err,
"\n 堆栈信息:\n", string(debug.Stack()))
InitRouter.go:103             debug.PrintStack()
InitRouter.go:104             log.Println("发生致命错误:", 局_上报错误)
InitRouter.go:105             /*
c.JSON(http.StatusInternalServerError, gin.H{
InitRouter.go:106                 "error":
"Internal Server Error",
InitRouter.go:107                 })*/
InitRouter.go:108             global.Q 快验.Z 置新用户消息(2, 局_上报错误)

第 7 页

InitRouter.go:109         }
InitRouter.go:110     }()
InitRouter.go:111     c.Next()
InitRouter.go:112 }
InitRouter.go:113 }
InitRouter.go:114
InitRouter.go:115 // admin 路由 menu 需要鉴权 menu
InitRouter.go:116 // [已迁移到新架构 new/app/router/admin/admin.go] 所有 Admin 路由由新架构
注册

InitRouter.go:117
InitRouter.go:118 // Agent 路由 menu 需要鉴权 menu
InitRouter.go:119 // RouterUserApi UserApi 路由入口
InitRouter.go:120 func RouterUserApi(Router *gin.RouterGroup) *gin.RouterGroup {
InitRouter.go:121
InitRouter.go:122     baseRouter := Router.Group("/Api")
InitRouter.go:123     baseRouter.Use(middleware.UserApi 检查数据库连接()) //检查数据库是
否连接,连接后可以使用用户 Api,不然大量报错

InitRouter.go:124     baseRouter.Use(middleware.UserApi 无 Token 解密()) ///鉴权中间件 检
查 token 单独优先处理

InitRouter.go:125     baseRouter.Use(middleware.UserApi 解密()) ///鉴权中间件 检查
token

InitRouter.go:126     {
InitRouter.go:127         baseRouter.POST("", UserApi.UserApi_Api 不存在)
InitRouter.go:128         //其余的都在中间件内分配
InitRouter.go:129     }
InitRouter.go:130
InitRouter.go:131     return baseRouter
InitRouter.go:132 }
InitRouter.go:133
InitRouter.go:134 // WebApi 路由入口
InitRouter.go:135 // [已迁移到新架构 new/app/router/webApi2/webApi.go] 路由由
new/app/router/webApi2 注册

InitRouter.go:136 //func RouterWebApi(Router *gin.RouterGroup) *gin.RouterGroup {
InitRouter.go:137 //
InitRouter.go:138 // //=====

```

```

InitRouter.go:139 // baseRouter := Router.Group("/WebApi/") //WebApi 不做任何加密中间件
                  处理
InitRouter.go:140 // baseRouter.Use(middleware.IsWebApiHost())
InitRouter.go:141 // baseRouter.Use(middleware.IsTokenWebApi()) ///鉴权中间件 检查
                  token 单独优先处理
InitRouter.go:142 // {
InitRouter.go:143 //     for 键名, 键值 := range WebApi.J 集_UserApi 路由 {
InitRouter.go:144 //         baseRouter.GET(键名, 键值.Z 指向函数)
InitRouter.go:145 //         baseRouter.POST(键名, 键值.Z 指向函数)
InitRouter.go:146 //     }
InitRouter.go:147 // }
InitRouter.go:148 //
InitRouter.go:149 // return baseRouter
InitRouter.go:150 //}

Route.go:1 package router
Route.go:2
Route.go:3 import (
Route.go:4     "github.com/gin-gonic/gin"
Route.go:5     "server/new/app/router/admin"
Route.go:6     "server/new/app/router/agent"
Route.go:7     "server/new/app/router/webApi2"
Route.go:8     "server/new/app/router/webSocket"
                  第 8 页
Route.go:9     "server/new/app/router/webUser"
Route.go:10 )
Route.go:11
Route.go:12 // 总路由
Route.go:13 var AllRouter = new(RouterGroup)
Route.go:14
Route.go:15 type RouterGroup struct {
Route.go:16 }
Route.go:17
Route.go:18 func RouterInit(gin *gin.RouterGroup) *gin.RouterGroup {
Route.go:19
Route.go:20     Router := gin //返回路由实例
Route.go:21
Route.go:22     // 跨域, 如需跨域可以打开下面的注释
Route.go:23     router := admin.AllRouter{}
Route.go:24     router.InitAdminRouter(Router) //初始化管理员路由
Route.go:25
Route.go:26     routerAgent := agent.AllRouter{}
Route.go:27     routerAgent.InitAgentRouter(Router) //初始化 Agent 路由
Route.go:28
Route.go:29     routerWebApi := webApi2.AllRouter{}
Route.go:30     routerWebApi.InitWebApiRouter(Router) //初始化 WEBApi 路由
Route.go:31
Route.go:32     routerWebUser := webUser.AllRouter{}
Route.go:33     routerWebUser.InitWebUserRouter(Router) //初始化 WEBUser 路由

```

```

Route.go:34         routerWebSocket := websocket.AllRouter{}
Route.go:35         routerWebSocket.InitWebSocketRouter(router) //初始化WebSocket 路由

Route.go:36         return gin
Route.go:37     }
webApi.go:1 package webApi2
webApi.go:2
webApi.go:3 import (
webApi.go:4     "github.com/gin-gonic/gin"
webApi.go:5     controller "server/new/app/controller/webApi"
webApi.go:6     "server/new/app/router/middleware"
webApi.go:7 )
webApi.go:8
webApi.go:9 type 路由信息 struct {
webApi.go:10     Z 中文名 string
webApi.go:11     Z 指向函数 func(*gin.Context)
webApi.go:12 }
webApi.go:13
webApi.go:14 var J 集_UserApi 路由 2 = map[string]路由信息{}
webApi.go:15
webApi.go:16 type AllRouter struct {
webApi.go:17 }
webApi.go:18
webApi.go:19 func (r *AllRouter) InitWebApiRouter(router *gin.RouterGroup) {
webApi.go:20     // 跨域, 如需跨域可以打开下面的注释
webApi.go:21     webApiRouter := router.Group("webApi")
webApi.go:22
webApi.go:23     //回调支付
webApi.go:24     局_NotifyController := controller.NewPayNotifyController()
webApi.go:25     {
webApi.go:26         //http://anyueyinluo.w1.luyouxia.net/payNotify/240426223543000001
webApi.go:27         webApiRouter.POST("/payNotify/:order", 局
webApi.go:28         _NotifyController.PayNotify) //通用支付回调
webApi.go:29         webApiRouter.GET("/payNotify/:order", 局
webApi.go:30         _NotifyController.PayNotify) //通用支付回调
webApi.go:31         webApiRouter.POST("/payNotify2/:order", 局
webApi.go:32         _NotifyController.PayNotify2) //通用退款回调
webApi.go:33     }
webApi.go:34     //兼容旧版小叮当 因为服务器已配置
webApi.go:35     webApiRouter = router.Group("WebApi")
webApi.go:36     {
webApi.go:37         webApiRouter.POST("/PayXiaoDingDangNotify", 局
webApi.go:38         _NotifyController.PayNotify) //小叮当回调
webApi.go:39     }
webApi.go:40     //上面的不用令牌就可以访问
webApi.go:41     //下边的需要令牌才可以访问

```

```

webApi.go:37         webApiRouter = router.Group("WebApi")
webApi.go:38         webApiRouter.Use(middleware.IsWebApiHost())
webApi.go:39         webApiRouter.Use(middleware.IsTokenWebApi()) ///鉴权中间件 检查
token 单独优先处理

webApi.go:40         //云存储
webApi.go:41         局_CloudStorageController :=
controller.NewCloudStorageController()
webApi.go:42         {
webApi.go:43
webApi.go:44             J 集_UserApi 路由 2["CloudStorage/GetUploadToken"] = 路由信息{Z
中文名: "云存储_取文件上传授权", Z 指向函数: 局
_CloudStorageController.GetUploadToken}
webApi.go:45             J 集_UserApi 路由 2["CloudStorage/GetDownloadUrl"] = 路由信息{Z
中文名: "云存储_取外链", Z 指向函数: 局
_CloudStorageController.GetDownloadUrl}
webApi.go:46         }
webApi.go:47
webApi.go:48         // ===== 以下为从 api/WebApi 迁移的路由 =====
webApi.go:49
webApi.go:50         //任务池
webApi.go:51         局_TaskPoolController := controller.NewTaskPoolWebApiController()
webApi.go:52         {
webApi.go:53             J 集_UserApi 路由 2["TaskPoolGetTask"] = 路由信息{Z 中文名: "任
务处理获取", Z 指向函数: 局_TaskPoolController.TaskPoolGetTask}
webApi.go:54             J 集_UserApi 路由 2["TaskPoolSetTask"] = 路由信息{Z 中文名: "任
务处理返回", Z 指向函数: 局_TaskPoolController.TaskPoolSetTask}
webApi.go:55             J 集_UserApi 路由 2["TaskPoolNewData"] = 路由信息{Z 中文名: "任
务池_任务创建", Z 指向函数: 局_TaskPoolController.TaskPoolNewData}
webApi.go:56             J 集_UserApi 路由 2["TaskPoolGetData"] = 路由信息{Z 中文名: "任
务池_任务查询", Z 指向函数: 局_TaskPoolController.TaskPoolGetData}
webApi.go:57         }
webApi.go:58         //公共函数
webApi.go:59         {
webApi.go:60             J 集_UserApi 路由 2["RunJs"] = 路由信息{Z 中文名: "运行公共函数",
Z 指向函数: 局_TaskPoolController.RunJs}
webApi.go:61             J 集_UserApi 路由 2["RunJs/:JsName"] = 路由信息{Z 中文名: "执行
公共 Js 函数 2", Z 指向函数: 局_TaskPoolController.RunJs2}
webApi.go:62         }
webApi.go:63         //卡号相关
webApi.go:64         局_KaController := controller.NewKaWebApiController()
webApi.go:65         {
webApi.go:66             J 集_UserApi 路由 2["GetKaInfo"] = 路由信息{Z 中文名: "取卡号详细
信息", Z 指向函数: 局_KaController.GetKaInfo}
webApi.go:67             J 集_UserApi 路由 2["NewKa"] = 路由信息{Z 中文名: "新制卡号", Z
指向函数: 局_KaController.NewKa}
webApi.go:68         }

```

```

webApi.go:69          //支付订单
webApi.go:70          局_PayController := controller.NewPayWebApiController()
webApi.go:71          {

```

第 10 页

```

KuaiYanFull.go:1      package controller
KuaiYanFull.go:2
KuaiYanFull.go:3      import (
KuaiYanFull.go:4          "context"
KuaiYanFull.go:5          "encoding/json"
KuaiYanFull.go:6          "fmt"
KuaiYanFull.go:7          "os"
KuaiYanFull.go:8          "os/exec"
KuaiYanFull.go:9          "path"
KuaiYanFull.go:10         "runtime"
KuaiYanFull.go:11         "time"
KuaiYanFull.go:12
KuaiYanFull.go:13         "EFunc/utils"
KuaiYanFull.go:14         "github.com/gin-gonic/gin"
KuaiYanFull.go:15         "github.com/valyala/fastjson"
KuaiYanFull.go:16         "server/Service/KuaiYanUpdater"
KuaiYanFull.go:17         "server/Service/Ser_Ka"
KuaiYanFull.go:18         "server/Service/Ser_LinkUser"
KuaiYanFull.go:19         "server/Service/Ser_User"
KuaiYanFull.go:20         "server/global"
KuaiYanFull.go:21         "server/new/app/controller/Common"
KuaiYanFull.go:22         "server/structs/Http/response"
KuaiYanFull.go:23         utils2 "server/utils"
KuaiYanFull.go:24     )
KuaiYanFull.go:25
KuaiYanFull.go:26     type KuaiYan struct {
KuaiYanFull.go:27         Common.Common
KuaiYanFull.go:28     }
KuaiYanFull.go:29
KuaiYanFull.go:30     func NewKuaiYanController() *KuaiYan {
KuaiYanFull.go:31         return &KuaiYan{}
KuaiYanFull.go:32     }
KuaiYanFull.go:33
KuaiYanFull.go:34     type 结构请求_个人信息 struct {
KuaiYanFull.go:35         User      string `json:"user"`
KuaiYanFull.go:36         Password string `json:"password"`
KuaiYanFull.go:37         Qq         string `json:"qq"`
KuaiYanFull.go:38         CaptCha1 struct {
KuaiYanFull.go:39             CaptchaId   string `json:"captchaId"`
KuaiYanFull.go:40             CaptChaValue string `json:"captChaValue"`
KuaiYanFull.go:41         } `json:"captCha1"`
KuaiYanFull.go:42         CaptCha2 struct {
KuaiYanFull.go:43             CaptchaId   string `json:"captchaId"`
KuaiYanFull.go:44             CaptChaValue string `json:"captChaValue"`
KuaiYanFull.go:45         } `json:"captCha2"`
KuaiYanFull.go:46         CaptCha3 struct {
KuaiYanFull.go:47             CaptchaId   string `json:"captchaId"`
KuaiYanFull.go:48             CaptChaValue string `json:"captChaValue"`

```

```
KuaiYanFull.go:49     } `json:"captCha3"`
KuaiYanFull.go:50 }
```

第 11 页

```
KuaiYanFull.go:51
KuaiYanFull.go:52 type 结构请求_KaClassId struct {
KuaiYanFull.go:53     KaClassId int `json:"kaClassId"`
KuaiYanFull.go:54 }
KuaiYanFull.go:55
KuaiYanFull.go:56 type 结构请求_充值 struct {
KuaiYanFull.go:57     Ka          string `json:"ka"`
KuaiYanFull.go:58     InviteUser string `json:"inviteUser"`
KuaiYanFull.go:59 }
KuaiYanFull.go:60
KuaiYanFull.go:61 type 结构请求_余额充值 struct {
KuaiYanFull.go:62     Type string `json:"type"`
KuaiYanFull.go:63     C 充值金额 float64 `json:"rmb"`
KuaiYanFull.go:64     D 订单 ID string `json:"orderId"`
KuaiYanFull.go:65 }
KuaiYanFull.go:66
KuaiYanFull.go:67 // GetCaptchaApiList 取开启验证码接口列表
KuaiYanFull.go:68 func (k *KuaiYan) GetCaptchaApiList(c *gin.Context) {
KuaiYanFull.go:69     var 跳转过标记 = false
KuaiYanFull.go:70     标记:
KuaiYanFull.go:71     var 响应信息 string
KuaiYanFull.go:72     if !global.Q 快验.Q 取开启验证码接口列表(&响应信息) {
KuaiYanFull.go:73         var 错误代码 = 0
KuaiYanFull.go:74         global.Q 快验.Q 取错误信息(&错误代码)
KuaiYanFull.go:75         if 错误代码 == 109 && !跳转过标记 {
KuaiYanFull.go:76             global.Q 快验.J_Token = ""
KuaiYanFull.go:77             跳转过标记 = true
KuaiYanFull.go:78             goto 标记
KuaiYanFull.go:79         }
KuaiYanFull.go:80         response.FailWithMessage("取开启验证码接口列表失败:"+global.Q
快验.Q 取错误信息(nil), c)
KuaiYanFull.go:81         return
KuaiYanFull.go:82     }
KuaiYanFull.go:83     global.X 系统信息.K 开启验证码接口列表 = 响应信息
KuaiYanFull.go:84     response.OkWithDetailed(gin.H{"map": 响应信息}, "获取成功", c)
KuaiYanFull.go:85     return
KuaiYanFull.go:86 }
KuaiYanFull.go:87
KuaiYanFull.go:88 // GetCaptcha 取英数验证码
KuaiYanFull.go:89 func (k *KuaiYan) GetCaptcha(c *gin.Context) {
KuaiYanFull.go:90     var 请求 结构请求_个人信息
KuaiYanFull.go:91     err := c.ShouldBindJSON(&请求)
```

```

KuaiYanFull.go:92         if err != nil {
KuaiYanFull.go:93             response.FailWithMessage("提交参数错误:"+err.Error(), c)
KuaiYanFull.go:94             return
KuaiYanFull.go:95         }
KuaiYanFull.go:96         var 跳转过标记 = false
KuaiYanFull.go:97     标记:
KuaiYanFull.go:98         var 响应信息 string
KuaiYanFull.go:99         if !global.Q 快验.Q 取验证码(&响应信息, 1) {
KuaiYanFull.go:100             var 错误代码 = 0
KuaiYanFull.go:101             第 12 页
KuaiYanFull.go:101             global.Q 快验.Q 取错误信息(&错误代码)
KuaiYanFull.go:102             if 错误代码 == 109 && !跳转过标记 {
KuaiYanFull.go:103                 global.Q 快验.J_Token = "" //后台被强行提出就会失败 需要重
KuaiYanFull.go:103                 新获取 token
KuaiYanFull.go:104                 跳转过标记 = true
KuaiYanFull.go:105                 goto 标记
KuaiYanFull.go:106             }
KuaiYanFull.go:107             response.FailWithMessage("获取英数验证码失败:"+global.Q 快验.Q
KuaiYanFull.go:107             取错误信息(nil), c)
KuaiYanFull.go:108             return
KuaiYanFull.go:109         }
KuaiYanFull.go:110         局_json, _ := fastjson.Parse(响应信息)
KuaiYanFull.go:111         response.OkWithDetailed(gin.H{"captchaId": string(局
KuaiYanFull.go:111         _json.GetStringBytes("CaptchaId")), "picPath": string(局
KuaiYanFull.go:111         _json.GetStringBytes("CaptChaImg"))}, "获取成功", c)
KuaiYanFull.go:112         return
KuaiYanFull.go:113     }
KuaiYanFull.go:114
KuaiYanFull.go:115     // GetUserInfo 快验个人信息更新
KuaiYanFull.go:116     func (k *KuaiYan) GetUserInfo(c *gin.Context) {
KuaiYanFull.go:117         var 请求 结构请求_个人信息
KuaiYanFull.go:118         err := c.ShouldBindJSON(&请求)
KuaiYanFull.go:119         if err != nil {
KuaiYanFull.go:119             response.FailWithMessage("提交参数错误:"+err.Error(), c)
KuaiYanFull.go:120             return
KuaiYanFull.go:121         }
KuaiYanFull.go:122     }
KuaiYanFull.go:123     var 文件修改日期 = ""
KuaiYanFull.go:124     if len(os.Args) > 0 {
KuaiYanFull.go:125         文件修改日期 = os.Args[0]
KuaiYanFull.go:126         文件信息, err1 := os.Stat(文件修改日期)
KuaiYanFull.go:127         if 文件信息 != nil {
KuaiYanFull.go:128             if false {
KuaiYanFull.go:129                 // linux only
KuaiYanFull.go:130             } else {

```



```

KuaiYanFull.go:131             文件修改日期 = 文件信息.ModTime().String()
KuaiYanFull.go:132         }
KuaiYanFull.go:133     } else {
KuaiYanFull.go:134         文件修改日期 = err1.Error()
KuaiYanFull.go:135     }
KuaiYanFull.go:136     if len(文件修改日期) > 19 {
KuaiYanFull.go:137         文件修改日期 = 文件修改日期[:19]
KuaiYanFull.go:138     }
KuaiYanFull.go:139 }
KuaiYanFull.go:140
KuaiYanFull.go:141     var 局_软件用户信息 string
KuaiYanFull.go:142     if !global.Q 快验.Q 取软件用户信息(&局_软件用户信息, global.X 系统信
息.B 版本号当前) {
KuaiYanFull.go:143         global.Q 快验.J_Token = ""
KuaiYanFull.go:144         response.OkWithDetailed(gin.H{
KuaiYanFull.go:145             "user": "", "userClassName": "", "vipNumber": 0, "rmb":
0,
KuaiYanFull.go:146             "vipTime": 0, "email": "", "loginTime": 0, "loginIp":
"",
KuaiYanFull.go:147             "qq": "", "registerTime": 0, "appVer": global.X 系统信
息.B 版本号当前,
KuaiYanFull.go:148             "appVerUpdateTime": 文件修改日期, "appVerNew": global.X
系统信息.B 版本号最新,
KuaiYanFull.go:149             "linkTokenCount": 0, "agentUid":
global.GVA_Viper.GetInt("duid"),
KuaiYanFull.go:150             }, "获取成功", c)
第 13 页
KuaiYanFull.go:151     return
KuaiYanFull.go:152 }
KuaiYanFull.go:153
KuaiYanFull.go:154     局_应用用户信息 json, _ := fastjson.Parse(局_软件用户信息)
KuaiYanFull.go:155     global.X 系统信息.D 到期时间 = 局_应用用户信息
json.GetInt64("VipTime")
KuaiYanFull.go:156     global.X 系统信息.B 绑定信息 = string(局_应用用户信息
json.GetStringBytes("Key"))
KuaiYanFull.go:157     global.X 系统信息.Z 注册时间 = 局_应用用户信息
json.GetInt("RegisterTime")
KuaiYanFull.go:158     global.X 系统信息.Y 用户类型 = string(局_应用用户信息
json.GetStringBytes("UserClassName"))
KuaiYanFull.go:159     global.X 系统信息.J 积分 = 局_应用用户信息
json.GetFloat64("VipNumber")
KuaiYanFull.go:160     global.X 系统信息.D 登录时间 = 局_应用用户信息
json.GetInt("LoginTime")
KuaiYanFull.go:161
KuaiYanFull.go:162     局_是否需要更新 := false
KuaiYanFull.go:163     响应信息 := ""

```

```

KuaiYanFull.go:164      global.Q 快验.Q 取最新版本检测(&响应信息, global.X 系统信息.B 版本号当
                        前, false, &局_是否需要更新, &global.X 系统信息.B 版本号最新)

KuaiYanFull.go:165
KuaiYanFull.go:166      局_基础信息 := ""
KuaiYanFull.go:167      global.Q 快验.Q 取用户基础信息(&局_基础信息)
KuaiYanFull.go:168      局_基础信息 json, _ := fastjson.Parse(局_基础信息)
KuaiYanFull.go:169
KuaiYanFull.go:170      global.X 系统信息.Y 余额 = 局_基础信息 json.GetFloat64("RMB")
KuaiYanFull.go:171      global.X 系统信息.Y 邮箱 = string(局_基础信息
                        json.GetStringBytes("Email"))
KuaiYanFull.go:172      global.X 系统信息.D 登录 IP = string(局_基础信息
                        json.GetStringBytes("LoginIp"))
KuaiYanFull.go:173      global.X 系统信息.Qq = string(局_基础信息
                        json.GetStringBytes("Qq"))
KuaiYanFull.go:174
KuaiYanFull.go:175      局_在线计数 := 0
KuaiYanFull.go:176      if global.X 系统信息.D 到期时间 < time.Now().Unix() {
KuaiYanFull.go:177          局_在线计数 = int(Ser_LinkUser.Get 取在线总数(true, true))
KuaiYanFull.go:178          global.H 缓存.Set("在线数量", 局_在线计数, time.Minute*10)
KuaiYanFull.go:179      }
KuaiYanFull.go:180
KuaiYanFull.go:181      response.OkWithDetailed(gin.H{
KuaiYanFull.go:182          "user": global.X 系统信息.H 会员帐号, "userClassName":
                        global.X 系统信息.Y 用户类型,
KuaiYanFull.go:183          "vipNumber": global.X 系统信息.J 积分, "rmb": global.X 系统信
                        息.Y 余额,
KuaiYanFull.go:184          "vipTime": global.X 系统信息.D 到期时间, "email": global.X 系统
                        信息.Y 邮箱,
KuaiYanFull.go:185          "loginTime": global.X 系统信息.D 登录时间, "loginIp": global.X
                        系统信息.D 登录 IP,
KuaiYanFull.go:186          "qq": global.X 系统信息.Qq, "registerTime": global.X 系统信
                        息.Z 注册时间,
KuaiYanFull.go:187          "appVer": global.X 系统信息.B 版本号当前, "appVerUpdateTime":
                        文件修改日期,
KuaiYanFull.go:188          "appVerNew": global.X 系统信息.B 版本号最新, "linkTokenCount":
                        局_在线计数,
KuaiYanFull.go:189          "agentUid": global.GVA_Viper.GetInt("duid"),
KuaiYanFull.go:190      }, "获取成功", c)
KuaiYanFull.go:191      return
KuaiYanFull.go:192 }
KuaiYanFull.go:193
KuaiYanFull.go:194 // GetSmsCaptcha 取短信验证码
KuaiYanFull.go:195 func (k *KuaiYan) GetSmsCaptcha(c *gin.Context) {
KuaiYanFull.go:196     var 请求 结构请求_个人信息
KuaiYanFull.go:197     err := c.ShouldBindJSON(&请求)

```

```

KuaiYanFull.go:198     if err != nil {
KuaiYanFull.go:199         response.FailWithMessage("提交参数错误:"+err.Error(), c)
KuaiYanFull.go:200         return
第 14 页
KuaiYanFull.go:201     }
KuaiYanFull.go:202     var 响应信息 string
KuaiYanFull.go:203     if utils.W 文本_是否包含关键字(global.X 系统信息.K 开启验证码接口列表,
`"GetSMSCaptcha"`) {
KuaiYanFull.go:204         if utils.W 文本_是否包含关键字(global.X 系统信息.K 开启验证码接口
列表, `"GetSMSCaptcha":2`) {
KuaiYanFull.go:205             global.Q 快验.Z 置验证码信息(2, 请求.CaptCha2.CaptchaId,
请求.CaptCha2.CaptChaValue)
KuaiYanFull.go:206         } else {
KuaiYanFull.go:207             global.Q 快验.Z 置验证码信息(1, 请求.CaptCha1.CaptchaId,
请求.CaptCha1.CaptChaValue)
KuaiYanFull.go:208         }
KuaiYanFull.go:209     }
KuaiYanFull.go:210     if !global.Q 快验.Q 取短信验证码(&响应信息, 请求.User, "") {
KuaiYanFull.go:211         response.FailWithMessage(global.Q 快验.Q 取错误信息(nil), c)
KuaiYanFull.go:212         return
KuaiYanFull.go:213     }
KuaiYanFull.go:214     局_json, _ := fastjson.Parse(响应信息)
KuaiYanFull.go:215     response.OkWithDetailed(gin.H{"captchaId": string(局
_json.GetStringBytes("CaptchaId"))}, "发送成功", c)
KuaiYanFull.go:216     return
KuaiYanFull.go:217 }
KuaiYanFull.go:218
KuaiYanFull.go:219 // SetPassword 快验找回密码
KuaiYanFull.go:220 func (k *KuaiYan) SetPassword(c *gin.Context) {
KuaiYanFull.go:221     var 请求 结构请求_个人信息
KuaiYanFull.go:222     err := c.ShouldBindJSON(&请求)
KuaiYanFull.go:223     if err != nil {
KuaiYanFull.go:224         response.FailWithMessage("提交参数错误:"+err.Error(), c)
KuaiYanFull.go:225         return
KuaiYanFull.go:226     }
KuaiYanFull.go:227     if !global.Q 快验.M 密码找回或修改_绑定手机(请求.User, 请
求.Password, 请求.CaptCha3.CaptchaId, 请求.CaptCha3.CaptChaValue) {
KuaiYanFull.go:228         response.FailWithMessage(global.Q 快验.Q 取错误信息(nil), c)
KuaiYanFull.go:229         return
KuaiYanFull.go:230     }
KuaiYanFull.go:231     response.OkWithMessage("设置新密码["+请求.Password+"]成功", c)
KuaiYanFull.go:232     return
KuaiYanFull.go:233 }
KuaiYanFull.go:234
KuaiYanFull.go:235 // Register 快验注册
KuaiYanFull.go:236 func (k *KuaiYan) Register(c *gin.Context) {

```

```

KuaiYanFull.go:237     var 请求 结构请求_个人信息
KuaiYanFull.go:238     err := c.ShouldBindJSON(&请求)
KuaiYanFull.go:239     if err != nil {
KuaiYanFull.go:240         response.FailWithMessage("提交参数错误:"+err.Error(), c)
KuaiYanFull.go:241         return
KuaiYanFull.go:242     }
KuaiYanFull.go:243     global.Q 快验.Z 置验证码信息(3, 请求.CaptCha3.CaptchaId, 请
求.CaptCha3.CaptChaValue)
KuaiYanFull.go:244     if !global.Q 快验.Y 用户注册(请求.User, 请求.Password,
c.ClientIP(), utils.W 文本_取随机字符串(10), 请求.Qq, 请
求.Qq+"@qq.com", 请求.User) {
KuaiYanFull.go:245         response.FailWithMessage(global.Q 快验.Q 取错误信息(nil), c)
KuaiYanFull.go:246         return
KuaiYanFull.go:247     }
KuaiYanFull.go:248     response.OkWithMessage("注册成功", c)
KuaiYanFull.go:249     return
KuaiYanFull.go:250 }

```

第 15 页

```

KuaiYanFull.go:251
KuaiYanFull.go:252 // Login 快验登录
KuaiYanFull.go:253 func (k *KuaiYan) Login(c *gin.Context) {
KuaiYanFull.go:254     var 请求 结构请求_个人信息
KuaiYanFull.go:255     err := c.ShouldBindJSON(&请求)
KuaiYanFull.go:256     if err != nil {
KuaiYanFull.go:257         response.FailWithMessage("提交参数错误:"+err.Error(), c)
KuaiYanFull.go:258         return
KuaiYanFull.go:259     }
KuaiYanFull.go:260     var 响应信息 string
KuaiYanFull.go:261     if utils.W 文本_是否包含关键字(global.X 系统信息.K 开启验证码接口列表,
`"UserLogin") {
KuaiYanFull.go:262         if utils.W 文本_是否包含关键字(global.X 系统信息.K 开启验证码接口
列表, `"UserLogin":2`) || 请求.CaptCha2.CaptChaValue != "" {
KuaiYanFull.go:263             global.Q 快验.Z 置验证码信息(2, 请求.CaptCha2.CaptchaId,
请求.CaptCha2.CaptChaValue)
KuaiYanFull.go:264         } else {
KuaiYanFull.go:265             global.Q 快验.Z 置验证码信息(1, 请求.CaptCha1.CaptchaId,
请求.CaptCha1.CaptChaValue)
KuaiYanFull.go:266         }
KuaiYanFull.go:267     }
KuaiYanFull.go:268     if !global.Q 快验.D 登录_通用(&响应信息, 请求.User, 请求.Password,
c.ClientIP(), "", global.X 系统信息.B 版本号当前) {
KuaiYanFull.go:269         response.FailWithMessage(global.Q 快验.Q 取错误信息(nil), c)
KuaiYanFull.go:270         return
KuaiYanFull.go:271     }
KuaiYanFull.go:272     局_json, _ := fastjson.Parse(响应信息)

```

```

KuaiYanFull.go:273     global.X 系统信息.H 会员帐号 = 请求.User
KuaiYanFull.go:274     global.X 系统信息.H 会员密码 = 请求.Password
KuaiYanFull.go:275     global.X 系统信息.D 到期时间 = 局_json.GetInt64("VipTime")
KuaiYanFull.go:276     global.X 系统信息.B 绑定信息 = string(局
                        _json.GetStringBytes("Key"))
KuaiYanFull.go:277     global.X 系统信息.Z 注册时间 = 局_json.GetInt("RegisterTime")
KuaiYanFull.go:278
KuaiYanFull.go:279     局_是否需要更新 := false
KuaiYanFull.go:280     global.Q 快验.Q 取最新版本检测(&响应信息, global.X 系统信息.B 版本号当
                        前, false, &局_是否需要更新, &global.X 系统信息.B 版本号最新)
KuaiYanFull.go:281     global.Q 快验.Q 取用户余额(&global.X 系统信息.Y 余额)
KuaiYanFull.go:282     局_基础信息 := ""
KuaiYanFull.go:283     global.Q 快验.Q 取用户基础信息(&局_基础信息)
KuaiYanFull.go:284     局_基础信息 json, _ := fastjson.Parse(响应信息)
KuaiYanFull.go:285
KuaiYanFull.go:286     global.X 系统信息.Y 用户类型 = string(局
                        _json.GetStringBytes("UserClassName"))
KuaiYanFull.go:287     global.X 系统信息.J 积分 = 局_json.GetFloat64("VipNumber")
KuaiYanFull.go:288     global.X 系统信息.Y 邮箱 = string(局_基础信息
                        json.GetStringBytes("Email"))
KuaiYanFull.go:289     global.X 系统信息.D 登录时间 = 局_json.GetInt("LoginTime")
KuaiYanFull.go:290     global.X 系统信息.D 登录 IP = string(局_基础信息
                        json.GetStringBytes("LoginIp"))
KuaiYanFull.go:291     global.X 系统信息.Qq = string(局_基础信息
                        json.GetStringBytes("Qq"))
KuaiYanFull.go:292
KuaiYanFull.go:293     response.OkWithDetailed(gin.H{
KuaiYanFull.go:294         "user": global.X 系统信息.H 会员帐号, "userClassName":
                        global.X 系统信息.Y 用户类型,
KuaiYanFull.go:295         "vipNumber": global.X 系统信息.J 积分, "rmb": global.X 系统信
                        息.Y 余额,
KuaiYanFull.go:296         "vipTime": global.X 系统信息.D 到期时间, "email": global.X 系统
                        信息.Y 邮箱,
KuaiYanFull.go:297         "loginTime": global.X 系统信息.D 登录时间, "loginIp": global.X
                        系统信息.D 登录 IP,
KuaiYanFull.go:298         "qq": global.X 系统信息.Qq, "registerTime": global.X 系统信
                        息.Z 注册时间,
KuaiYanFull.go:299         "appVer": global.X 系统信息.B 版本号当前, "appVerNew":
                        global.X 系统信息.B 版本号最新,
KuaiYanFull.go:300     }, "获取成功", c)

                                第 16 页

KuaiYanFull.go:301     return
KuaiYanFull.go:302 }
KuaiYanFull.go:303

```

```

KuaiYanFull.go:304 // OutLogin 快验注销
KuaiYanFull.go:305 func (k *KuaiYan) OutLogin(c *gin.Context) {
KuaiYanFull.go:306     var 响应信息 string
KuaiYanFull.go:307     if !global.Q 快验.Y 用户登录注销() {
KuaiYanFull.go:308         response.FailWithMessage("注销失败:"+global.Q 快验.Q 取错误信息
KuaiYanFull.go:309         (nil), c)
KuaiYanFull.go:310         return
KuaiYanFull.go:311     }
KuaiYanFull.go:312     global.X 系统信息.K 开启验证码接口列表 = 响应信息
KuaiYanFull.go:313     response.OkWithMessage("注销成功", c)
KuaiYanFull.go:314     return
KuaiYanFull.go:315 }
KuaiYanFull.go:316 // GetIsBuyKaList 取可购买充值卡列表
KuaiYanFull.go:317 func (k *KuaiYan) GetIsBuyKaList(c *gin.Context) {
KuaiYanFull.go:318     var 响应信息 string
KuaiYanFull.go:319     if !global.Q 快验.Q 取可购买卡类列表(&响应信息) {
KuaiYanFull.go:320         response.FailWithMessage(global.Q 快验.Q 取错误信息(nil), c)
KuaiYanFull.go:321         return
KuaiYanFull.go:322     }
KuaiYanFull.go:323     局_json, _ := fastjson.Parse(响应信息)
KuaiYanFull.go:324     局_可买充值卡 := 局_json.GetArray("Data")
KuaiYanFull.go:325     var data = make([]gin.H, len(局_可买充值卡))
KuaiYanFull.go:326     for 索引, _ := range 局_可买充值卡 {
KuaiYanFull.go:327         data[索引] = gin.H{
KuaiYanFull.go:328             "id": 局_可买充值卡[索引].GetInt("Id"), "name": string(局
KuaiYanFull.go:329             _可买充值卡[索引].GetStringBytes("Name")),
KuaiYanFull.go:330             "money": 局_可买充值卡[索引].GetFloat64("Money"),
KuaiYanFull.go:331         }
KuaiYanFull.go:332     }
KuaiYanFull.go:333     response.OkWithDetailed(data, "获取成功", c)
KuaiYanFull.go:334     return
KuaiYanFull.go:335 }
KuaiYanFull.go:336 // GetPurchasedKaList 购买充值卡记录
KuaiYanFull.go:337 func (k *KuaiYan) GetPurchasedKaList(c *gin.Context) {
KuaiYanFull.go:338     var 响应信息 string
KuaiYanFull.go:339     if !global.Q 快验.Q 取已购买卡号列表(&响应信息, 5) {
KuaiYanFull.go:340         response.FailWithMessage(global.Q 快验.Q 取错误信息(nil), c)
KuaiYanFull.go:341         return
KuaiYanFull.go:342     }
KuaiYanFull.go:343     局_json, _ := fastjson.Parse(响应信息)
KuaiYanFull.go:344     局_可买充值卡 := 局_json.GetArray("Data")
KuaiYanFull.go:345     var data = make([]gin.H, len(局_可买充值卡))

```

```

KuaiYanFull.go:346     for 索引, _ := range 局_可买充值卡 {
KuaiYanFull.go:347         data[索引] = gin.H{
KuaiYanFull.go:348             "id": 局_可买充值卡[索引].GetInt("Id"), "name": string(局
_可买充值卡[索引].GetStringBytes("Name")),
KuaiYanFull.go:349             "money": 局_可买充值卡[索引].GetFloat64("Money"), "num":
局_可买充值卡[索引].GetInt("Num"),
KuaiYanFull.go:350             "numMax": 局_可买充值卡[索引].GetInt("NumMax"),
"kaClassId": 局_可买充值卡[索引].GetInt("KaClassId"),
第 17 页
KuaiYanFull.go:351             "kaClassName": string(局_可买充值卡[索
引].GetStringBytes("KaClassName")),
KuaiYanFull.go:352             "registerTime": 局_可买充值卡[索
引].GetInt("RegisterTime"),
KuaiYanFull.go:353         }
KuaiYanFull.go:354     }
KuaiYanFull.go:355     response.OkWithDetailed(data, "获取成功", c)
KuaiYanFull.go:356     return
KuaiYanFull.go:357 }
KuaiYanFull.go:358
KuaiYanFull.go:359 // GetPayStatus 取支付通道状态
KuaiYanFull.go:360 func (k *KuaiYan) GetPayStatus(c *gin.Context) {
KuaiYanFull.go:361     var 响应信息 string
KuaiYanFull.go:362     if !global.Q 快验.Q 余额充值_取支付通道状态(&响应信息) {
KuaiYanFull.go:363         response.FailWithMessage(global.Q 快验.Q 取错误信息(nil), c)
KuaiYanFull.go:364         return
KuaiYanFull.go:365     }
KuaiYanFull.go:366     response.OkWithData(响应信息, c)
KuaiYanFull.go:367     return
KuaiYanFull.go:368 }
KuaiYanFull.go:369
KuaiYanFull.go:370 // PayMoneyToKa 余额购买充值卡
KuaiYanFull.go:371 func (k *KuaiYan) PayMoneyToKa(c *gin.Context) {
KuaiYanFull.go:372     var 请求 结构请求_KaClassId
KuaiYanFull.go:373     err := c.ShouldBindJSON(&请求)
KuaiYanFull.go:374     if err != nil {
KuaiYanFull.go:375         response.FailWithMessage("提交参数错误:"+err.Error(), c)
KuaiYanFull.go:376         return
KuaiYanFull.go:377     }
KuaiYanFull.go:378     var 响应信息 string
KuaiYanFull.go:379     if !global.Q 快验.Y 余额购买充值卡(&响应信息, 请求.KaClassId) {
KuaiYanFull.go:380         response.FailWithMessage(global.Q 快验.Q 取错误信息(nil), c)
KuaiYanFull.go:381         return
KuaiYanFull.go:382     }
KuaiYanFull.go:383     局_json, _ := fastjson.Parse(响应信息)

```

```

KuaiYanFull.go:384         response.OkWithDetailed(gin.H{"kaName": string(局
                           _json.GetStringBytes("KaName"))}, "购买成功", c)
KuaiYanFull.go:385         return
KuaiYanFull.go:386     }
KuaiYanFull.go:387
KuaiYanFull.go:388 // UseKa 卡号充值
KuaiYanFull.go:389 func (k *KuaiYan) UseKa(c *gin.Context) {
KuaiYanFull.go:390     var 请求 结构请求_充值
KuaiYanFull.go:391     err := c.ShouldBindJSON(&请求)
KuaiYanFull.go:392     if err != nil {
KuaiYanFull.go:393         response.FailWithMessage("提交参数错误:"+err.Error(), c)
KuaiYanFull.go:394         return
KuaiYanFull.go:395     }
KuaiYanFull.go:396     if !global.Q 快验.K 卡号充值(global.X 系统信息.H 会员帐号, 请求.Ka,
    请求.InviteUser) {
KuaiYanFull.go:397         response.FailWithMessage(global.Q 快验.Q 取错误信息(nil), c)
KuaiYanFull.go:398         return
KuaiYanFull.go:399     }
KuaiYanFull.go:400     response.OkWithMessage("充值成功", c)

```

第 18 页

```

KuaiYanFull.go:401     return
KuaiYanFull.go:402 }
KuaiYanFull.go:403
KuaiYanFull.go:404 // GetPayPC 余额充值
KuaiYanFull.go:405 func (k *KuaiYan) GetPayPC(c *gin.Context) {
KuaiYanFull.go:406     var 请求 结构请求_余额充值
KuaiYanFull.go:407     err := c.ShouldBindJSON(&请求)
KuaiYanFull.go:408     if err != nil {
KuaiYanFull.go:409         response.FailWithMessage("提交参数错误:"+err.Error(), c)
KuaiYanFull.go:410         return
KuaiYanFull.go:411     }
KuaiYanFull.go:412     var 响应信息 string
KuaiYanFull.go:413     var 订单 ID = 请求.D 订单 ID
KuaiYanFull.go:414     if 订单 ID != "" {
KuaiYanFull.go:415         if !global.Q 快验.D 订单_取状态(&响应信息, 订单 ID) {
KuaiYanFull.go:416             局_错误代号 := 0
KuaiYanFull.go:417             response.FailWithMessage(global.Q 快验.Q 取错误信息(&局_错
    误代号), c)
KuaiYanFull.go:418         } else {
KuaiYanFull.go:419             parse, _ := fastjson.Parse(响应信息)
KuaiYanFull.go:420             response.OkWithData(gin.H{"status":
    parse.GetInt("Status")}, c)
KuaiYanFull.go:421         }
KuaiYanFull.go:422         return
KuaiYanFull.go:423     }

```



```

KuaiYanFull.go:424         if !global.Q 快验.D 订单_购买余额(&响应信息, 请求.Type, global.X 系统
                           信息.H 会员帐号, 请求.C 充值金额, &订单 ID) {
KuaiYanFull.go:425             response.FailWithMessage(global.Q 快验.Q 取错误信息(nil), c)
KuaiYanFull.go:426             return
KuaiYanFull.go:427         }
KuaiYanFull.go:428         var 局_MAP gin.H
KuaiYanFull.go:429         _ = json.Unmarshal([]byte(响应信息), &局_MAP)
KuaiYanFull.go:430         局_MAP["status"] = 1
KuaiYanFull.go:431         response.OkWithData(局_MAP, c)
KuaiYanFull.go:432         return
KuaiYanFull.go:433     }
KuaiYanFull.go:434
KuaiYanFull.go:435 // Updater 更新程序
KuaiYanFull.go:436 func (k *KuaiYan) Updater(c *gin.Context) {
KuaiYanFull.go:437     if KuaiYanUpdater.J_系统更新状态 > 0 {
KuaiYanFull.go:438         response.FailWithMessage("已经在更新中当前进
                           度:"+KuaiYanUpdater.J_系统更新提示, c)
KuaiYanFull.go:439         if KuaiYanUpdater.J_系统更新状态 == 2 {
KuaiYanFull.go:440             KuaiYanUpdater.J_系统更新提示 = ""
KuaiYanFull.go:441             KuaiYanUpdater.J_系统更新状态 = 0
KuaiYanFull.go:442         }
KuaiYanFull.go:443         return
KuaiYanFull.go:444     }
KuaiYanFull.go:445     var 响应信息 string
KuaiYanFull.go:446     if !global.Q 快验.Q 取新版本下载地址(&响应信息) {
KuaiYanFull.go:447         response.FailWithMessage("获取新版本下载地址失败:"+global.Q 快
                           验.Q 取错误信息(nil), c)
KuaiYanFull.go:448         return
KuaiYanFull.go:449     }
KuaiYanFull.go:450     if !utils2.X 系统_权限检测() {
                           第 19 页
KuaiYanFull.go:451         response.FailWithMessage("系统无写入文件权限,请检查权限是否 777
                           或 755,运行用户是否为 root", c)
KuaiYanFull.go:452         return
KuaiYanFull.go:453     }
KuaiYanFull.go:454     KuaiYanUpdater.J_系统更新提示 = "准备中"
KuaiYanFull.go:455     go KuaiYanUpdater.K 快验系统开始更新(响应信息, func(执行程序本地路径
                           string) {
KuaiYanFull.go:456         fmt.Printf("系统更新%v,马上启动新版本程序继承端口,退出旧版本
                           \n\n", 执行程序本地路径)
KuaiYanFull.go:457         global.GVA_LOG.Info("系统更新,马上启动新版本程序继承端口,退出旧
                           版本\n")
KuaiYanFull.go:458         err := os.Chmod(执行程序本地路径, 0755)
KuaiYanFull.go:459         if err != nil {

```

```

KuaiYanFull.go:460             KuaiYanUpdater.J_系统更新状态 = 2
KuaiYanFull.go:461             KuaiYanUpdater.J_系统更新提示 = fmt.Sprintf("启动修改下载
文件权限失败:%v\n", err.Error())
KuaiYanFull.go:462             return
KuaiYanFull.go:463         }
KuaiYanFull.go:464         cmd := exec.Command(执行程序本地路径)
KuaiYanFull.go:465         cmd.Stdout = os.Stdout
KuaiYanFull.go:466         cmd.Stderr = os.Stderr
KuaiYanFull.go:467         cmd.Stdin = os.Stdin
KuaiYanFull.go:468         if err = cmd.Start(); err != nil {
KuaiYanFull.go:469             KuaiYanUpdater.J_系统更新状态 = 2
KuaiYanFull.go:470             KuaiYanUpdater.J_系统更新提示 = fmt.Sprintf("启动新更新后
程序失败:%v\n", err.Error())
KuaiYanFull.go:471             return
KuaiYanFull.go:472         }
KuaiYanFull.go:473         err = global.GVA_Gin.Shutdown(context.Background())
KuaiYanFull.go:474         if err != nil {
KuaiYanFull.go:475             KuaiYanUpdater.J_系统更新状态 = 2
KuaiYanFull.go:476             KuaiYanUpdater.J_系统更新提示 = "下载文件成功,重启失败,请手
动重启系统:" + err.Error()
KuaiYanFull.go:477             return
KuaiYanFull.go:478         }
KuaiYanFull.go:479         os.Exit(0)
KuaiYanFull.go:480     })
KuaiYanFull.go:481     var filename string
KuaiYanFull.go:482     filename = path.Base(os.Args[0])
KuaiYanFull.go:483     response.OkWithMessage("开始更新,当前:"+filename, c)
KuaiYanFull.go:484     return
KuaiYanFull.go:485 }
KuaiYanFull.go:486
KuaiYanFull.go:487 // K 快验心跳 内部函数 - 保留原有心跳逻辑
KuaiYanFull.go:488 var 心跳容错计数 = 0
KuaiYanFull.go:489
KuaiYanFull.go:490 func K 快验心跳() {
KuaiYanFull.go:491     if global.Q 快验.J_Token == "" {
KuaiYanFull.go:492         return
KuaiYanFull.go:493     }
KuaiYanFull.go:494     var 响应信息 string
KuaiYanFull.go:495     var 当前状态 int
KuaiYanFull.go:496     if !global.Q 快验.X 心跳(&响应信息, &当前状态) {
KuaiYanFull.go:497         心跳容错计数++
KuaiYanFull.go:498         if 心跳容错计数 >= 3 {
KuaiYanFull.go:499             global.Q 快验.J_Token = ""
KuaiYanFull.go:500             global.X 系统信息.H 会员帐号 = ""

```

```

KuaiYanFull.go:501             global.X 系统信息.D 到期时间 = 0
KuaiYanFull.go:502             global.X 系统信息.Y 用户类型代号 = 0
KuaiYanFull.go:503         }
KuaiYanFull.go:504         return
KuaiYanFull.go:505     }
KuaiYanFull.go:506     心跳容错计数 = 0

KuaiYanFull.go:507
KuaiYanFull.go:508     局_设备信息, err := getServerInfoForKuaiYan()
KuaiYanFull.go:509     if err != nil {
KuaiYanFull.go:510         return
KuaiYanFull.go:511     }
KuaiYanFull.go:512     局_动态标记 := fmt.Sprintf("%s %dH%.2fG %dG %d 协程,用户数:%d,卡总
数:%d,在线数:%d",
KuaiYanFull.go:513         utils.S 三元(global.Q 快验.J 集_连接方式 == 0, "直连", "网关"),
KuaiYanFull.go:514         runtime.NumCPU(),
KuaiYanFull.go:515         utils.Float64 除 int64(utils.Int64 到 Float64(int64(局_设备信
息.Ram.TotalMB)), 1024, 2),
KuaiYanFull.go:516         局_设备信息.Disk.TotalGB,
KuaiYanFull.go:517         runtime.NumGoroutine(),
KuaiYanFull.go:518         Ser_User.Q 取总数(),
KuaiYanFull.go:519         Ser_Ka.Q 取总数(),
KuaiYanFull.go:520         Ser_LinkUser.Get 取在线总数(true, true),
KuaiYanFull.go:521     )
KuaiYanFull.go:522     if 局_设备信息.Os.GOOS != "linux" {
KuaiYanFull.go:523         局_动态标记 += " " + 局_设备信息.Os.GOOS
KuaiYanFull.go:524     }
KuaiYanFull.go:525     global.Q 快验.Z 置动态标记(局_动态标记)
KuaiYanFull.go:526 }
KuaiYanFull.go:527
KuaiYanFull.go:528 func getServerInfoForKuaiYan() (*utils2.Server, error) {
KuaiYanFull.go:529     return getServerInfo()
KuaiYanFull.go:530 }

VMP.go:1     package VMP
VMP.go:2
VMP.go:3     import (
VMP.go:4         "crypto/sha1"
VMP.go:5         "encoding/base64"
VMP.go:6         "errors"
VMP.go:7         "github.com/gin-gonic/gin"
VMP.go:8         "math/big"
VMP.go:9         "math/rand"
VMP.go:10        "server/new/app/models/common"
VMP.go:11        "time"
VMP.go:12    )
VMP.go:13
VMP.go:14    var L_VMP VMP
VMP.go:15
VMP.go:16    func init() {

```

```
VMP.go:17         L_VMP = VMP{}
VMP.go:18
VMP.go:19     }
VMP.go:20
```

第 21 页

```
VMP.go:21     type VMP struct {
VMP.go:22     }
VMP.go:23
VMP.go:24     // nBits Rsa 位数 1024
VMP.go:25     // vPrivate RsaBase64 私钥
VMP.go:26     // vModulus RsaBase64 模数
VMP.go:27     // vProductCode base64 产品代码
VMP.go:28     // /耗时: 7 ms 实测速度极快,万次才7 毫秒 测试配置 16H40G
VMP.go:29     func (j *VMP) J 计算授权码(c *gin.Context, VmpRsa common.VmpRsa, 授权信
息 common.VmpParams) (授权码 string, err error) {
VMP.go:30         exported := struct {
VMP.go:31             Algorithm string
VMP.go:32             Bits      int
VMP.go:33             PrivateKey string
VMP.go:34             Modulus      string
VMP.go:35             ProductCode string
VMP.go:36         }{
VMP.go:37             Algorithm: "RSA",
VMP.go:38             Bits:       VmpRsa.Rsa 位数,
VMP.go:39             PrivateKey: VmpRsa.RsaBase64 私钥,
VMP.go:40             Modulus:    VmpRsa.RsaBase64 模数,
VMP.go:41             ProductCode: VmpRsa.Base64 产品代码,
VMP.go:42         }
VMP.go:43
VMP.go:44         serialData, err2 := 打包数据(授权信息, exported.ProductCode)
VMP.go:45         if err2 != nil {
VMP.go:46             err = err2
VMP.go:47             return
VMP.go:48         }
VMP.go:49         serialBin := serialData
VMP.go:50
VMP.go:51         hash := sha1.Sum(serialBin)
VMP.go:52         serialBin = append(serialBin, 0xFF)
VMP.go:53         for i := 3; i >= 0; i-- {
VMP.go:54             serialBin = append(serialBin, hash[i])
VMP.go:55         }
VMP.go:56
VMP.go:57         // 生成 8 到 12 之间的随机整数
VMP.go:58         //$size = rand(8, 16);
VMP.go:59         //for ($i = 0; $i < $size; $i++) $padding_front[] = rand(1,
255);
VMP.go:60         // 改为固定值 (示例用全 0 填充)
VMP.go:61         paddingFront := make([]byte, 8)
```

```

VMP.go:62         for i := range paddingFront {
VMP.go:63             paddingFront[i] = byte(rand.Int31n(255) + 1)
VMP.go:64         }
VMP.go:65
VMP.go:66         paddingFront = append([]byte{0, 2}, paddingFront...)
VMP.go:67         paddingFront = append(paddingFront, 0)
VMP.go:68
VMP.go:69         contentType := len(serialBin) + len(paddingFront)
VMP.go:70         rest := exported.Bits/8 - contentType

                                第 22 页

VMP.go:71         if rest < 0 {
VMP.go:72             err = errors.New("内容太大")
VMP.go:73             return
VMP.go:74         }
VMP.go:75
VMP.go:76         paddingBack := make([]byte, rest)
VMP.go:77         finalSerial := append(paddingFront, serialBin...)
VMP.go:78         finalSerial = append(finalSerial, paddingBack...)
VMP.go:79
VMP.go:80         n, _ := base64.StdEncoding.DecodeString(exported.Modulus)
VMP.go:81         d, _ := base64.StdEncoding.DecodeString(exported.PrivateKey)
VMP.go:82
VMP.go:83         encoded := S十进制编码(finalSerial)
VMP.go:84         modulus := S十进制编码(n)
VMP.go:85         private := S十进制编码(d)
VMP.go:86
VMP.go:87         res := new(big.Int).Exp(encoded, private, modulus) // 模幂运算
VMP.go:88         resBytes := S十进制解码(res)
VMP.go:89         授权码 = base64.StdEncoding.EncodeToString(resBytes)
VMP.go:90
VMP.go:91         return
VMP.go:92     }
VMP.go:93
VMP.go:94     // 辅助函数
VMP.go:95     func S十进制编码(data []byte) *big.Int {
VMP.go:96         result := big.NewInt(0)
VMP.go:97         for _, b := range data {
VMP.go:98             result.Mul(result, big.NewInt(256))
VMP.go:99             result.Add(result, big.NewInt(int64(b)))
VMP.go:100         }
VMP.go:101         return result
VMP.go:102     }
VMP.go:103
VMP.go:104     func S十进制解码(n *big.Int) []byte {
VMP.go:105         result := []byte{}
VMP.go:106         zero := big.NewInt(0)
VMP.go:107         twoFiftySix := big.NewInt(256)
VMP.go:108
VMP.go:109         for n.Cmp(zero) > 0 {

```

```

VMP.go:110             mod := new(big.Int)
VMP.go:111             mod.Mod(n, twoFiftySix)
VMP.go:112             result = append([]byte{byte(mod.Int64())}, result...)
VMP.go:113             n.Div(n, twoFiftySix)
VMP.go:114         }
VMP.go:115         return result
VMP.go:116     }
VMP.go:117
VMP.go:118     func 打包数据(params common.VmpParams, productCode string) (结果数据
[]byte, err error) {
VMP.go:119         serial := []byte{1, 1} // Version
VMP.go:120         // 用户名

第 23 页
VMP.go:121         if len(params.UserName) > 255 {
VMP.go:122             err = errors.New("用户名太长")
VMP.go:123             return
VMP.go:124         }
VMP.go:125         serial = append(serial, 2, byte(len(params.UserName)))
VMP.go:126         serial = append(serial, []byte(params.UserName)...)
VMP.go:127
VMP.go:128         // Email
VMP.go:129         if len(params.Email) > 255 {
VMP.go:130             err = errors.New("E-Mail 太长")
VMP.go:131             return
VMP.go:132         }
VMP.go:133         serial = append(serial, 3, byte(len(params.Email)))
VMP.go:134         serial = append(serial, []byte(params.Email)...)
VMP.go:135
VMP.go:136         // Hardware ID
VMP.go:137         hwid, _ := base64.StdEncoding.DecodeString(params.Hwid)
VMP.go:138         if len(hwid) == 0 || len(hwid) > 255 || len(hwid)%4 != 0 {
VMP.go:139             err = errors.New("机器码 无效")
VMP.go:140             return
VMP.go:141         }
VMP.go:142         serial = append(serial, 4, byte(len(hwid)))
VMP.go:143         serial = append(serial, hwid...)
VMP.go:144
VMP.go:145         // Expire Date
VMP.go:146         serial = append(serial, 5)
VMP.go:147         serial = appendDate(serial, params.ExpireDate)
VMP.go:148
VMP.go:149         // Time Limit
VMP.go:150         if params.TimeLimit < 0 || params.TimeLimit > 255 {
VMP.go:151             err = errors.New("无效的时限")
VMP.go:152             return
VMP.go:153         }
VMP.go:154         serial = append(serial, 6, byte(params.TimeLimit))
VMP.go:155
VMP.go:156         // Product Code
VMP.go:157         pc, _ := base64.StdEncoding.DecodeString(productCode)
VMP.go:158         if len(pc) != 8 {

```

```

VMP.go:159         err = errors.New("产品代码无效")
VMP.go:160         return
VMP.go:161     }
VMP.go:162     serial = append(serial, 7)
VMP.go:163     serial = append(serial, pc...)
VMP.go:164
VMP.go:165     // User Data
VMP.go:166     if len(params.UserData) > 255 {
VMP.go:167         err = errors.New("用户数据过长")
VMP.go:168         return
VMP.go:169     }
VMP.go:170     serial = append(serial, 8, byte(len(params.UserData)))
    第 24 页
VMP.go:171     serial = append(serial, params.UserData...)
VMP.go:172
VMP.go:173     // Max Build Date
VMP.go:174     serial = append(serial, 9)
VMP.go:175     serial = appendDate(serial, params.MaxBuildDate)
VMP.go:176     结果数据 = serial
VMP.go:177     return
VMP.go:178 }
VMP.go:179
VMP.go:180 func appendDate(serial []byte, a common.S 时间) []byte {
VMP.go:181     var t time.Time
VMP.go:182     t = time.Date(a.Year, time.Month(a.Month), a.Day, 0, 0, 0, 0,
time.UTC)
VMP.go:183     year, month, day := t.Date()
VMP.go:184     return append(serial, byte(day), byte(month), byte(year%256),
byte(year/256))
VMP.go:185
VMP.go:186 }
rmbPay.go:1 package rmbPay
rmbPay.go:2
rmbPay.go:3 import (
rmbPay.go:4     . "EFunc/utils"
rmbPay.go:5     "encoding/json"
rmbPay.go:6     "errors"
rmbPay.go:7     "fmt"
rmbPay.go:8     "github.com/gin-gonic/gin"
rmbPay.go:9     "github.com/gogf/gf/v2/encoding/gjson"
rmbPay.go:10    "go.uber.org/zap"
rmbPay.go:11    "gorm.io/gorm"
rmbPay.go:12    "gorm.io/gorm/clause"
rmbPay.go:13    App 服务 "server/Service/Ser_AppInfo"
rmbPay.go:14    "server/global"
rmbPay.go:15    "server/new/app/logic/agent/L_setting"
rmbPay.go:16    "server/new/app/logic/common/agent"
rmbPay.go:17    "server/new/app/logic/common/ka"
rmbPay.go:18    "server/new/app/logic/common/log"
rmbPay.go:19    "server/new/app/logic/common/setting"
rmbPay.go:20    "server/new/app/logic/webUser/cpsPayOrder"

```

```

rmbPay.go:21         m "server/new/app/models/common"
rmbPay.go:22         "server/new/app/models/constant"
rmbPay.go:23         dbm "server/new/app/models/db"
rmbPay.go:24         "server/new/app/service"
rmbPay.go:25         DB "server/structs/db"
rmbPay.go:26         "server/utils/Qqwry"
rmbPay.go:27         "strconv"
rmbPay.go:28         "sync"
rmbPay.go:29         "time"
rmbPay.go:30     )
rmbPay.go:31
rmbPay.go:32     var L_rmbPay rmbPay
rmbPay.go:33
rmbPay.go:34     func init() {
第 25 页
rmbPay.go:35         L_rmbPay = rmbPay{
rmbPay.go:36             已注册通道: make(map[string]RmbPayItem, 10),
rmbPay.go:37             Map 订单类型: map[int]string{
rmbPay.go:38                 constant.D 订单类型_余额充值: "余额充值",
rmbPay.go:39                 constant.D 订单类型_购卡直冲: "购卡直冲",
rmbPay.go:40                 constant.D 订单类型_支付购卡: "支付购卡",
rmbPay.go:41             },
rmbPay.go:42         }
rmbPay.go:43
rmbPay.go:44     }
rmbPay.go:45
rmbPay.go:46     type rmbPay struct {
rmbPay.go:47         // 逻辑中使用的某个变量
rmbPay.go:48         订单号计数 int
rmbPay.go:49         订单号初始值 int
rmbPay.go:50         订单号时间戳 int64
rmbPay.go:51         // 与变量对应的使用互斥锁
rmbPay.go:52         锁 sync.Mutex
rmbPay.go:53         已注册通道 map[string]RmbPayItem
rmbPay.go:54         Map 订单类型 map[int]string
rmbPay.go:55     }
rmbPay.go:56
rmbPay.go:57     // 注册通道接口
rmbPay.go:58     type RmbPayItem interface {
rmbPay.go:59         D 订单创建(c *gin.Context, 参数 *m.PayParams) (req m.Request, err error)
rmbPay.go:60         D 订单退款(c *gin.Context, 参数 *m.PayParams) (err error)
rmbPay.go:61         D 订单支付回调(c *gin.Context, 参数 *m.PayParams) (响应信息 string, 响应代码 int, err error)
rmbPay.go:62         D 订单退款回调(c *gin.Context, 参数 *m.PayParams) (响应信息 string, 响应代码 int, err error)

```



```

rmbPay.go:63         Q取通道名称() string
rmbPay.go:64         Q取订单 id(c *gin.Context, 参数 *m.PayParams) string
rmbPay.go:65     }
rmbPay.go:66
rmbPay.go:67     func (j *rmbPay) Z注册接口(通道 RmbPayItem) {
rmbPay.go:68         j.已注册通道[通道.Q取通道名称()] = 通道
rmbPay.go:69     }
rmbPay.go:70
rmbPay.go:71     func (j *rmbPay) D订单创建(c *gin.Context, 参数 m.PayParams) (req
m.Request, err error) {

rmbPay.go:72
rmbPay.go:73         参数.Z支付配置 s = setting.Q在线支付配置()
rmbPay.go:74         参数.Z支付配置, _ = json.Marshal(&参数.Z支付配置 s)
rmbPay.go:75
rmbPay.go:76         参数.Type = j.Pay_显示名称转原名(参数.Type)
rmbPay.go:77
rmbPay.go:78         局_通道, ok := j.已注册通道[参数.Type]
rmbPay.go:79         if !ok {
rmbPay.go:80             err = errors.New("支付方式未配置")
rmbPay.go:81             return
rmbPay.go:82         }
rmbPay.go:83
rmbPay.go:84         if 参数.Rmb <= 0 {

第 26 页

rmbPay.go:85             err = errors.New("支付金额必须大于 0")
rmbPay.go:86             return
rmbPay.go:87         }
rmbPay.go:88         参数.PayOrder = j.获取新订单号()
rmbPay.go:89
rmbPay.go:90         参数.Y异步回调地址 = setting.Q系统设置().X系统地址 +
"/webApi/payNotify/" + 参数.PayOrder

rmbPay.go:91
rmbPay.go:92         switch 参数.UidType {
rmbPay.go:93         default:
rmbPay.go:94             err = errors.New("UidType 错误")
rmbPay.go:95             return
rmbPay.go:96         case 1:
rmbPay.go:97             tx := *global.GVA_DB
rmbPay.go:98             if info, err2 := service.NewUser(c, &tx).Info(参数.Uid);
err2 == nil {
rmbPay.go:99                 参数.User = info.User
rmbPay.go:100             }
rmbPay.go:101         case 2:
rmbPay.go:102             tx := *global.GVA_DB
rmbPay.go:103             if info, err2 := service.NewKa(c, &tx).Info(参数.Uid); err2
== nil {

```

```

rmbPay.go:104             参数.User = info.Name
rmbPay.go:105         }
rmbPay.go:106     }
rmbPay.go:107
rmbPay.go:108     if 参数.User == "" {
rmbPay.go:109         if 参数.ProcessingType == constant.D 订单类型_余额充值 || 参
数.ProcessingType == constant.D 订单类型_购卡直冲 {
rmbPay.go:110             err = errors.New("用户名不能为空")
rmbPay.go:111             return
rmbPay.go:112         }
rmbPay.go:113     }
rmbPay.go:114     tx := *global.GVA_DB
rmbPay.go:115     var 局_通道数据 m.Request
rmbPay.go:116
rmbPay.go:117     参数.S 商品名称 = App 服务.AppId 取应用名称(参数.E 额外信
息.Get("AppId").Int()) + j.Q 取提示信息(&参数)
rmbPay.go:118
rmbPay.go:119     if 参数.ReceivedUid > 0 && agent.L_agent.Id 功能权限检测(c, 参
数.ReceivedUid, DB.D 代理功能_代收款) {
rmbPay.go:120         var 局代理 Info DB.DB_User
rmbPay.go:121         var 代理在线支付信息 m.Z 在线支付
rmbPay.go:122         if 局代理 Info, err = service.NewUser(c, &tx).Info(参
数.ReceivedUid); err == nil {
rmbPay.go:123             if 代理在线支付信息, err = L_setting.Q 取代理在线支付信息
(c, 参数.ReceivedUid); err == nil {
rmbPay.go:124                 局_代收款金额 := 参数.Rmb + L_rmbPay.Pay_指定 Uid 待支
付金额(c, 参数.ReceivedUid)
rmbPay.go:125                 if 局代理 Info.Rmb > 局_代收款金额 {
rmbPay.go:126                     参数.Z 支付配置 s = 代理在线支付信息
rmbPay.go:127                     参数.Z 支付配置, _ = json.Marshal(&参数.Z 支付配
置 s)
rmbPay.go:128                     局_通道数据, err = 局_通道.D 订单创建(c, &参数)
rmbPay.go:129                     global.GVA_LOG.Error(参数.PayOrder+"代付订单创建
失败", zap.Error(err))
rmbPay.go:130                     if err == nil {
rmbPay.go:131                         goto 下单成功
rmbPay.go:132                     }
rmbPay.go:133                     } else {
rmbPay.go:134                         err = errors.New("代理余额不足(" + Float64 到文
本(局_代收款金额, 2) + "=未关闭代收款订单总额)")
第 27 页
rmbPay.go:135                     }
rmbPay.go:136                 }
rmbPay.go:137             }
rmbPay.go:138         }

```

```

rmbPay.go:139          参数.Z 支付配置 s = setting.Q 在线支付配置()
rmbPay.go:140          参数.Z 支付配置, _ = json.Marshal(&参数.Z 支付配置 s)
rmbPay.go:141          参数.ReceivedUid = 0
rmbPay.go:142          if err != nil {
rmbPay.go:143              参数.E 额外信息.Set("代收款 err", err.Error())
rmbPay.go:144          }
rmbPay.go:145      } else {
rmbPay.go:146          参数.ReceivedUid = 0
rmbPay.go:147      }
rmbPay.go:148
rmbPay.go:149      局_通道数据, err = 局_通道.D 订单创建(c, &参数)
rmbPay.go:150      if err != nil {
rmbPay.go:151          return
rmbPay.go:152      }
rmbPay.go:153      下单成功:
rmbPay.go:154          参数.Status = constant.D 订单状态_等待支付
rmbPay.go:155          参数.Time = time.Now().Unix()
rmbPay.go:156          参数.Ip = c.ClientIP()
rmbPay.go:157          参数.Extra = "{}"
rmbPay.go:158          if 字节数组, err2 := json.Marshal(参数.E 额外信息); err2 == nil {
rmbPay.go:159              参数.Extra = string(字节数组)
rmbPay.go:160          }
rmbPay.go:161
rmbPay.go:162          s := service.NewRmbPayService(&tx)
rmbPay.go:163          _, err = s.Create(参数.DB_LogRMBPayOrder)
rmbPay.go:164          if err != nil {
rmbPay.go:165              return
rmbPay.go:166          }
rmbPay.go:167          req = 局_通道数据
rmbPay.go:168          //余额支付单独处理,直接回调
rmbPay.go:169          if 局_通道.Q 取通道名称() == "余额支付" {
rmbPay.go:170              c.Set("余额支付 payOrder", 参数.PayOrder)
rmbPay.go:171              响应信息, 响应代码 := j.D 订单回调(c)
rmbPay.go:172              fmt.Print(响应信息, 响应代码)
rmbPay.go:173          }
rmbPay.go:174
rmbPay.go:175          return
rmbPay.go:176      }
rmbPay.go:177
rmbPay.go:178      func (j *rmbPay) D 订单退款(c *gin.Context, 参数 m.PayParams, 追回资产
bool, 备注 string) (err error) {
rmbPay.go:179          var info struct {
rmbPay.go:180              user          DB.DB_User
rmbPay.go:181              Agent           DB.DB_User
rmbPay.go:182              LogMoney          []DB.DB_LogMoney

```

```

rmbPay.go:183             LogVipNumber []DB.DB_LogVipNumber
rmbPay.go:184             卡类详情             dbm.DB_KaClass

                                第 28 页

rmbPay.go:185             软件用户详情             DB.DB_AppUser
rmbPay.go:186             app 详情             DB.DB_AppInfo
rmbPay.go:187         }
rmbPay.go:188
rmbPay.go:189         db := *global.GVA_DB
rmbPay.go:190         if 参数.DB_LogRMBPayOrder, err =
service.NewRmbPayService(&db).Info2(map[string]interface{}{"PayOrder":
参数.PayOrder}); err != nil {

rmbPay.go:191             err = errors.New("订单不存在")
rmbPay.go:192             return
rmbPay.go:193         }
rmbPay.go:194
rmbPay.go:195         if 参数.Status != constant.D 订单状态_成功 {
rmbPay.go:196             err = errors.New("仅成功状态订单可操作支持退款")
rmbPay.go:197             return
rmbPay.go:198         }
rmbPay.go:199
rmbPay.go:200         //禁止退款走管理员设置
rmbPay.go:201         参数.Z 支付配置 s = setting.Q 在线支付配置()
rmbPay.go:202         参数.Z 支付配置, _ = json.Marshal(&参数.Z 支付配置 s)
rmbPay.go:203         参数.E 额外信息, _ = gjson.LoadJson(参数.Extra)
rmbPay.go:204         if 参数.Z 支付配置 s.J 禁止退款 {
rmbPay.go:205             err = errors.New("已禁止退款,请手动前往服务器数据库,修改配置信息
文件 禁止退款:true")
rmbPay.go:206             return
rmbPay.go:207         }
rmbPay.go:208         //判断是否为代收款如果是代收款读取代收用户 id
rmbPay.go:209         if 参数.ReceivedUid > 0 {
rmbPay.go:210             参数.Z 支付配置 s = m.Z 在线支付{} //重新清零数据防止下边读取失败,
依然使用系统配置
rmbPay.go:211             if 参数.Z 支付配置 s, err = L_setting.Q 取代理在线支付信息(c, 参
数.ReceivedUid); err == nil {
rmbPay.go:212                 参数.Z 支付配置, _ = json.Marshal(&参数.Z 支付配置 s)
rmbPay.go:213             }
rmbPay.go:214         }
rmbPay.go:215
rmbPay.go:216         局_通道, ok := j.已注册通道[参数.Type]
rmbPay.go:217         if !ok {
rmbPay.go:218             err = errors.New("支付方式未配置")
rmbPay.go:219             return
rmbPay.go:220         }
rmbPay.go:221

```

```

rmbPay.go:222         err = db.Transaction(func(tx *gorm.DB) (err error) {
rmbPay.go:223
rmbPay.go:224             //加锁重新查
rmbPay.go:225             err =
tx.Model(DB.DB_LogRMBPayOrder{}).Clauses(clause.Locking{Strength:
"UPDATE"}).Where("Id = ?", 参数.Id).First(&参
数.DB_LogRMBPayOrder).Error
rmbPay.go:226             if err != nil {
rmbPay.go:227                 return errors.New("订单不存在")
rmbPay.go:228             }
rmbPay.go:229             if 参数.Status != constant.D 订单状态_成功 { //重新确认一次订单
状态
rmbPay.go:230                 err = errors.New("仅成功状态订单可操作支持退款")
rmbPay.go:231                 return
rmbPay.go:232             }
rmbPay.go:233             参数.Status = constant.D 订单状态_退款中
rmbPay.go:234

```

第 29 页

```

rmbPay.go:235             if 追回资产 && 参数.ProcessingType == constant.D 订单类型_余额
充值 { //追回余额
rmbPay.go:236                 if 参数.UidType == 1 { //只有账号模式的应用,才能扣余额
rmbPay.go:237                     err =
tx.Model(DB.DB_User{}).Clauses(clause.Locking{Strength:
"UPDATE"}).Where("Id = ?", 参数.Uid).First(&info.user).Error
rmbPay.go:238                     if err != nil {
rmbPay.go:239                         return errors.New("用户不存在,无法减余额")
rmbPay.go:240                     }
rmbPay.go:241                     err = tx.Model(DB.DB_User{}).Where("Id = ?", 参
数.Uid).Update("Rmb", gorm.Expr("RMB - ?", 参数.Rmb)).Error
rmbPay.go:242                     if err != nil {
rmbPay.go:243                         return errors.New("减余额失败")
rmbPay.go:244                     }
rmbPay.go:245                     info.LogMoney = append(info.LogMoney,
DB.DB_LogMoney{
rmbPay.go:246                         User:  info.user.User,
rmbPay.go:247                         Time:  time.Now().Unix(),
rmbPay.go:248                         Ip:    c.ClientIP(),
rmbPay.go:249                         Count: Float64 取负值(参数.Rmb),
rmbPay.go:250                         Note:  fmt.Sprintf("管理员操作退款,订单:%s,扣除
用户余额%s|新余额=%s", 参数.PayOrder, Float64 到文本(参数.Rmb, 2),
Float64 到文本(info.user.Rmb, 2)),
rmbPay.go:251                     })
rmbPay.go:252                 }
rmbPay.go:253             }
rmbPay.go:254             if 追回资产 && 参数.ProcessingType == constant.D 订单类型_购卡
直冲 { //购卡直冲 追回卡类时间余额 积分

```

```

rmbPay.go:255             info.卡类详情, err = service.NewKaClass(c, tx).Info(参
数.E 额外信息.Get("KaClassId").Int())
rmbPay.go:256             if err != nil {
rmbPay.go:257                 err = errors.New("追回资产时,发现卡类 id" + 参数.E 额
外信息.Get("KaClassId").String() + "已不存在")
rmbPay.go:258                 return
rmbPay.go:259             }
rmbPay.go:260             if 参数.UidType == 1 && info.卡类详情.Rmb != 0 { //只有账
号模式的应用,才能扣余额
rmbPay.go:261                 err =
tx.Model(DB.DB_User{}).Clauses(clause.Locking{Strength:
"UPDATE"}).Where("Id = ?", 参数.Uid).First(&info.user).Error
rmbPay.go:262                 if err != nil {
rmbPay.go:263                     return errors.New("追回资产时,发现用户不存在,无法
减余额")
rmbPay.go:264                 }
rmbPay.go:265                 err = tx.Model(DB.DB_User{}).Where("Id = ?", 参
数.Uid).Update("Rmb", gorm.Expr("RMB - ?", 参数.Rmb)).Error
rmbPay.go:266                 if err != nil {
rmbPay.go:267                     return errors.New("减余额失败")
rmbPay.go:268                 }
rmbPay.go:269                 info.LogMoney = append(info.LogMoney,
DB.DB_LogMoney{
rmbPay.go:270                     User:  info.user.User,
rmbPay.go:271                     Time:  time.Now().Unix(),
rmbPay.go:272                     Ip:    c.ClientIP(),
rmbPay.go:273                     Count: Float64 取负值(参数.Rmb),
rmbPay.go:274                     Note:  fmt.Sprintf("管理员操作退款,订单:%s,扣除
用户余额%s|新余额=%s", 参数.PayOrder, Float64 到文本(参数.Rmb, 2),
Float64 到文本(info.user.Rmb, 2)),
rmbPay.go:275                     })
rmbPay.go:276                 }
rmbPay.go:277
rmbPay.go:278                 err = tx.Model(DB.DB_AppUser{}).Table("db_AppUser_"+参
数.E 额外信息.Get("AppId").String()).Clauses(clause.Locking{Strength:
"UPDATE"}).Where("Uid = ?", 参数.E 额外信
息.Get("AppUserUid").Int()).First(&info.软件用户详情).Error
rmbPay.go:279                 if err != nil {
rmbPay.go:280                     return errors.New("应用:" + 参数.E 额外信
息.Get("AppId").String() + "软件用户 id" + 参数.E 额外信
息.Get("AppUserUid").String() + "已不存在")
rmbPay.go:281                 }
rmbPay.go:282                 info.软件用户详情.VipTime -= info.卡类详情.VipTime
rmbPay.go:283                 info.软件用户详情.VipNumber -= info.卡类详情.VipNumber
rmbPay.go:284                 _, err = service.NewAppUser(c, tx, 参数.E 额外信
息.Get("AppId").Int()).UpdateUid(info.软件用户详情.Uid,
map[string]interface{}{

```

第 30 页

```

rmbPay.go:285         "VipTime":  info.软件用户详情.VipTime,
rmbPay.go:286         "VipNumber": info.软件用户详情.VipNumber,
rmbPay.go:287     })
rmbPay.go:288     if err != nil {
rmbPay.go:289         return err
rmbPay.go:290     }
rmbPay.go:291     if info.卡类详情.VipTime != 0 {
rmbPay.go:292         局_is 计点 := App 服务.App 是否为计点(参数.E 额外信
息.Get("AppId").Int())
rmbPay.go:293         info.LogVipNumber = append(info.LogVipNumber,
DB.DB_LogVipNumber{
rmbPay.go:294             User:  参数.User,
rmbPay.go:295             AppId: 参数.E 额外信息.Get("AppId").Int(),
rmbPay.go:296             Type:   S 三元(局_is 计点, constant.Log_type_点数,
constant.Log_type_时间),
rmbPay.go:297             Time:  time.Now().Unix(),
rmbPay.go:298             Ip:    c.ClientIP(),
rmbPay.go:299             Count: Float64 取负值(Int64 到 Float64(info.卡类
详情.VipTime)),
rmbPay.go:300             Note:  fmt.Sprintf("管理员操作退款,订单:%s,扣除
软件用户"+S 三元(局_is 计点, "点数", "会员时间"), 参数.PayOrder),
rmbPay.go:301         })
rmbPay.go:302     }
rmbPay.go:303     if info.卡类详情.VipNumber != 0 {
rmbPay.go:304         info.LogVipNumber = append(info.LogVipNumber,
DB.DB_LogVipNumber{
rmbPay.go:305             User:  参数.User,
rmbPay.go:306             AppId: 参数.E 额外信息.Get("AppId").Int(),
rmbPay.go:307             Type:  constant.Log_type_积分,
rmbPay.go:308             Time:  time.Now().Unix(),
rmbPay.go:309             Ip:    c.ClientIP(),
rmbPay.go:310             Count: Float64 取负值(info.卡类详情.VipNumber),
rmbPay.go:311             Note:  fmt.Sprintf("管理员操作退款,订单:%s,扣除
软件用户积分|新积分=%s", 参数.PayOrder, Float64 到文本(info.软件用户详
情.VipNumber, 2)),
rmbPay.go:312         })
rmbPay.go:313     }
rmbPay.go:314 }
rmbPay.go:315
rmbPay.go:316     if 追回资产 && 参数.ProcessingType == constant.D 订单类型_支付
购卡 { //追回积分
rmbPay.go:317         c.Set("tx", tx)
rmbPay.go:318         err = ka.L_ka.K 卡号追回(c, 参数.E 额外信
息.Get("Id").Int(), c.GetString("User"))
rmbPay.go:319         if err != nil {

```

```

rmbPay.go:320             return err
rmbPay.go:321         }
rmbPay.go:322         if 局_临时, ok2 := c.Get("info.LogVipNumber"); ok2 {
rmbPay.go:323             info.LogVipNumber = append(info.LogVipNumber, 局_临时.([]DB.DB_LogVipNumber)...)
rmbPay.go:324         }
rmbPay.go:325         if 局_临时, ok2 := c.Get("info.LogMoney"); ok2 {
rmbPay.go:326             info.LogMoney = append(info.LogMoney, 局_临时.([]DB.DB_LogMoney)...)
rmbPay.go:327         }
rmbPay.go:328     }
rmbPay.go:329     //判断是否有代理分成,如果有代理分成,对应扣除
rmbPay.go:330     for 索引 := range 参数.E 额外信息.Len("分成详细") {
rmbPay.go:331         局_uid := 参数.E 额外信息.Get("分成详细." +
strconv.Itoa(索引) + ".Uid").Int()
rmbPay.go:332         局_金额 := 参数.E 额外信息.Get("分成详细." +
strconv.Itoa(索引) + ".S 实际分成金额").Float64()
rmbPay.go:333         var 代理详情 DB.DB_User
rmbPay.go:334         代理详情, err = service.NewUser(c, tx).Info(局_uid)
rmbPay.go:335         第 31 页
rmbPay.go:336         if err != nil {
rmbPay.go:337             return errors.Join(err, errors.New("代理"+代理详情.User+",不存在,无法扣除分成"))
rmbPay.go:338         }
rmbPay.go:339         err = tx.Model(DB.DB_User{}).Where("Id = ?", 局_uid).Update("Rmb", gorm.Expr("RMB - ?", 局_金额)).Error
rmbPay.go:340
rmbPay.go:341         if err != nil {
rmbPay.go:342             return errors.Join(err, errors.New("代理分成用户扣除分成失败"))
rmbPay.go:343         }
rmbPay.go:344         代理详情.Rmb = Float64 减 float64(代理详情.Rmb, 局_金额, 2)
rmbPay.go:345         info.LogMoney = append(info.LogMoney, DB.DB_LogMoney{
rmbPay.go:346             User: 代理详情.User,
rmbPay.go:347             Time: time.Now().Unix(),
rmbPay.go:348             Ip: c.ClientIP(),
rmbPay.go:349             Count: Float64 取负值(局_金额),
rmbPay.go:350             Note: fmt.Sprintf("管理员操作用户退款,订单:%s,扣除订单分成%s|新余额=%s", 参数.PayOrder, Float64 到文本(局_金额, 2), Float64 到文本(代理详情.Rmb, 2)),
rmbPay.go:351         })
rmbPay.go:352     }
rmbPay.go:353     //err = 参数.E 额外信息.Set("卡类金额", 局_卡类信息.Money)
rmbPay.go:354     //err = 参数.E 额外信息.Set("调价详情", 调价信息列表)
rmbPay.go:355     //err = 参数.E 额外信息.Set("总调价", 总调价)

```



```

rmbPay.go:355         for 索引 := range 参数.E 额外信息.Len("调价详情") {
rmbPay.go:356             局_uid := 参数.E 额外信息.Get("调价详情." +
rmbPay.go:357                 strconv.Itoa(索引) + ".AgentId").Int()
rmbPay.go:358             局_金额 := 参数.E 额外信息.Get("分成详细." +
rmbPay.go:359                 strconv.Itoa(索引) + ".Markup").Float64()
rmbPay.go:360             var 代理详情 DB.DB_User
rmbPay.go:361             代理详情, err = service.NewUser(c, tx).Info(局_uid)
rmbPay.go:362             if err != nil {
rmbPay.go:363                 return errors.Join(err, errors.New("代理"+代理详
rmbPay.go:364                     情.User+",不存在,无法扣除调价分成"))
rmbPay.go:365             }
rmbPay.go:366             err = tx.Model(DB.DB_User{}).Where("Id = ?", 局
rmbPay.go:367                 _uid).Update("Rmb", gorm.Expr("RMB - ?", 局_金额)).Error
rmbPay.go:368             if err != nil {
rmbPay.go:369                 return errors.Join(err, errors.New("代理调价用户扣除
rmbPay.go:370                     调价失败"))
rmbPay.go:371             }
rmbPay.go:372             代理详情.Rmb = Float64 减 float64(代理详情.Rmb, 局_金额, 2)
rmbPay.go:373             info.LogMoney = append(info.LogMoney, DB.DB_LogMoney{
rmbPay.go:374                 User: 代理详情.User,
rmbPay.go:375                 Time: time.Now().Unix(),
rmbPay.go:376                 Ip: c.ClientIP(),
rmbPay.go:377                 Count: Float64 取负值(局_金额),
rmbPay.go:378                 Note: fmt.Sprintf("管理员操作用户退款,订单:%s,扣除订
rmbPay.go:379                     单代理调价%s|新余额=%s", 参数.PayOrder, Float64 到文本(局_金额, 2),
rmbPay.go:380                     Float64 到文本(代理详情.Rmb, 2)),
rmbPay.go:381             })
rmbPay.go:382         }
rmbPay.go:383         参数.Status = constant.D 订单状态_退款成功
rmbPay.go:384         参数.E 额外信息.Set("退款时间", time.Now().Format("2006-01-02
rmbPay.go:385             15:04:05"))
rmbPay.go:386         参数.Extra = 参数.E 额外信息.String()
rmbPay.go:387         data := map[string]interface{}{
rmbPay.go:388             "Status": 参数.Status,
rmbPay.go:389             "Extra": 参数.Extra,
rmbPay.go:390         }
rmbPay.go:391         if 备注 != "" {
rmbPay.go:392             第 32 页
rmbPay.go:393             data["Note"] = 备注
rmbPay.go:394         }
rmbPay.go:395         if err = tx.Model(DB.DB_LogRMBPayOrder{}).Where("Id = ?", 参
rmbPay.go:396             数.Id).Updates(data).Error; err != nil {
rmbPay.go:397             return errors.Join(err, errors.New("订单状态更新失败"))
rmbPay.go:398         }

```

```

rmbPay.go:390          参数.Y 异步回调地址 = setting.Q 系统设置().X 系统地址 +
"/webApi/payNotify2/" + 参数.PayOrder //微信可能用到

rmbPay.go:391
rmbPay.go:392          if err == nil && 参数.ReceivedUid > 0 { //如果是代收款订单, 要
恢复已扣的余额
rmbPay.go:393          err =
tx.Model(DB.DB_User{}).Clauses(clause.Locking{Strength:
"UPDATE"}).Where("Id = ?", 参数.ReceivedUid).First(&info.user).Error
rmbPay.go:394          if err != nil {
rmbPay.go:395              return errors.New("代收款用户不存在,无法恢复余额")
rmbPay.go:396          }
rmbPay.go:397          err = tx.Model(DB.DB_User{}).Where("Id = ?", 参
数.ReceivedUid).Update("Rmb", gorm.Expr("RMB + ?", 参数.Rmb)).Error
rmbPay.go:398          if err != nil {
rmbPay.go:399              return errors.New("代收款用户恢复余额失败")
rmbPay.go:400          }
rmbPay.go:401          err =
tx.Model(DB.DB_User{}).Clauses(clause.Locking{Strength:
"UPDATE"}).Where("Id = ?", 参数.ReceivedUid).First(&info.Agent).Error
rmbPay.go:402          if err != nil {
rmbPay.go:403              return errors.New("代收款用户不存在,无法恢复余额")
rmbPay.go:404          }
rmbPay.go:405          info.LogMoney = append(info.LogMoney, DB.DB_LogMoney{
rmbPay.go:406              User:  info.Agent.User,
rmbPay.go:407              Time:  time.Now().Unix(),
rmbPay.go:408              Ip:    c.ClientIP(),
rmbPay.go:409              Count: 参数.Rmb,
rmbPay.go:410              Note:  fmt.Sprintf("管理员操作代收款订单 id:%s,第三方
订单:%s,退款,恢复代扣余额%s|新余额=%s", 参数.PayOrder, 参数.PayOrder2,
Float64 到文本(参数.Rmb, 2), Float64 到文本(info.Agent.Rmb, 2)),
rmbPay.go:411              })
rmbPay.go:412          }
rmbPay.go:413          err = 局_通道.D 订单退款(c, &参数) //最后处理因为不可恢复,所以退
款结果作为最终条件

rmbPay.go:414          return err
rmbPay.go:415      })
rmbPay.go:416      //最后写出日志
rmbPay.go:417      if err == nil {
rmbPay.go:418          c.Set("tx", &db)
rmbPay.go:419          if err = log.L_log.S 输出日志(c, info.LogMoney); err != nil {
rmbPay.go:420              global.GVA_LOG.Error("输出日志失败!", zap.Any("err",
err))
rmbPay.go:421          }
rmbPay.go:422          if err = log.L_log.S 输出日志(c, info.LogVipNumber); err !=
rmbPay.go:423          nil {
rmbPay.go:424              global.GVA_LOG.Error("输出日志失败!", zap.Any("err",
err))

```

```

rmbPay.go:425         }
rmbPay.go:426     }
rmbPay.go:427
rmbPay.go:428         return
rmbPay.go:429     }
rmbPay.go:430
rmbPay.go:431 func (j *rmbPay) D 订单回调(c *gin.Context) (响应信息 string, 响应代码
int) {
rmbPay.go:432     响应代码 = 200
rmbPay.go:433     响应信息 = "订单不存在"
rmbPay.go:434     var 参数 m.PayParams

                                第 33 页

rmbPay.go:435     参数.Z 支付配置 s = setting.Q 在线支付配置()
rmbPay.go:436     参数.Z 支付配置, _ = json.Marshal(&参数.Z 支付配置 s)

rmbPay.go:437
rmbPay.go:438     orderId := c.Param("order")
rmbPay.go:439     if orderId == "" || orderId == "123456" {
rmbPay.go:440         for i, _ := range j.已注册通道 {
rmbPay.go:441             if orderId = j.已注册通道[i].Q 取订单 id(c, &参数);
orderId != "" {
rmbPay.go:442                 break
rmbPay.go:443             }
rmbPay.go:444         }
rmbPay.go:445     }

rmbPay.go:446     if orderId == "" {
rmbPay.go:447         return
rmbPay.go:448     }
rmbPay.go:449
rmbPay.go:450     tx := *global.GVA_DB
rmbPay.go:451     s := service.NewRmbPayService(&tx)

rmbPay.go:452
rmbPay.go:453     var err error
rmbPay.go:454     参数.DB_LogRMBPayOrder, err = s.Info2(gin.H{"PayOrder": orderId})
rmbPay.go:455     参数.E 额外信息, _ = gjson.LoadJson(参数.Extra)
rmbPay.go:456     if err != nil || 参数.Status != constant.D 订单状态_等待支付 {
rmbPay.go:457         return
rmbPay.go:458     }
rmbPay.go:459
rmbPay.go:460     局_通道, ok := j.已注册通道[参数.Type]
rmbPay.go:461     if !ok {
rmbPay.go:462         err = errors.New("支付方式未配置")
rmbPay.go:463         return
rmbPay.go:464     }

rmbPay.go:465     响应信息, 响应代码, err = 局_通道.D 订单支付回调(c, &参数)
rmbPay.go:467     if err != nil {
rmbPay.go:468         global.GVA_LOG.Error("订单回调失败!", zap.Any("err", err))
rmbPay.go:469         return

```

```

rmbPay.go:470     }
rmbPay.go:471     err = 参数.E 额外信息.Set("回调时间", time.Now().Unix())
rmbPay.go:472     err = 参数.E 额外信息.Set("回调 ip", c.ClientIP())
rmbPay.go:473     err = 参数.E 额外信息.Set("回调 ua", c.GetHeader("User-Agent"))
rmbPay.go:474     参数.Extra = 参数.E 额外信息.String()
rmbPay.go:475
rmbPay.go:476     db := *global.GVA_DB
rmbPay.go:477     //先加锁修改为待处理
rmbPay.go:478     err = db.Transaction(func(tx *gorm.DB) error {
rmbPay.go:479         var 局_订单信息 DB.DB_LogRMBPayOrder
rmbPay.go:480         err = tx.Model(DB.DB_LogRMBPayOrder{}).
rmbPay.go:481             Clauses(clause.Locking{Strength: "UPDATE"}).
rmbPay.go:482             Where("Id=?", 参数.Id).
rmbPay.go:483             First(&局_订单信息).Error //加锁再查一次
rmbPay.go:484             if err != nil || 局_订单信息.Status != constant.D 订单状态_等
待支付 { //有其他线程抢到任务了并处理了

                第 34 页

                return err
            }
            //确定订单状态没问题,而且数据已经是行锁状态,开始更新状态等信息
            参数.Status = constant.D 订单状态_已付待处理
            err = tx.Model(DB.DB_LogRMBPayOrder{}).
                Where("Id=?", 参数.Id).
                Updates(map[string]interface{}{
                    "PayOrder":      参数.PayOrder,
                    "PayOrder2":     参数.PayOrder2,
                    "User":           参数.User,
                    "Uid":            参数.Uid,
                    "UidType":        参数.UidType,
                    "Status":         参数.Status,
                    "ProcessingType": 参数.ProcessingType,
                    "Extra":          参数.Extra,
                    "Rmb":            参数.Rmb, //小叮当可能改变实际支付
金额
                }).Error
            return err //提交事务自动解锁
        })
rmbPay.go:503     if err != nil {
rmbPay.go:504         return
rmbPay.go:505     }
rmbPay.go:506
rmbPay.go:507     _ = j.Z 支付成功_后处理(c, &参数)
rmbPay.go:508     _ = cpsPayOrder.L_cpsPayOrder.C 处理佣金发放_线程安全(c, &参数)
rmbPay.go:510

```

```

rmbPay.go:511         return
rmbPay.go:512     }
rmbPay.go:513     func (j *rmbPay) D 订单退款回调(c *gin.Context) (响应信息 string, 响应代
                                码 int) {
rmbPay.go:514         响应代码 = 200
rmbPay.go:515         响应信息 = "订单不存在"
rmbPay.go:516         var 参数 m.PayParams
rmbPay.go:517         参数.Z 支付配置 s = setting.Q 在线支付配置()
rmbPay.go:518         参数.Z 支付配置, _ = json.Marshal(&参数.Z 支付配置 s)
rmbPay.go:519         参数.E 额外信息, _ = gjson.LoadJson(参数.Extra)
rmbPay.go:520         orderId := c.Param("order")
rmbPay.go:521         if orderId == "" {
rmbPay.go:522             for i, _ := range j.已注册通道 {
rmbPay.go:523                 if orderId = j.已注册通道[i].Q 取订单 id(c, &参数);
orderId != "" {
rmbPay.go:524                     break
rmbPay.go:525                 }
rmbPay.go:526             }
rmbPay.go:527         }
rmbPay.go:528
rmbPay.go:529         if orderId == "" {
rmbPay.go:530             return
rmbPay.go:531         }
rmbPay.go:532         tx := *global.GVA_DB
rmbPay.go:533         s := service.NewRmbPayService(&tx)
rmbPay.go:534

```

第 35 页

```

rmbPay.go:535     var err error
rmbPay.go:536     参数.DB_LogRMBPayOrder, err = s.Info2(gin.H{"PayOrder": orderId})
rmbPay.go:537     if err != nil || 参数.Status != constant.D 订单状态_退款中 {
rmbPay.go:538         return
rmbPay.go:539     }
rmbPay.go:540
rmbPay.go:541     局_通道, ok := j.已注册通道[参数.Type]
rmbPay.go:542     if !ok {
rmbPay.go:543         err = errors.New("支付方式未配置")
rmbPay.go:544         return
rmbPay.go:545     }
rmbPay.go:546
rmbPay.go:547     响应信息, 响应代码, err = 局_通道.D 订单退款回调(c, &参数)
rmbPay.go:548     if err != nil {
rmbPay.go:549         参数.Status = constant.D 订单状态_退款失败
rmbPay.go:550         参数.Note = 参数.Note + err.Error()
rmbPay.go:551     } else {
rmbPay.go:552         参数.Status = constant.D 订单状态_退款成功
rmbPay.go:553     }
rmbPay.go:554     err = 参数.E 额外信息.Set("退款回调时间", time.Now().Unix())

```

```

rmbPay.go:555     err = 参数.E 额外信息.Set("退款回调 ip", c.ClientIP())
rmbPay.go:556     err = 参数.E 额外信息.Set("退款回调 ua", c.GetHeader("User-Agent"))
rmbPay.go:557     参数.Extra = 参数.E 额外信息.String()

rmbPay.go:558
rmbPay.go:559     db := *global.GVA_DB
rmbPay.go:560     //先加锁修改为待处理
rmbPay.go:561     err = db.Model(DB.DB_LogRMBPayOrder{}).
rmbPay.go:562         Where("Id=?", 参数.Id).
rmbPay.go:563         Updates(map[string]interface{}{
rmbPay.go:564             "Status": 参数.Status,
rmbPay.go:565             "Extra": 参数.Extra,
rmbPay.go:566         }).Error
rmbPay.go:567     return
rmbPay.go:568 }
rmbPay.go:569 func (j *rmbPay) Z 支付成功_后处理(c *gin.Context, 参数 *m.PayParams)
(err error) {
rmbPay.go:570     if 参数.Status != constant.D 订单状态_已付待处理 {
rmbPay.go:571         return
rmbPay.go:572     }
rmbPay.go:573     var info struct {
rmbPay.go:574         LogMoney      []DB.DB_LogMoney
rmbPay.go:575         LogVipNumber   []DB.DB_LogVipNumber
rmbPay.go:576         LogKa           []DB.DB_LogKa
rmbPay.go:577         /*
rmbPay.go:578             user 用户详情 DB.DB_User*/
rmbPay.go:579         app 用户详情 DB.DB_AppUser
rmbPay.go:580         卡类详情      dbm.DB_KaClass
rmbPay.go:581         卡号详情      DB.DB_Ka
rmbPay.go:582         app 详情      DB.DB_AppInfo
rmbPay.go:583     }
rmbPay.go:584     var 临时数据 interface{}

第 36 页

rmbPay.go:585     var ok bool
rmbPay.go:586
rmbPay.go:587     //这里就是转账了,需要开启事务保证
rmbPay.go:588     db := *global.GVA_DB
rmbPay.go:589     //先加锁修改为待处理
rmbPay.go:590     err = db.Transaction(func(tx *gorm.DB) error {
rmbPay.go:591         //重新加锁,确定状态
rmbPay.go:592         c.Set("tx", tx) //防止 K 卡类直冲_事务 死锁
rmbPay.go:593         defer delete(c.Keys, "tx")
rmbPay.go:594
rmbPay.go:595         var 局_订单信息 DB.DB_LogRMBPayOrder
rmbPay.go:596         err = tx.Model(DB.DB_LogRMBPayOrder{}).
rmbPay.go:597             Clauses(clause.Locking{Strength: "UPDATE"}).
rmbPay.go:598             Where("Id=?", 参数.Id).

```

```

rmbPay.go:599             First(&局_订单信息).Error //加锁再查一次
rmbPay.go:600             if err != nil || 局_订单信息.Status != constant.D 订单状态_已
付待处理 { //有其他线程抢到任务了并处理了
rmbPay.go:601                 return err
rmbPay.go:602             }
rmbPay.go:603             参数.DB_LogRMBPayOrder = 局_订单信息
rmbPay.go:604             参数.E 额外信息, _ = gjson.LoadJson(参数.Extra)
rmbPay.go:605
rmbPay.go:606             局_订单信息.Status = constant.D 订单状态_已付待处理
rmbPay.go:607             switch 参数.ProcessingType {
rmbPay.go:608             default:
rmbPay.go:609                 return errors.New("ProcessingType 错误")
rmbPay.go:610             case constant.D 订单类型_余额充值: //0
rmbPay.go:611                 err = tx.Model(DB.DB_User{}).Where("Id = ?", 参
数.Uid).Update("RMB", gorm.Expr("RMB + ?", 参数.Rmb)).Error
rmbPay.go:612                 if err != nil {
rmbPay.go:613                     return errors.Join(err, errors.New(strconv.Itoa(参
数.Uid)+"Id 余额增加失败"))
rmbPay.go:614                 }
rmbPay.go:615                 var 局_新余额 float64
rmbPay.go:616                 err =
tx.Model(DB.DB_User{}).Select("Rmb").Where("Id=?", 参数.Uid).First(&
局_新余额).Error
rmbPay.go:617                 if err != nil {
rmbPay.go:618                     return errors.Join(err, errors.New(strconv.Itoa(参
数.Uid)+"新余额读取失败"))
rmbPay.go:619                 }
rmbPay.go:620                 info.LogMoney = append(info.LogMoney, DB.DB_LogMoney{
rmbPay.go:621                     User: 参数.User,
rmbPay.go:622                     Time: time.Now().Unix(),
rmbPay.go:623                     Ip: 参数.Ip,
rmbPay.go:624                     Count: 参数.Rmb,
rmbPay.go:625                     Note: fmt.Sprintf("余额充值支付订单:%s,付款成功|新余
额=%v", 参数.PayOrder, Float64 到文本(局_新余额, 2)),
rmbPay.go:626                 })
rmbPay.go:627             case constant.D 订单类型_购卡直冲: //1
rmbPay.go:628                 var 卡类 ID, AppUserId int
rmbPay.go:629
rmbPay.go:630                 if 卡类 ID = 参数.E 额外信息.Get("KaClassId").Int(); 卡类
ID == 0 {
rmbPay.go:631                     return errors.New("订单 id:%s,扩展信息 KaClassId 不正
确")
rmbPay.go:632                 }
rmbPay.go:633

```

```

rmbPay.go:634         if AppUserId = 参数.E 额外信息.Get("AppUserId").Int();
AppUserId == 0 {
第 37 页
rmbPay.go:635         return errors.New("订单 id:%s,扩展信息 AppUserId 不正
确")
rmbPay.go:636     }
rmbPay.go:637
rmbPay.go:638         if err = ka.L_ka.K 卡类直冲_事务(c, 卡类 ID, AppUserId);
err != nil {
rmbPay.go:639             return err
rmbPay.go:640         }
rmbPay.go:641         if 临时数据, ok = c.Get("logMoney"); ok { //判断是否有
rmb 充值的日志
rmbPay.go:642             info.LogMoney = append(info.LogMoney, 临时数
据.(DB.DB_LogMoney))
rmbPay.go:643             info.LogMoney[len(info.LogMoney)-1].Note = "购卡直冲
支付订单:" + 参数.PayOrder + info.LogMoney[len(info.LogMoney)-1].Note
rmbPay.go:644         }
rmbPay.go:645
rmbPay.go:646         if 临时数据, ok = c.Get("logVipNumber"); ok { //判断是否
有积分充值的日志
rmbPay.go:647             info.LogVipNumber = append(info.LogVipNumber, 临时
数据.(DB.DB_LogVipNumber))
rmbPay.go:648             info.LogVipNumber[len(info.LogVipNumber)-1].Note =
"购卡直冲支付订单:" + 参数.PayOrder +
info.LogVipNumber[len(info.LogVipNumber)-1].Note
rmbPay.go:649         }
rmbPay.go:650
rmbPay.go:651         if 临时数据, ok = c.Get("info.app 详情"); ok {
rmbPay.go:652             info.app 详情 = 临时数据.(DB.DB_AppInfo)
rmbPay.go:653         }
rmbPay.go:654         if 临时数据, ok = c.Get("info.卡类详情"); ok {
rmbPay.go:655             info.卡类详情 = 临时数据.(dbm.DB_KaClass)
rmbPay.go:656         }
rmbPay.go:657         if text, ok2 := c.Get("info.app 用户详情"); ok2 {
rmbPay.go:658             info.app 用户详情 = text.(DB.DB_AppUser)
rmbPay.go:659         }
rmbPay.go:660         参数.E 额外信息.Get("卡类 ID", 卡类 ID)
rmbPay.go:661         参数.E 额外信息.Get("卡类名称", info.卡类详情.Name)
rmbPay.go:662         参数.E 额外信息.Get("应用", info.app 详情.AppName)
rmbPay.go:663         参数.Note = 参数.Note + "充值卡类 ID:" + strconv.Itoa(卡类
ID) + ",应用:" + info.app 详情.AppName + ",卡类:" + info.卡类详情.Name
rmbPay.go:664         参数.E 额外信息.Set("AgentUid", info.app 用户详
情.AgentUid)
rmbPay.go:665

```



```

rmbPay.go:666                //判断代理是否有分成,如果有进行处理
rmbPay.go:667                if err = j.代理分成(c, 参数, 参数.E 额外信息.Get("卡类金额
rmbPay.go:668                    ").Float64()); err != nil {
rmbPay.go:669                    return err
rmbPay.go:670                } else {
rmbPay.go:671                    if 临时数据, ok = c.Get("LogMoney"); ok && 临时数
rmbPay.go:672                        据 != nil {
rmbPay.go:673                            info.LogMoney = append(info.LogMoney, 临时数
rmbPay.go:674                                据.([]DB.DB_LogMoney)...)
rmbPay.go:675                        }
rmbPay.go:676                    }
rmbPay.go:677                case constant.D 订单类型_支付购卡: //3
rmbPay.go:678                    //没有订单信息没有 Uid,用户名,需要修改
rmbPay.go:679                    if 参数.E 额外信息.Get("KaClassId").Int() == 0 {
rmbPay.go:680                        return errors.New("扩展信息 KaClassId 不正确")
rmbPay.go:681                    }
rmbPay.go:682                    if info.卡类详情, err = service.NewKaClass(c,
rmbPay.go:683                        tx).Info(参数.E 额外信息.Get("KaClassId").Int()); err != nil {
rmbPay.go:684                        return errors.Join(err, errors.New(fmt.Sprintf("卡
rmbPay.go:685                            类:%d 取详情失败", 参数.E 额外信息.Get("KaClassId").Int())))
rmbPay.go:686                    }
rmbPay.go:687                    if info.app 详情, err = service.NewAppInfo(c,
rmbPay.go:688                        tx).Info(info.卡类详情.AppId); err != nil {
rmbPay.go:689                        return errors.Join(err,
rmbPay.go:690                            errors.New(fmt.Sprintf("AppId:%d 取详情失败", 参数.E 额外信
rmbPay.go:691                                息.Get("AppId").Int())))
rmbPay.go:692                    }

```

第 38 页

```

rmbPay.go:685                info.卡号详情, err = ka.L_ka.Ka 单卡创建(c, info.卡类详
rmbPay.go:686                    情.Id, -1, "系统自动", "支付购卡订单 ID:"+参数.PayOrder, "", 0)
rmbPay.go:687                if err != nil {
rmbPay.go:688                    return errors.Join(err, errors.New("卡号创建失败"))
rmbPay.go:689                }
rmbPay.go:690
rmbPay.go:691                err = 参数.E 额外信息.Set("Id", info.卡号详情.Id)
rmbPay.go:692                err = 参数.E 额外信息.Set("卡号", info.卡号详情.Name)
rmbPay.go:693                err = 参数.E 额外信息.Set("卡类", info.卡类详情.Name)
rmbPay.go:694                err = 参数.E 额外信息.Set("应用", info.app 详情.AppName)
rmbPay.go:695
rmbPay.go:696                参数.Note = 参数.Note + "购卡:" + info.卡号详情.Name + ",
rmbPay.go:697                    应用:" + info.app 详情.AppName + ",卡类:" + info.卡类详情.Name
rmbPay.go:698                    局_文本 := fmt.Sprintf("支付购卡订单 ID:%s,卡类:%d,消
rmbPay.go:699                        费:%.2f)", 参数.PayOrder, info.卡号详情.KaClassId, 参数.Rmb)
rmbPay.go:700                info.LogKa = append(info.LogKa, DB.DB_LogKa{

```

```

rmbPay.go:699         User:      "支付购卡",
rmbPay.go:700         UserType: constant.Log_卡操作用户_系统自动,
rmbPay.go:701         Ka:       info.卡类详情.Name,
rmbPay.go:702         KaType:   constant.Log_卡操作_增,
rmbPay.go:703         Time:     time.Now().Unix(),
rmbPay.go:704         Ip:       参数.Ip,
rmbPay.go:705         Note:     局_文本,
rmbPay.go:706     })
rmbPay.go:707     if text, ok2 := c.Get("info.app 用户详情"); ok2 {
rmbPay.go:708         info.app 用户详情 = text.(DB.DB_AppUser)
rmbPay.go:709     }
rmbPay.go:710
rmbPay.go:711     if info.app 用户详情.AgentUid != 0 {
rmbPay.go:712         参数.E 额外信息.Set("AgentUid", info.app 用户详
情.AgentUid)
rmbPay.go:713     } else {
rmbPay.go:714         //支付购卡,如果用户没登陆,可能没有用户代理标志,就需要使
用在线代理标志
rmbPay.go:715         参数.E 额外信息.Set("AgentUid", 参数.E 额外信息.Get("
在线信息 AgentUid").Int())
rmbPay.go:716     }
rmbPay.go:717
rmbPay.go:718     //判断代理是否有分成,如果有进行处理
rmbPay.go:719     if err = j.代理分成(c, 参数, 参数.E 额外信息.Get("卡类金额
").Float64()); err != nil {
rmbPay.go:720         return err
rmbPay.go:721     } else {
rmbPay.go:722         if 临时数据, ok = c.Get("LogMoney"); ok {
rmbPay.go:723             info.LogMoney = append(info.LogMoney, 临时数
据.([]DB.DB_LogMoney)...)
rmbPay.go:724         }
rmbPay.go:725     }
rmbPay.go:726 }
rmbPay.go:727 //判断是否为代收款
rmbPay.go:728 if 参数.ReceivedUid > 0 {
rmbPay.go:729     var 局_info DB.DB_User
rmbPay.go:730     //加锁再查一次 锁定数值 防止并发数据错误
rmbPay.go:731     err =
tx.Model(DB.DB_User{}).Clauses(clause.Locking{Strength:
"UPDATE"}).Where("Id=?", 参数.ReceivedUid).First(&局_info).Error
rmbPay.go:732     if err != nil {
rmbPay.go:733         return errors.Join(err, errors.New(strconv.Itoa(参
数.ReceivedUid)+"代理信息读取失败"))
rmbPay.go:734     }

```

```

rmbPay.go:735 //只有有信任度的代理,才可以代收款,所以可以扣到一定值的负数
rmbPay.go:736 err = tx.Model(DB.DB_User{}).Where("Id = ?", 参
数.ReceivedUid).Update("RMB", gorm.Expr("RMB - ?", 参数.Rmb)).Error
rmbPay.go:737 if err != nil {
rmbPay.go:738     return errors.Join(err, errors.New(strconv.Itoa(参
数.ReceivedUid)+"Id 余额减少失败"))
rmbPay.go:739 }
rmbPay.go:740 //再查一次
rmbPay.go:741 err = tx.Model(DB.DB_User{}).Where("Id=?", 参
数.ReceivedUid).First(&局_info).Error
rmbPay.go:742 if err != nil {
rmbPay.go:743     return errors.Join(err, errors.New(strconv.Itoa(参
数.ReceivedUid)+"新余额读取失败"))
rmbPay.go:744 }
rmbPay.go:745 str := fmt.Sprintf("用户%s,%s 订单 ID:%s,第三方订单
ID:%s,%s 代收款:¥%s ,|新余额≈%s", 参数.User, j.Map 订单类型[参
数.ProcessingType], 参数.PayOrder, 参数.PayOrder2, 参数.Type, Float64
到文本(参数.Rmb, 2), Float64 到文本(局_info.Rmb, 2))

rmbPay.go:746 info.LogMoney = append(info.LogMoney, DB.DB_LogMoney{
rmbPay.go:747     User: 局_info.User,
rmbPay.go:748     Time: time.Now().Unix(),
rmbPay.go:749     Ip: 参数.Ip,
rmbPay.go:750     Count: Float64 取负值(参数.Rmb),
rmbPay.go:751     Note: str,
rmbPay.go:752 })
rmbPay.go:753 }
rmbPay.go:754 //如果能走到这里说明上面处理成功了, 订单状态改为成功
rmbPay.go:755 参数.Status = constant.D 订单状态_成功
rmbPay.go:756 参数.Extra = 参数.E 额外信息.String()
rmbPay.go:757 err = tx.Model(DB.DB_LogRMBPayOrder{}).
rmbPay.go:758     Where("Id=?", 参数.Id).
rmbPay.go:759     Updates(map[string]interface{}{
rmbPay.go:760         "Status": constant.D 订单状态_成功,
rmbPay.go:761         "Extra": 参数.Extra,
rmbPay.go:762         "Note": 参数.Note,
rmbPay.go:763     }).Error
rmbPay.go:764 return err //最后一步提交事务
rmbPay.go:765 })
rmbPay.go:766 if err != nil { //如果有错误,只修改备注,然后等人工处理
rmbPay.go:767     参数.Note = 参数.Note + err.Error()
rmbPay.go:768     err = db.Model(DB.DB_LogRMBPayOrder{}).
rmbPay.go:769         Where("Id=?", 参数.Id).
rmbPay.go:770         Updates(map[string]interface{}{"Note": 参数.Note +
err.Error()}).Error
rmbPay.go:771     if err != nil {

```

```

rmbPay.go:772             global.GVA_LOG.Error("更新数据库失败!", zap.Any("err",
err))
                        }
rmbPay.go:773             } else {
rmbPay.go:774                 //最后写出日志
rmbPay.go:775                 if err = log.L_log.S 输出日志(c, info.LogKa); err != nil {
rmbPay.go:776                     global.GVA_LOG.Error("输出日志失败!", zap.Any("err",
err))
rmbPay.go:777                 }
rmbPay.go:778                 if err = log.L_log.S 输出日志(c, info.LogMoney); err != nil {
rmbPay.go:779                     global.GVA_LOG.Error("输出日志失败!", zap.Any("err",
err))
rmbPay.go:780                 }
rmbPay.go:781                 if err = log.L_log.S 输出日志(c, info.LogVipNumber); err !=
rmbPay.go:782                 nil {
rmbPay.go:783                     global.GVA_LOG.Error("输出日志失败!", zap.Any("err",
err))
rmbPay.go:784                 }

```

第 40 页

```

rmbPay.go:785             }
rmbPay.go:786             return err
rmbPay.go:787         }
rmbPay.go:788         func (j *rmbPay) 代理分成(c *gin.Context, 参数 *m.PayParams,
rmbPay.go:789         AgentMoney float64) (err error) {
rmbPay.go:790             var info struct {
rmbPay.go:791                 LogMoney []DB.DB_LogMoney
rmbPay.go:792             }
rmbPay.go:793             //下边这两个可空
rmbPay.go:794             var AgentUid int
rmbPay.go:795             AgentUid = 参数.E 额外信息.Get("AgentUid").Int()
rmbPay.go:796
rmbPay.go:797             if AgentUid > 0 && AgentMoney > 0 {
rmbPay.go:798                 var tx *gorm.DB
rmbPay.go:799                 if tempObj, ok := c.Get("tx"); ok {
rmbPay.go:800                     tx = tempObj.(*gorm.DB)
rmbPay.go:801                 } else {
rmbPay.go:802                     db := *global.GVA_DB
rmbPay.go:803                     tx = &db
rmbPay.go:804                 }
rmbPay.go:805                 var 局_价格组成 struct {
rmbPay.go:806                     总调价 float64
rmbPay.go:807                     调价详情 []dbm.DB_KaClassUpPrice
rmbPay.go:808                     购买数量 int64
rmbPay.go:809
rmbPay.go:810                     卡类金额 float64
rmbPay.go:811                 }
rmbPay.go:812                 //err = 参数.E 额外信息.Set("卡类金额", 局_卡类信息.Money)

```

```

rmbPay.go:813 //err = 参数.E 额外信息.Set("调价详情", 调价信息列表)
rmbPay.go:814 //err = 参数.E 额外信息.Set("总调价", 总调价)
rmbPay.go:815 局_价格组成.总调价 = 参数.E 额外信息.Get("总调价").Float64()
rmbPay.go:816 局_价格组成.购买数量 = 1
rmbPay.go:817 局_价格组成.卡类金额 = 参数.E 额外信息.Get("卡类金额").Float64()
rmbPay.go:818 err = 参数.E 额外信息.Get("调价详情").Scan(&局_价格组成.调价详
情)

rmbPay.go:819 //先分成 代理调价信息的价格
rmbPay.go:820 if 局_价格组成.总调价 > 0 {
rmbPay.go:821     for _, v := range 局_价格组成.调价详情 {
rmbPay.go:822         分成金额 := Float64 乘 int64(v.Markup, 局_价格组成.购
买数量) //有多少卡就分多少个
rmbPay.go:823         err = tx.Model(DB.DB_User{}).Where("Id = ?",
v.AgentId).Update("RMB", gorm.Expr("RMB + ?", 分成金额)).Error
rmbPay.go:824         if err != nil {
rmbPay.go:825             return errors.Join(err, fmt.Errorf("代理分成失
败,请检查原因%d,%s", v.AgentId, Float64 到文本(分成金额, 2)))
rmbPay.go:826         }
rmbPay.go:827         var 局_userInfo DB.DB_User
rmbPay.go:828         err = tx.Model(DB.DB_User{}).Where("Id = ?",
v.AgentId).Find(&局_userInfo).Error
rmbPay.go:829         if err != nil {
rmbPay.go:830             return errors.Join(err, fmt.Errorf("代理分成后,
读取代理数据失败请检查原因%d,%s", v.AgentId, Float64 到文本(分成金额,
2)))
rmbPay.go:831         }
rmbPay.go:832
rmbPay.go:833 // 构建日志记录
rmbPay.go:834 var 局_临时日志 DB.DB_LogMoney
第 41 页
rmbPay.go:835 局_临时日志.Time = time.Now().Unix()
rmbPay.go:836 局_临时日志.Ip = c.ClientIP() + " " + Qqwry.Ip 查信息
2(c.ClientIP())
rmbPay.go:837 局_临时日志.User = 局_userInfo.User
rmbPay.go:838 局_临时日志.Count = 分成金额
rmbPay.go:839 局_临时文本 1, 局_临时文本 2 := "", ""
rmbPay.go:840 if agent.L_agent.Id 功能权限检测(c, 局_userInfo.Id,
DB.D 代理功能_查看归属软件用户) {
rmbPay.go:841     局_临时文本 1 = 参数.User
rmbPay.go:842     局_临时文本 2 = 参数.Note
rmbPay.go:843 }
rmbPay.go:844 局_日志前缀 := fmt.Sprintf("用户%s%s%s 订单 ID:%s", 局
_临时文本 1, j.Map 订单类型[参数.ProcessingType], 局_临时文本 2, 参
数.PayOrder)

```

```

rmbPay.go:845             局_临时日志.Note = 局_日志前缀 + fmt.Sprintf("调价分
成:¥%s(%s*%d),|新余额=%s",
rmbPay.go:846             Float64 到文本(分成金额, 2),
rmbPay.go:847             Float64 到文本(v.Markup, 2),
rmbPay.go:848             局_价格组成.购买数量, Float64 到文本(局
_userInfo.Rmb, 2))
rmbPay.go:849             info.LogMoney = append(info.LogMoney, 局_临时日志)
rmbPay.go:850         }
rmbPay.go:851     }
rmbPay.go:852     //代理分成
rmbPay.go:853     //开始分利润 20240202 mark 处理重构以后改事务
rmbPay.go:854     代理分成数据, err3 := agent.L_agent.D 代理分成计算(c,
AgentUid, AgentMoney)
rmbPay.go:855     if err3 == nil {
rmbPay.go:856         for 局_索引 := range 代理分成数据 {
rmbPay.go:857             d := 代理分成数据[局_索引] //太长了,放个变量里
rmbPay.go:858             err = tx.Model(DB.DB_User{}).Where("Id = ?",
d.Uid).Update("RMB", gorm.Expr("RMB + ?", d.S 实际分成金额)).Error
rmbPay.go:859             if err != nil {
rmbPay.go:860                 return errors.Join(err,
errors.New(strconv.Itoa(d.Uid)+"Id 余额增加失败"))
rmbPay.go:861             }
rmbPay.go:862             var 局_新余额 float64
rmbPay.go:863             err =
tx.Model(DB.DB_User{}).Select("Rmb").Where("Id=?", d.Uid).First(&局_
新余额).Error
rmbPay.go:864             if err != nil {
rmbPay.go:865                 return errors.Join(err,
errors.New(strconv.Itoa(d.Uid)+"新余额读取失败"))
rmbPay.go:866             }
rmbPay.go:867             局_临时文本 1 := S 三元(agent.L_agent.Id 功能权限检测
(c, d.Uid, DB.D 代理功能_查看归属软件用户), 参数.User, "")
rmbPay.go:868             局_临时文本 2 := S 三元(agent.L_agent.Id 功能权限检测
(c, d.Uid, DB.D 代理功能_查看归属软件用户), 参数.Note, "")
rmbPay.go:869             str := fmt.Sprintf("用户%s%s%s 订单 ID:%s,分成:¥%s
(¥%s(实价)*(%d%-%d%)),|新余额=%s", 局_临时文本 1, j.Map 订单类型[参
数.ProcessingType], 局_临时文本 2, 参数.PayOrder, Float64 到文本(d.S 实际
分成金额, 2), Float64 到文本(AgentMoney, 2), d.F 分成百分比, d.F 分给下级
百分比, Float64 到文本(局_新余额, 2))
rmbPay.go:870
rmbPay.go:871             info.LogMoney = append(info.LogMoney,
DB.DB_LogMoney{
rmbPay.go:872                 User: d.User,
rmbPay.go:873                 Time: time.Now().Unix(),
rmbPay.go:874                 Ip: 参数.Ip,
rmbPay.go:875                 Count: d.S 实际分成金额,
rmbPay.go:876                 Note: str,

```

```

rmbPay.go:877             })
rmbPay.go:878         }
rmbPay.go:879     }
rmbPay.go:880         // 分成结束===== 记录分成情况，后续退款对应扣除
rmbPay.go:881         参数.E 额外信息.Set("分成详细", 代理分成数据)
rmbPay.go:882     }
rmbPay.go:883     if info.LogMoney != nil {
rmbPay.go:884         c.Set("LogMoney", info.LogMoney)
rmbPay.go:885     }
rmbPay.go:886
rmbPay.go:887     return
rmbPay.go:888 }
rmbPay.go:889
rmbPay.go:890 func (j *rmbPay) Pay_取支付通道状态() gin.H {
rmbPay.go:891     局_数组 := j.Pay_取支付通道基本信息()
rmbPay.go:892     局 map := make(gin.H, len(局_数组))
rmbPay.go:893
rmbPay.go:894     for _, v := range 局_数组 {
rmbPay.go:895         if v.Alias != "" {
rmbPay.go:896             局 map[v.Alias] = v.Status
rmbPay.go:897         } else {
rmbPay.go:898             局 map[v.Name] = v.Status
rmbPay.go:899         }
rmbPay.go:900     }
rmbPay.go:901     return 局 map
rmbPay.go:902 }
rmbPay.go:903
rmbPay.go:904 type 支付通道基本信息 struct {
rmbPay.go:905     Id      int    `json:"Id"`
rmbPay.go:906     Name    string `json:"Name"`
rmbPay.go:907     Alias   string `json:"Alias"` //显示名称
rmbPay.go:908     Status  bool   `json:"Status"` //开关
rmbPay.go:909     RMB     int    `json:"RMB"`    //最大金额
rmbPay.go:910 }
rmbPay.go:911
rmbPay.go:912 func (j *rmbPay) Pay_取支付通道基本信息() []支付通道基本信息 {
rmbPay.go:913     支付配置 := setting.Q 在线支付配置()
rmbPay.go:914     if &支付配置 == nil {
rmbPay.go:915         return []支付通道基本信息{}
rmbPay.go:916     }
rmbPay.go:917     支付通道列表 := []支付通道基本信息{
rmbPay.go:918         {Id: 1, Name: "支付宝 PC", Alias: 支付配置.Z 支付宝显示名称,
        Status: 支付配置.Z 支付宝开关, RMB: 支付配置.Z 支付宝单次最大金额},

```

```

rmbPay.go:919         {Id: 2, Name: "支付宝当面付", Alias: 支付配置.Z 支付宝当面付显示
名称, Status: 支付配置.Z 支付宝当面付开关, RMB: 支付配置.Z 支付宝单次最大
金额},

rmbPay.go:920         {Id: 3, Name: "支付宝 H5", Alias: 支付配置.Z 支付宝 H5 显示名称,
Status: 支付配置.Z 支付宝 H5 开关, RMB: 支付配置.Z 支付宝单次最大金额},

rmbPay.go:921         {Id: 4, Name: "微信支付", Alias: 支付配置.W 微信支付显示名称,
Status: 支付配置.W 微信支付开关, RMB: 支付配置.W 微信支付单次最大金额},

rmbPay.go:922         {Id: 5, Name: "小叮当", Alias: 支付配置.X 小叮当支付显示名称,
Status: 支付配置.X 小叮当支付开关, RMB: 支付配置.X 小叮当单次最大金额},

rmbPay.go:923         {Id: 6, Name: "虎皮椒", Alias: 支付配置.H 虎皮椒支付显示名称,
Status: 支付配置.H 虎皮椒支付开关, RMB: 支付配置.H 虎皮椒单次最大金额},

rmbPay.go:924         {Id: 7, Name: "易支付", Alias: 支付配置.Y 易支付显示名称,
Status: 支付配置.Y 易支付开关, RMB: 支付配置.Y 易支付最大金额},

rmbPay.go:925         {Id: 8, Name: "易支付 2", Alias: 支付配置.Y 易支付 2 显示名称,
Status: 支付配置.Y 易支付 2 开关, RMB: 支付配置.Y 易支付 2 最大金额},

rmbPay.go:926         {Id: 9, Name: "余额支付", Alias: 支付配置.Y 余额支付显示名称,
Status: 支付配置.Y 余额支付开关, RMB: 99999999},

rmbPay.go:927     }
rmbPay.go:928     return 支付通道列表
rmbPay.go:929 }
rmbPay.go:930
rmbPay.go:931 func (j *rmbPay) Pay_指定 Uid 待支付金额(c *gin.Context, Uid int) (金额
float64) {
rmbPay.go:932     // 开启事务,检测上层是否有事务,如果有直接使用,没有就创建一个
rmbPay.go:933     var tx *gorm.DB
rmbPay.go:934     if tempObj, ok := c.Get("tx"); ok {
第 43 页
rmbPay.go:935         tx = tempObj.(*gorm.DB)
rmbPay.go:936     } else {
rmbPay.go:937         db := *global.GVA_DB
rmbPay.go:938         tx = &db
rmbPay.go:939     }
rmbPay.go:940     //获取该 uid 等待支付的金额总数
rmbPay.go:941     err := tx.Model(DB.DB_LogRMBPayOrder{}).Select("sum(Rmb) as
Rmb").Where("ReceivedUid=? and Status=?", Uid, constant.D 订单状态_等
待支付).First(&金额).Error
rmbPay.go:942     if err != nil {
rmbPay.go:943         //如果出错,就返回 0 报错一般是 rmb 字段为 null 但是给的变量类型
为 float64 暂不影响,以后再查
rmbPay.go:944         //global.GVA_LOG.Error("获取指定 Uid 待支付金额!",
zap.Any("err", err))
rmbPay.go:945     }
rmbPay.go:946     return
rmbPay.go:947 }
ka.go:1 package ka
ka.go:2
ka.go:3 import (

```



```

ka.go:4         . "EFunc/utils"
ka.go:5         "errors"
ka.go:6         "fmt"
ka.go:7         "github.com/gin-gonic/gin"
ka.go:8         "go.uber.org/zap"
ka.go:9         "gorm.io/gorm"
ka.go:10        "gorm.io/gorm/clause"
ka.go:11        "server/Service/Ser_AppInfo"
ka.go:12        "server/Service/Ser_AppUser"
ka.go:13        "server/global"
ka.go:14        "server/new/app/logic/common/log"
ka.go:15        "server/new/app/logic/common/userClass"
ka.go:16        "server/new/app/models/constant"
ka.go:17        dbm "server/new/app/models/db"
ka.go:18        "server/new/app/service"
ka.go:19        DB "server/structs/db"
ka.go:20        "strconv"
ka.go:21        "time"
ka.go:22    )
ka.go:23
ka.go:24    var L_ka ka
ka.go:25
ka.go:26    func init() {
ka.go:27        L_ka = ka{}
ka.go:28    }
ka.go:29
ka.go:30
ka.go:31    type ka struct {
ka.go:32    }
ka.go:33
ka.go:34
ka.go:35    func (j *ka) K 卡类直冲_事务(c *gin.Context, 卡类 ID, 软件用户 Uid int)
ka.go:36    (err error) {
ka.go:37        //已优化,事务处理,数据库内直接加减乘除计算字段值,可以并发,不出错
ka.go:38        var info struct {
ka.go:39            卡类详情      dbm.DB_KaClass
ka.go:40            第 44 页
ka.go:41            app 用户详情  DB.DB_AppUser
ka.go:42            user 用户详情 DB.DB_User
ka.go:43            app 详情     DB.DB_AppInfo
ka.go:44            is 卡号      bool
ka.go:45            is 计点      bool
ka.go:46        }
ka.go:47        //第一个查询不用 tx 直接用全局即可,后面事务的才用 tx
ka.go:48        db := *global.GVA_DB
ka.go:49        S_KaClass := service.NewKaClass(c, &db)
ka.go:50        if info.卡类详情, err = S_KaClass.Info(卡类 ID); err != nil {
ka.go:51            err = errors.New("卡类不存在")
ka.go:52            return
ka.go:53        }

```

```

ka.go:51
ka.go:52         if info.app 用户详情, err = service.NewAppUser(c, &db, info.卡类详
情.AppId).InfoUid(软件用户Uid); err != nil {

ka.go:53             err = errors.New("软件用户不存在")
ka.go:54             return
ka.go:55         }
ka.go:56         if info.app 详情, err = service.NewAppInfo(c, &db).Info(info.卡类
详情.AppId); err != nil {

ka.go:57             err = errors.New("应用不存在")
ka.go:58             return
ka.go:59         }
ka.go:60         info.is 卡号 = S 三元(info.app 详情.AppType == 3 || info.app 详
情.AppType == 4, true, false)

ka.go:61         info.is 计点 = S 三元(info.app 详情.AppType == 2 || info.app 详
情.AppType == 4, true, false)

ka.go:62
ka.go:63         //检测用户分组是否相同 不相同处理
ka.go:64         if info.卡类详情.UserClassId == info.app 用户详情.UserClassId ||
info.app 用户详情.UserClassId == 0 {

ka.go:65             //分类相同,或用户为未分类 不处理
ka.go:66         } else {
ka.go:67             if info.卡类详情.NoUserClass == 2 {
ka.go:68                 return errors.New("用户类型不同无法充值.")
ka.go:69             }
ka.go:70         }
ka.go:71         //到这里基本就都没问题了,开启事务,增加卡使用次数,更新用户信息就可以了
ka.go:72         // 开启事务,检测上层是否有事务,如果有直接使用,没有就创建一个
ka.go:73         //20260414 如果在事务内执行,一定要传递 tx 不然可能造成死锁,
ka.go:74         var tx *gorm.DB
ka.go:75         if tempObj, ok := c.Get("tx"); ok {
ka.go:76             tx = tempObj.(*gorm.DB)
ka.go:77         } else {
ka.go:78             db = *global.GVA_DB
ka.go:79             tx = &db
ka.go:80         }
ka.go:81         //在事务中执行数据库操作,使用的是 tx 变量,不是 db。
ka.go:82         err = tx.Transaction(func(tx *gorm.DB) error {
ka.go:83             //卡库存减少成功,开始增加客户数据 ,重新加锁读取 App 用户信息,防止
并发数据错误

ka.go:84             err =
tx.Model(DB.DB_AppUser{}).Clauses(clause.Locking{Strength:
"UPDATE"}).Table("db_AppUser_"+strconv.Itoa(info.卡类详
情.AppId)).Where("Uid=?", info.app 用户详情.Uid).First(&info.app 用户详
情).Error

ka.go:85             if err != nil {

```

```

ka.go:86         return errors.Join(err, errors.New("未注册应用???感觉不可
能,之前读取过,请联系管理员"))
ka.go:87     }
                                第 45 页
ka.go:88         //处理新信息
ka.go:89         客户 expr := map[string]interface{}{}
ka.go:90         客户 expr["VipNumber"] = gorm.Expr("VipNumber + ?", info.卡类
详情.VipNumber) //积分不会变直接增加即可
ka.go:91         if info.卡类详情.VipNumber != 0 {
ka.go:92             //日志仅写到上下文内,由实际业务处理是否写入日志和修改备注信息
ka.go:93             c.Set("logVipNumber", DB.DB_LogVipNumber{
ka.go:94                 User:   Ser_AppUser.Uid 取 User(info.卡类详情.AppId,
info.app 用户详情.Uid),
ka.go:95                 AppId: info.卡类详情.AppId,
ka.go:96                 Ip:     c.ClientIP(),
ka.go:97                 Type:   1,
ka.go:98                 Time:   time.Now().Unix(),
ka.go:99                 Count: info.卡类详情.VipNumber,
ka.go:100                Note:  "应用 ID:" + strconv.Itoa(info.卡类详
情.AppId) + "卡类 Id:" + strconv.Itoa(info.卡类详情.Id) + "充值积分",
ka.go:101                })
ka.go:102         }
ka.go:103
ka.go:104         if info.卡类详情.MaxOnline > 0 {
ka.go:105             客户 expr["MaxOnline"] = info.卡类详情.MaxOnline //最大在
线数直接赋值处理即可
ka.go:106         }
ka.go:107         局_现行时间戳 := time.Now().Unix()
ka.go:108         if info.卡类详情.VipTime != 0 { //只有时间增减不为 0 的时候设置
的用户分类才有效
ka.go:109             if info.app 用户详情.UserClassId == info.卡类详
情.UserClassId {
ka.go:110                 //分类相同,正常处理时间或点数
ka.go:111                 if Ser_AppInfo.App 是否为计点(info.卡类详情.AppId) ||
info.app 用户详情.VipTime > 局_现行时间戳 {
ka.go:112                     //如果为计点 或 时间大于现在时间直接加就行了
ka.go:113                     客户 expr["VipTime"] = gorm.Expr("VipTime + ?",
info.卡类详情.VipTime)
ka.go:114                 } else {
ka.go:115                     //如果为计时 已经过期很久了,直接现行时间戳加卡时间
ka.go:116                     客户 expr["VipTime"] = 局_现行时间戳 + info.卡类
详情.VipTime
ka.go:117                 }
ka.go:118             } else {

```

```

ka.go:119                                //用户类型不同, 根据权重处理
ka.go:120                                var 局_旧用户类型权重, 局_新用户类型权重
DB.DB_UserClass
ka.go:121                                if info.app 用户详情.UserClassId > 0 {
ka.go:122                                    if 局_旧用户类型权重, err =
service.NewUserClass(c, tx).Info(info.app 用户详情.UserClassId);
err != nil {
ka.go:123                                        return errors.Join(err, errors.New("读取旧
用户类型权重失败"))
ka.go:124                                    }
ka.go:125                                } else {
ka.go:126                                    局_旧用户类型权重.Weight = 1
ka.go:127                                }
ka.go:128
ka.go:129                                if info.卡类详情.UserClassId > 0 {
ka.go:130                                    if 局_新用户类型权重, err =
service.NewUserClass(c, tx).Info(info.卡类详情.UserClassId); err !=
nil {
ka.go:131                                        return errors.Join(err, errors.New("读取新
用户类型权重失败"))
ka.go:132                                    }
ka.go:133                                } else {
ka.go:134                                    局_新用户类型权重.Weight = 1
ka.go:135                                }
ka.go:136
ka.go:137                                if info.is 计点 {
第 46 页
ka.go:138                                    //转换结果值,转后再增加新类型 值
ka.go:139                                    客户 expr["VipTime"] = gorm.Expr("VipTime * ?
/ ? +?", 局_旧用户类型权重.Weight, 局_新用户类型权重.Weight, info.卡类详
情.VipTime)
ka.go:140                                } else {
ka.go:141                                    if info.app 用户详情.VipTime < 局_现行时间戳 {
ka.go:142                                        //已经过期了直接赋值新类型 现行时间+新时间就可
以了
ka.go:143                                        客户 expr["VipTime"] = 局_现行时间戳 + info.
卡类详情.VipTime
ka.go:144                                    } else {
ka.go:145                                        //先计算还剩多长时间,剩余时间权重转换转换结果
值,+现在时间+卡增减时间
ka.go:146                                        客户 expr["VipTime"] =
gorm.Expr("(VipTime-?) * ? / ? +?", 局_现行时间戳, 局_旧用户类型权
重.Weight, 局_新用户类型权重.Weight, 局_现行时间戳+info.卡类详
情.VipTime)
ka.go:147                                    }
ka.go:148                                }

```

```

ka.go:149                                     //最后更换类型,防止前面用到卡类 id,计算权重转换类型错误
ka.go:150                                     客户 expr["UserClassId"] = info.卡类详情.UserClassId
ka.go:151                                     }
ka.go:152                                     }
ka.go:153                                     //更新客户数据
ka.go:154                                     err =
tx.Model(DB.DB_AppUser{}).Table("db_AppUser_"+strconv.Itoa(info.卡类
详情.AppId)).Where("Id = ?", info.app 用户详情.Id).Updates(&客户
expr).Error
ka.go:155                                     if err != nil {
ka.go:156                                     return errors.Join(err, errors.New("充值失败,重试"))
ka.go:157                                     }
ka.go:158                                     //处理账号的 RMB 增减
ka.go:159                                     if !info.is 卡号 && info.卡类详情.RMb > 0 {
ka.go:160                                     err =
tx.Model(DB.DB_User{}).Clauses(clause.Locking{Strength:
"UPDATE"}).Where("Id=?", info.app 用户详情.Uid).First(&info.user 用户详
情).Error
ka.go:161                                     if err != nil {
ka.go:162                                     return errors.Join(err, errors.New("用户账号不存在
"))
ka.go:163                                     }
ka.go:164                                     err = tx.Model(DB.DB_User{}).Where("Id = ?", info.app 用
户详情.Uid).Update("RMB", gorm.Expr("RMB + ?", info.卡类详
情.RMb)).Error
ka.go:165                                     if err != nil {
ka.go:166                                     return errors.Join(err, errors.New("充值余额时失败,
请重试"))
ka.go:167                                     }
ka.go:168                                     var 局_新余额 float64
ka.go:169                                     err = tx.Model(DB.DB_User{}).Select("Rmb").Where("Id
= ?", info.user 用户详情.Id).First(&局_新余额).Error
ka.go:170                                     if err != nil {
ka.go:171                                     return errors.Join(err, errors.New("充值后读取新余额
失败"))
ka.go:172                                     }
ka.go:173                                     //日志仅写到上下文内,由实际业务处理是否写入日志和修改备注信息
ka.go:174                                     c.Set("logMoney", DB.DB_LogMoney{
ka.go:175                                     User:  info.user 用户详情.User,
ka.go:176                                     Ip:    c.ClientIP(),
ka.go:177                                     Count: info.卡类详情.RMb,
ka.go:178                                     Note:  "应用 ID:" + strconv.Itoa(info.卡类详
情.AppId) + "卡类 Id:" + strconv.Itoa(info.卡类详情.Id) + "充值余额|新余
额≈" + Float64 到文本(局_新余额, 2),
ka.go:179                                     })
ka.go:180                                     }
ka.go:181                                     return nil

```

```

ka.go:182         })
ka.go:183         //写到上下文,备用
ka.go:184         c.Set("info.卡类详情", info.卡类详情)
ka.go:185         c.Set("info.app 用户详情", info.app 用户详情)
ka.go:186         c.Set("info.user 用户详情", info.user 用户详情)
ka.go:187         c.Set("info.app 详情", info.app 详情)
                                第 47 页
ka.go:188         return err
ka.go:189     }
ka.go:190
ka.go:191     // 有效期 0=9999999999 无限制
ka.go:192     func (j *ka) Ka 单卡创建(c *gin.Context, 卡类 id, 制卡人 Id int, 制卡人账
号 string, 管理员备注 string, 代理备注 string, 有效期时间戳 int64) (卡信
息切片 DB.DB_Ka, err error) {
ka.go:193         var info struct {
ka.go:194             卡类信息 dbm.DB_KaClass
ka.go:195         }
ka.go:196
ka.go:197         var tx *gorm.DB
ka.go:198         if tempObj, ok := c.Get("tx"); ok {
ka.go:199             tx = tempObj.(*gorm.DB)
ka.go:200         } else {
ka.go:201             db := *global.GVA_DB
ka.go:202             tx = &db
ka.go:203         }
ka.go:204
ka.go:205         if info.卡类信息, err = service.NewKaClass(c, tx).Info(卡类 id);
err != nil { //估计是卡类不存在
ka.go:206             return 卡信息切片, err
ka.go:207         }
ka.go:208
ka.go:209         for I := 0; I < 10; I++ {
ka.go:210             卡信息切片.Name = info.卡类信息.Prefix
ka.go:211             卡信息切片.Name += 生成随机字符串(info.卡类信息.KaLength-
len(info.卡类信息.Prefix)-1, info.卡类信息.KaStringType)
ka.go:212             卡信息切片.Name += 生成校验字符(卡信息切片.Name)
ka.go:213             _, err2 := service.NewKa(c, tx).Info2(gin.H{"Name": 卡信息切
片.Name})
ka.go:214             if err2 != nil { //如果有错误,说明没这卡号,可以使用
ka.go:215                 break
ka.go:216             }
ka.go:217             if I == 9 {
ka.go:218                 return 卡信息切片, errors.New("创建失败,连续 10 次没有随机
到不重复卡号,请尝试删除无用卡号,再重新制卡")
ka.go:219             }
ka.go:220         }

```

```

ka.go:221
ka.go:222         卡信息切片.AppId = info.卡类信息.AppId
ka.go:223         卡信息切片.KaClassId = info.卡类信息.Id
ka.go:224         卡信息切片.Status = 1
ka.go:225         卡信息切片.RegisterId = 制卡人 Id
ka.go:226         卡信息切片.RegisterUser = 制卡人账号
ka.go:227         卡信息切片.RegisterTime = time.Now().Unix()
ka.go:228         卡信息切片.AdminNote = 管理员备注
ka.go:229         卡信息切片.AgentNote = 代理备注
ka.go:230         卡信息切片.VipTime = info.卡类信息.VipTime
ka.go:231         卡信息切片.InviteCount = info.卡类信息.InviteCount
ka.go:232         卡信息切片.RMb = info.卡类信息.RMb
ka.go:233         卡信息切片.VipNumber = info.卡类信息.VipNumber
ka.go:234         卡信息切片.Money = info.卡类信息.Money
ka.go:235         卡信息切片.AgentMoney = info.卡类信息.AgentMoney
ka.go:236         卡信息切片.UserClassId = info.卡类信息.UserClassId
ka.go:237         卡信息切片.NoUserClass = info.卡类信息.NoUserClass
                                第 48 页
ka.go:238         卡信息切片.KaType = info.卡类信息.KaType
ka.go:239         卡信息切片.MaxOnline = info.卡类信息.MaxOnline
ka.go:240         卡信息切片.Num = 0
ka.go:241         卡信息切片.NumMax = info.卡类信息.Num
ka.go:242         卡信息切片.User = ""
ka.go:243         卡信息切片.UserTime = ""
ka.go:244         卡信息切片.UseTime = 0
ka.go:245         卡信息切片.InviteUser = ""
ka.go:246         卡信息切片.EndTime = 9999999999
ka.go:247         if 有效期时间戳 != 0 {
ka.go:248             卡信息切片.EndTime = 有效期时间戳
ka.go:249         }
ka.go:250         return 卡信息切片, tx.Model(DB.DB_Ka{}).Create(&卡信息切片).Error
ka.go:251     }
ka.go:252
ka.go:253     // 来源 AppId int, 卡号, 充值用户, 推荐人, 来源 IP string)
ka.go:254     func (j *ka) K 卡号充值_事务(c *gin.Context, 来源 AppId int, 卡号, 充值
用户, 推荐人 string) (err error) {
ka.go:255         var info struct {
ka.go:256             卡号详情      DB.DB_Ka
ka.go:257             app 用户详情  DB.DB_AppUser
ka.go:258             user 用户详情 DB.DB_User
ka.go:259             ka 用户详情   DB.DB_Ka

```

```

ka.go:260         app 详情      DB.DB_AppInfo
ka.go:261         is 卡号      bool
ka.go:262         is 计点      bool
ka.go:263         ip          string
ka.go:264
ka.go:265         app 用户详情_推荐人 DB.DB_AppUser
ka.go:266         user 用户详情_推荐人 DB.DB_User
ka.go:267         ka 用户详情_推荐人  DB.DB_Ka
ka.go:268         用户类型_推荐人      DB.DB_UserClass
ka.go:269         新用户类型_推荐人      DB.DB_UserClass
ka.go:270
ka.go:271         logMoney      []DB.DB_LogMoney      //余额日志
ka.go:272         logVipNumber []DB.DB_LogVipNumber //积分,点数日志
ka.go:273
ka.go:274     }
ka.go:275
ka.go:276     if c == nil || c.Request == nil {
ka.go:277         info.ip = "未知" // 处理 `gin.Context` 为 `nil` 的情况
ka.go:278     } else {
ka.go:279         info.ip = c.ClientIP() //有可能报错 如果 c 不是外部请求过来的
ka.go:280     }
ka.go:281     //第一个查询不用 tx 直接用全局即可,后面事务的才用 tx
ka.go:282     db := *global.GVA_DB
ka.go:283     S_Ka := service.NewKa(c, &db)
ka.go:284     if info.卡号详情, err = S_Ka.InfoKa(卡号); err != nil {
ka.go:285         err = errors.New("卡号不存在")
ka.go:286         return
ka.go:287     }

```

第 49 页

```

ka.go:288     if info.卡号详情.Status == 2 {
ka.go:289         return errors.New("卡号已冻结,无法充值")
ka.go:290     }
ka.go:291     if info.卡号详情.Num >= info.卡号详情.NumMax {
ka.go:292         //开启事务前检测一次,过滤降低数据库压力
ka.go:293         return errors.New("卡号已经使用到最大次数")
ka.go:294     }
ka.go:295     if info.卡号详情.EndTime != 0 && info.卡号详情.EndTime <
time.Now().Unix() {
ka.go:296         //开启事务前检测一次,过滤降低数据库压力
ka.go:297         return errors.New("卡号已过有效期")
ka.go:298     }
ka.go:299     if 来源 AppId != 0 && 来源 AppId != info.卡号详情.AppId {
ka.go:300         return errors.New("不是本应用卡号")
ka.go:301     }

```



```

ka.go:302         if 充值用户 == 推荐人 {
ka.go:303             return errors.New("充值用户和推荐人不能相同")
ka.go:304         }
ka.go:305         if info.卡号详情.KaType == 2 && W文本_是否包含关键字(info.卡号详
情.User, 充值用户+",") {
ka.go:306             return errors.New("账号已使用本卡号充值过了,请勿重复充值")
ka.go:307         }
ka.go:308         if info.app 详情, err = service.NewAppInfo(c, &db).Info(info.卡号
详情.AppId); err != nil {
ka.go:309             err = errors.New("应用不存在")
ka.go:310             return
ka.go:311         }
ka.go:312
ka.go:313         //只有代理的卡号,才需要判断这个,webapi 和管理员的卡号不用
ka.go:314         if info.app 用户详情.AgentUid > 0 && info.卡号详情.RegisterId > 0
{
ka.go:315             //判断卡号制卡人,是否为当前用户的归属代理,无归属代理可以充值,有归
属代理,只允许充值归属代理的卡号
ka.go:316             switch info.app 详情.AgentKaUseModel {
ka.go:317                 case 1: //仅限充值自己的卡号
ka.go:318                     if info.卡号详情.RegisterId != info.app 用户详情.AgentUid
{
ka.go:319                         return errors.New("卡号异常,非该用户归属代理制卡")
ka.go:320                     }
ka.go:321                 }
ka.go:322             }
ka.go:323             info.is 卡号 = S三元(info.app 详情.AppType == 3 || info.app 详
情.AppType == 4, true, false)
ka.go:324             info.is 计点 = S三元(info.app 详情.AppType == 2 || info.app 详
情.AppType == 4, true, false)
ka.go:325
ka.go:326             if !info.is 卡号 {
ka.go:327                 info.user 用户详情, err = service.NewUser(c, &db).InfoName(充
值用户)
ka.go:328                 if err != nil {
ka.go:329                     return errors.New("用户不存在")
ka.go:330                 }
ka.go:331                 if info.ka 用户详情.Status == 2 {
ka.go:332                     return errors.New("用户已冻结,无法充值")
ka.go:333                 }
ka.go:334                 info.app 用户详情, err = service.NewAppUser(c, &db, info.卡号
详情.AppId).InfoUid(info.user 用户详情.Id)
ka.go:335             } else {
ka.go:336                 info.ka 用户详情, err = service.NewKa(c, &db).InfoKa(充值用户)
ka.go:337                 if err != nil {

```

```

ka.go:338         return errors.New("用户不存在")
ka.go:339     }
ka.go:340     if info.ka 用户详情.Status == 2 {
ka.go:341         return errors.New("用户已冻结,无法充值")
ka.go:342     }
ka.go:343     info.app 用户详情, err = service.NewAppUser(c, &db, info.卡号
详情.AppId).InfoUid(info.ka 用户详情.Id)
ka.go:344     }
ka.go:345     if err != nil {
ka.go:346         return errors.New("用户未登录过本应用,请先操作登录")
ka.go:347     }
ka.go:348     if info.app 用户详情.Status == 2 {
ka.go:349         return errors.New("app 用户已冻结,无法充值")
ka.go:350     }
ka.go:351
ka.go:352     if 推荐人 != "" {
ka.go:353         if !info.is 卡号 {
ka.go:354             info.user 用户详情_推荐人, err = service.NewUser(c,
&db).InfoName(推荐人)
ka.go:355             if err != nil {
ka.go:356                 return errors.New("推荐人用户不存在")
ka.go:357             }
ka.go:358             if info.user 用户详情_推荐人.Status == 2 {
ka.go:359                 return errors.New("推荐人用户已冻结,无法充值")
ka.go:360             }
ka.go:361             info.app 用户详情_推荐人, err = service.NewAppUser(c,
&db, info.卡号详情.AppId).InfoUid(info.user 用户详情_推荐人.Id)
ka.go:362         } else {
ka.go:363             info.ka 用户详情_推荐人, err = service.NewKa(c,
&db).InfoKa(推荐人)
ka.go:364             if err != nil {
ka.go:365                 return errors.New("推荐人用户不存在")
ka.go:366             }
ka.go:367             if info.ka 用户详情_推荐人.Status == 2 {
ka.go:368                 return errors.New("推荐人用户已冻结,无法充值")
ka.go:369             }
ka.go:370             info.app 用户详情_推荐人, err = service.NewAppUser(c,
&db, info.卡号详情.AppId).InfoUid(info.ka 用户详情_推荐人.Id)
ka.go:371         }
ka.go:372         if err != nil {
ka.go:373             return errors.New("推荐人用户未登录过本应用,请先操作登录")
ka.go:374         }
ka.go:375         if info.app 用户详情_推荐人.Status == 2 {

```

```

ka.go:376             return errors.New("推荐人 app 用户已冻结,无法充值")
ka.go:377         }
ka.go:378     }
ka.go:379     //检测用户分组是否相同 不相同处理
ka.go:380     if info.卡号详情.UserClassId == info.app 用户详情.UserClassId ||
info.app 用户详情.UserClassId == 0 {
ka.go:381         //分类相同,或用户为未分类 不处理
ka.go:382     } else {
ka.go:383         if info.卡号详情.NoUserClass == 2 {
ka.go:384             return errors.New("用户类型不同无法充值.")
ka.go:385         }
ka.go:386     }
ka.go:387     //到这里基本就都没问题了,开启事务,增加卡使用次数,更新用户信息就可以了

第 51 页

ka.go:388     // 开启事务,检测上层是否有事务,如果有直接使用,没有就创建一个
ka.go:389     var db2 *gorm.DB
ka.go:390     if tempObj, ok := c.Get("tx"); ok {
ka.go:391         db2 = tempObj.(*gorm.DB)
ka.go:392     } else {
ka.go:393         db = *global.GVA_DB
ka.go:394         db2 = &db
ka.go:395     }
ka.go:396     //在事务中执行数据库操作,使用的是 tx 变量, 不是 db。
ka.go:397     err = db2.Transaction(func(tx *gorm.DB) error {
ka.go:398         //加锁重新读卡信息
ka.go:399         err = tx.Model(DB.DB_Ka{}).Clauses(clause.Locking{Strength:
"UPDATE"}).Where("id=?", info.卡号详情.Id).First(&info.卡号详情).Error
ka.go:400         if err != nil {
ka.go:401             return errors.Join(err, errors.New("卡号不存在"))
ka.go:402         }
ka.go:403         if info.卡号详情.Num >= info.卡号详情.NumMax {
ka.go:404             return errors.Join(err, errors.New("卡号已经使用到最大次
数"))
ka.go:405         }
ka.go:406         //已用次数+1
ka.go:407         //RowsAffected 用于返回 sql 执行后影响的行数
ka.go:408         m := map[string]interface{}{}
ka.go:409         m["Num"] = gorm.Expr("Num + 1")
ka.go:410         m["User"] = gorm.Expr("CONCAT(User,?)", 充值用户+",")
ka.go:411         m["UserTime"] = gorm.Expr("CONCAT(UserTime,?)",
strconv.Itoa(int(time.Now().Unix()))+",")
ka.go:412         m["UseTime"] = time.Now().Unix()
ka.go:413         m["InviteUser"] = gorm.Expr("CONCAT(InviteUser,?)", 推荐人
+",") //空推荐人也增加,这样才能和用户充值顺序对应
ka.go:414         rowsAffected := tx.Model(DB.DB_Ka{}).

```

```

ka.go:415             Where("Id = ?", info.卡号详
情.Id).Updates(&m).RowsAffected
ka.go:416             if rowsAffected == 0 {
ka.go:417                 return errors.New("卡号不存在或已经使用到最大次数")
ka.go:418             }
ka.go:419             //卡库存减少成功,开始增加客户数据 ,重新加锁读取 App 用户信息,防止
并发数据错误
ka.go:420             err =
tx.Model(DB.DB_AppUser{}).Clauses(clause.Locking{Strength:
"UPDATE"}).Table("db_AppUser_"+strconv.Itoa(info.卡号详
情.AppId)).Where("Uid=?", info.app 用户详情.Uid).First(&info.app 用户详
情).Error
ka.go:421             if err != nil {
ka.go:422                 return errors.Join(err, errors.New("未注册应用???感觉不可
能,之前读取过,请联系管理员"))
ka.go:423             }
ka.go:424             //处理新信息
ka.go:425             客户 expr := map[string]interface{}{}
ka.go:426             if info.卡号详情.VipNumber > 0 {
ka.go:427                 客户 expr["VipNumber"] = gorm.Expr("VipNumber + ?",
info.卡号详情.VipNumber) //积分不会变直接增加即可
ka.go:428                 info.logVipNumber = append(info.logVipNumber,
DB.DB_LogVipNumber{
ka.go:429                     AppId: info.卡号详情.AppId,
ka.go:430                     Count: info.卡号详情.VipNumber,
ka.go:431                     Ip:     info.ip,
ka.go:432                     Note:  "应用 ID:" + strconv.Itoa(info.卡号详
情.AppId) + "卡号 Id:" + strconv.Itoa(info.卡号详情.Id) + "充值积分",
ka.go:433                     Time:  time.Now().Unix(),
ka.go:434                     Type:  1,
ka.go:435                     User:  充值用户,
ka.go:436                 })
ka.go:437             }

```

第 52 页

```

ka.go:438
ka.go:439             if info.卡号详情.MaxOnline > 0 {
ka.go:440                 客户 expr["MaxOnline"] = info.卡号详情.MaxOnline //最大在
线数直接赋值处理即可
ka.go:441             }
ka.go:442
ka.go:443             局_现行时间戳 := time.Now().Unix()
ka.go:444             if info.卡号详情.VipTime != 0 { //只有时间增减不为 0 的时候设置
的用户分类才有效
ka.go:445                 if info.app 用户详情.UserClassId == info.卡号详
情.UserClassId {
ka.go:446                     //分类相同,正常处理时间或点数

```

```

ka.go:447             if Ser_AppInfo.App 是否为计点(info.卡号详情.AppId) ||
info.app 用户详情.VipTime > 局_现行时间戳 {
ka.go:448                 //如果为计点 或 时间大于现在时间直接加就行了
ka.go:449                 客户 expr["VipTime"] = gorm.Expr("VipTime + ?",
info.卡号详情.VipTime)
ka.go:450             } else {
ka.go:451                 //如果为计时 已经过期很久了,直接现行时间戳加卡时间
ka.go:452                 客户 expr["VipTime"] = 局_现行时间戳 + info.卡号
详情.VipTime
ka.go:453             }
ka.go:454         } else {
ka.go:455             //用户类型不同, 根据权重处理
ka.go:456             var 局_旧用户类型权重, 局_新用户类型权重
DB.DB_UserClass
ka.go:457             if info.app 用户详情.UserClassId > 0 {
ka.go:458                 if 局_旧用户类型权重, err =
service.NewUserClass(c, tx).Info(info.app 用户详情.UserClassId);
err != nil {
ka.go:459                     return errors.Join(err, errors.New("读取旧
用户类型权重失败"))
ka.go:460                 }
ka.go:461             } else {
ka.go:462                 局_旧用户类型权重.Weight = 1
ka.go:463             }
ka.go:464
ka.go:465             if info.卡号详情.UserClassId > 0 {
ka.go:466                 if 局_新用户类型权重, err =
service.NewUserClass(c, tx).Info(info.卡号详情.UserClassId); err !=
nil {
ka.go:467                     return errors.Join(err, errors.New("读取新
用户类型权重失败"))
ka.go:468                 }
ka.go:469             } else {
ka.go:470                 局_新用户类型权重.Weight = 1
ka.go:471             }
ka.go:472
ka.go:473             if info.is 计点 {
ka.go:474                 //转换结果值,转后再增加新类型 值
ka.go:475                 客户 expr["VipTime"] = gorm.Expr("VipTime * ?
/ ? +?", 局_旧用户类型权重.Weight, 局_新用户类型权重.Weight, info.卡号详
情.VipTime)
ka.go:476                 info.logVipNumber = append(info.logVipNumber,
DB.DB_LogVipNumber{
ka.go:477                     User:  info.user 用户详情.User,
ka.go:478                     Ip:    info.ip,
ka.go:479                     Count: info.卡号详情.VipNumber,

```

```

ka.go:480             Note: "应用 ID:" + strconv.Itoa(info.卡号
详情.AppId) + "卡号 Id:" + strconv.Itoa(info.卡号详情.Id) + "充值点数",
ka.go:481             })
ka.go:482             } else {
ka.go:483                 if info.app 用户详情.VipTime < 局_现行时间戳 {
ka.go:484                     //已经过期了直接赋值新类型 现行时间+新时间就可
以了
ka.go:485                     客户 expr["VipTime"] = 局_现行时间戳 + info.
卡号详情.VipTime
ka.go:486                 } else {
ka.go:487                     //先计算还剩多长时间,剩余时间权重转换转换结果
值,+现在时间+卡增减时间
第 53 页
ka.go:488                     客户 expr["VipTime"] =
gorm.Expr("(VipTime-?) * ? / ? +?", 局_现行时间戳, 局_旧用户类型权
重.Weight, 局_新用户类型权重.Weight, 局_现行时间戳+info.卡号详
情.VipTime)
ka.go:489                 }
ka.go:490             }
ka.go:491             //最后更换类型,防止前面用到卡类 id,计算权重转换类型错误
ka.go:492             客户 expr["UserClassId"] = info.卡号详情.UserClassId
ka.go:493         }
ka.go:494     }
ka.go:495     //更新客户数据
ka.go:496     err =
tx.Model(DB.DB_AppUser{}).Table("db_AppUser_"+strconv.Itoa(info.卡号
详情.AppId)).Where("Id = ?", info.app 用户详情.Id).Updates(&客户
expr).Error
ka.go:497     if err != nil {
ka.go:498         return errors.Join(err, errors.New("充值失败,重试"))
ka.go:499     }
ka.go:500     //处理账号的 RMB 增减
ka.go:501     if !info.is 卡号 && info.卡号详情.RMb > 0 {
ka.go:502         err =
tx.Model(DB.DB_User{}).Clauses(clause.Locking{Strength:
"UPDATE"}).Where("Id=?", info.app 用户详情.Uid).First(&info.user 用户详
情).Error
ka.go:503         if err != nil {
ka.go:504             return errors.Join(err, errors.New("用户账号不存在
"))
ka.go:505         }
ka.go:506         err = tx.Model(DB.DB_User{}).Where("Id = ?", info.app 用
户详情.Uid).Update("RMB", gorm.Expr("RMB + ?", info.卡号详
情.RMb)).Error
ka.go:507         if err != nil {
ka.go:508             return errors.Join(err, errors.New("充值余额时失败,
请重试"))

```

```

ka.go:509         }
ka.go:510         var 局_新余额 float64
ka.go:511         err = tx.Model(DB.DB_User{}).Select("Rmb").Where("Id
= ?", info.user 用户详情.Id).First(&局_新余额).Error
ka.go:512         if err != nil {
ka.go:513             return errors.Join(err, errors.New("充值后读取新余额
失败"))
ka.go:514         }
ka.go:515         //日志仅写到上下文内,由实际业务处理是否写入日志和修改备注信息
ka.go:516         info.logMoney = append(info.logMoney, DB.DB_LogMoney{
ka.go:517             User:  info.user 用户详情.User,
ka.go:518             Ip:    info.ip,
ka.go:519             Count: info.卡号详情.Rmb,
ka.go:520             Note:  "应用 ID:" + strconv.Itoa(info.卡号详
情.AppId) + "卡号 Id:" + strconv.Itoa(info.卡号详情.Id) + "充值余额|新余
额=" + Float64 到文本(局_新余额, 2),
ka.go:521         })
ka.go:522     }
ka.go:523     //开始处理推荐人
ka.go:524     if info.卡号详情.InviteCount > 0 && info.app 用户详情_推荐
人.Id != 0 {
ka.go:525         //处理新信息
ka.go:526         推荐人 expr := map[string]interface{}{}
ka.go:527         if info.app 用户详情_推荐人.UserClassId == info.卡号详
情.UserClassId {
ka.go:528             //分类相同,正常处理时间或点数
ka.go:529             if info.is 计点 || info.app 用户详情_推荐人.VipTime >
局_现行时间戳 {
ka.go:530                 //如果为计点 或 时间大于现在时间直接加就行了
ka.go:531                 推荐人 expr["VipTime"] = gorm.Expr("VipTime
+ ?", info.卡号详情.InviteCount)
ka.go:532             } else {
ka.go:533                 //如果为计时 已经过期很久了,直接现行时间戳加卡时间
ka.go:534                 推荐人 expr["VipTime"] = 局_现行时间戳 + info.卡
号详情.InviteCount
ka.go:535             }
ka.go:536         } else {
ka.go:537             第 54 页
ka.go:538             //用户类型不同,根据权重处理
ka.go:539             if info.用户类型_推荐人, err =
service.NewUserClass(c, tx).Info(info.app 用户详情_推荐人.UserClassId);
err != nil {
ka.go:540                 return errors.New("读取推荐人用户类型详情失败,可
能已不存在 id:" + strconv.Itoa(info.app 用户详情_推荐人.UserClassId))

```

```

ka.go:541                                }
ka.go:542                                if info.新用户类型_推荐人, err =
service.NewUserClass(c, tx).Info(info.卡号详情.UserClassId); err !=
nil {
ka.go:543                                    return errors.New("读取卡号详情 UserClassId 失败,
可能已不存在 id:" + strconv.Itoa(info.卡号详情.UserClassId))
ka.go:544                                }
ka.go:545
ka.go:546                                if info.is 计点 {
ka.go:547                                    //计算推荐人用户类型的实际点数,
ka.go:548                                    //这里有点绕,比如增加 1 秒,但是这个新用户类型权重为
2, 荐人类型权重为 1
ka.go:549                                    //那么实际新用户类型是推荐人类型的两倍,转换到推荐人
类型值应该为 2
ka.go:550                                    //所以 卡秒数+新用户类型权重=2,在除以推荐人用户类
型权重=2
ka.go:551                                    局_增减时间点数 := info.卡号详情.InviteCount *
info.新用户类型_推荐人.Weight / info.用户类型_推荐人.Weight
ka.go:552                                    推荐人 expr["VipTime"] = gorm.Expr("VipTime
+?", 局_增减时间点数)
ka.go:553                                    info.logVipNumber = append(info.logVipNumber,
DB.DB_LogVipNumber{
ka.go:554                                        User:  info.user 用户详情.User,
ka.go:555                                        Ip:    info.ip,
ka.go:556                                        Count: Int64 到 Float64(局_增减时间点数),
ka.go:557                                        Note:  "应用 ID:" + strconv.Itoa(info.卡号
详情.AppId) + "用户:" + 充值用户 + ",使用充值卡号 Id:" +
strconv.Itoa(info.卡号详情.Id) + ",获得推荐人增加点数",
ka.go:558                                    })
ka.go:559
ka.go:560                                } else {
ka.go:561                                    局_增减时间点数 := info.卡号详情.InviteCount *
info.新用户类型_推荐人.Weight / info.用户类型_推荐人.Weight
ka.go:562                                    if info.app 用户详情_推荐人.VipTime < 局_现行时间
戳 {
ka.go:563                                        //已经过期了 现行时间+新时间就可以了
ka.go:564                                        推荐人 expr["VipTime"] = 局_现行时间戳 + 局_
增减时间点数
ka.go:565                                    } else {
ka.go:566                                        //原来的值+推荐人增加点数权重转换结果就好了
ka.go:567                                        推荐人 expr["VipTime"] =
gorm.Expr("VipTime+?", 局_增减时间点数)
ka.go:568                                    }
ka.go:569                                }
ka.go:570                            }

```



```

ka.go:571         err =
tx.Model(DB.DB_AppUser{}).Table("db_AppUser_"+strconv.Itoa(info.卡号
详情.AppId)).Where("Uid = ?", info.app 用户详情_推荐人.Uid).Updates(&推
荐人 expr).Error

ka.go:572         if err != nil {
ka.go:573             return errors.Join(err, errors.New("推荐人更新失败
"))
        }
ka.go:574     }
ka.go:575     }
ka.go:576
ka.go:577     return nil
ka.go:578 })
ka.go:579 //写到上下文,备用
ka.go:580 c.Set("info.卡类详情", info.卡号详情)
ka.go:581 c.Set("info.app 用户详情", info.app 用户详情)
ka.go:582 c.Set("info.user 用户详情", info.user 用户详情)
ka.go:583 c.Set("info.app 详情", info.app 详情)
ka.go:584 if len(info.logMoney) > 0 {
ka.go:585     //最后写出日志
ka.go:586     if err = log.L_log.S 输出日志(c, info.logMoney); err != nil {
ka.go:587         global.GVA_LOG.Error("输出日志失败!", zap.Any("err",
err))

                                第 55 页

ka.go:588     }
ka.go:589 }
ka.go:590
ka.go:591 if len(info.logVipNumber) > 0 {
ka.go:592     if err = log.L_log.S 输出日志(c, info.logVipNumber); err !=
nil {
ka.go:593         global.GVA_LOG.Error("输出日志失败!", zap.Any("err",
err))
ka.go:594     }
ka.go:595 }
ka.go:596
ka.go:597 //卡号充值时,如果充值的用户没有归属代理,则自动设置该卡号的制卡人为归属
代理
ka.go:598 if info.app 用户详情.AgentUid == 0 && info.卡号详情.RegisterId > 0
{
ka.go:599     j.Z 置归属代理(c, info.卡号详情.AppId, info.app 用户详情.Uid,
info.卡号详情.RegisterId) //失败也不影响
ka.go:600 }
ka.go:601
ka.go:602     return err
ka.go:603 }
ka.go:604
ka.go:605 // 已用充值卡将相应的卡使用者和推荐人强行扣回充值卡面值,可能扣成负数
ka.go:606 func (j *ka) K 卡号追回(c *gin.Context, Id int, 操作人 string) (err
error) {

```

```

ka.go:607         var info struct {
ka.go:608             卡号详情          DB.DB_Ka
ka.go:609             卡号应用          DB.DB_AppInfo
ka.go:610             is 卡号          bool
ka.go:611             is 计点          bool
ka.go:612             vipTime 名称      string
ka.go:613             已充值用户        []DB.DB_AppUser
ka.go:614             操作人详情        DB.DB_User
ka.go:615             LogMoney          []DB.DB_LogMoney
ka.go:616             LogVipNumber      []DB.DB_LogVipNumber
ka.go:617         }
ka.go:618         // 开启事务,检测上层是否有事务,如果有直接使用,没有就创建一个
ka.go:619         var db *gorm.DB
ka.go:620         if tempObj, ok := c.Get("tx"); ok {
ka.go:621             db = tempObj.(*gorm.DB)
ka.go:622         } else {
ka.go:623             db2 := *global.GVA_DB
ka.go:624             db = &db2
ka.go:625         }
ka.go:626         if info.卡号详情, err = service.NewKa(c, db).Info(Id); err != nil
{
ka.go:627             err = errors.Join(err, errors.New("卡号不存在"))
ka.go:628             return
ka.go:629         }
ka.go:630         if info.卡号详情.Num == 0 {
ka.go:631             return errors.New("卡号未使用")
ka.go:632         }
ka.go:633
ka.go:634         //防 sb 客户放负值
ka.go:635         if info.卡号详情.VipTime < 0 || info.卡号详情.VipNumber < 0 ||
info.卡号详情.RMb < 0 {
ka.go:636             return errors.New("追回的卡号,充值的时间点数,积分,rmb,不能为负
数,请手动处理")
ka.go:637         }

```

第 56 页

```

ka.go:638
ka.go:639         if info.卡号应用, err = service.NewAppInfo(c, db).Info(info.卡号详
情.AppId); err != nil {
ka.go:640             err = errors.Join(err, errors.New("应用不存在")) //概率较小,但
是有可能,比如制卡使用后把应用删除了,然后代理追回卡号
ka.go:641             return
ka.go:642         }
ka.go:643         info.is 卡号 = info.卡号应用.AppType == 3 || info.卡号应用.AppType
== 4
ka.go:644         info.is 计点 = info.卡号应用.AppType == 2 || info.卡号应用.AppType
== 4

```

```

ka.go:645
ka.go:646         if info.is 卡号 && info.卡号详情.User == "" {
ka.go:647             return errors.New("卡号模式应用,已转换软件用户的卡号,无法追回")
ka.go:648         }
ka.go:649
ka.go:650         if info.卡号详情.User == "" {
ka.go:651             return errors.New("无已充值用户,但有使用次数,可能手动修改")
ka.go:652         }
ka.go:653
ka.go:654         info.vipTime 名称 = S 三元(info.is 计点, "点数", "时间")
ka.go:655
ka.go:656         已用用户数组 := W 文本_分割文本(info.卡号详情.User, ",")
ka.go:657         已用推荐人数组 := W 文本_分割文本(info.卡号详情.InviteUser, ",")
ka.go:658
ka.go:659         err = db.Transaction(func(tx *gorm.DB) error {
ka.go:660             //tx = tx.Debug()
ka.go:661             for 索引, 值 := range 已用用户数组 {
ka.go:662                 if 值 == "" {
ka.go:663                     continue //如果值为空,到循环尾
ka.go:664                 }
ka.go:665
ka.go:666                 var 临时软件用户 info DB.DB_AppUser
ka.go:667                 var 临时账号 info DB.DB_User
ka.go:668                 var 临时卡号 info DB.DB_Ka
ka.go:669
ka.go:670                 if info.is 卡号 {
ka.go:671                     tempTx :=
tx.Model(DB.DB_Ka{}).Clauses(clause.Locking{Strength:
"UPDATE"}).First(&临时卡号 info, "Name = ?", 值)
ka.go:672                     if tempTx.Error != nil {
ka.go:673                         return errors.Join(tempTx.Error, errors.New(值
+"已充卡号不存在"))
ka.go:674                     }
ka.go:675                     } else {
ka.go:676                         tempTx :=
tx.Model(DB.DB_User{}).Clauses(clause.Locking{Strength:
"UPDATE"}).First(&临时账号 info, "User = ?", 值)
ka.go:677                         if tempTx.Error != nil {
ka.go:678                             return errors.Join(tempTx.Error, errors.New("
已充账号不存在"))
ka.go:679                         }
ka.go:680                     }
ka.go:681                     tempTx :=
tx.Model(DB.DB_AppUser{}).Table("db_AppUser_"+strconv.Itoa(info.卡号
详情.AppId)).Clauses(clause.Locking{Strength: "UPDATE"}).
ka.go:682                     First(&临时软件用户 info, "Uid = ?", S 三元(info.is 卡
号, 临时卡号 info.Id, 临时账号 info.Id))

```

```

ka.go:683         if tempTx.Error != nil {
ka.go:684             return errors.Join(tempTx.Error, errors.New(值+"已
充软件用户 info 不存在"))
ka.go:685         }
ka.go:686         临时软件用户 info.VipNumber = Float64 减 float64(临时软件用
户 info.VipNumber, info.卡号详情.VipNumber, 2)
ka.go:687
第 57 页
ka.go:688         局_临时整数, err2 := userClass.L_userClass.J 计算权重值
2(c, info.卡号详情.UserClassId, 临时软件用户 info.UserClassId, info.卡号
详情.VipTime)
ka.go:689         if err2 != nil {
ka.go:690             return errors.Join(tempTx.Error, errors.New(值+"计
算权重值失败"))
ka.go:691         }
ka.go:692         临时软件用户 info.VipTime -= 局_临时整数
ka.go:693         update := map[string]interface{}{
ka.go:694             "VipTime": 临时软件用户 info.VipTime,
ka.go:695             "VipNumber": 临时软件用户 info.VipNumber,
ka.go:696         }
ka.go:697         err =
tx.Model(DB.DB_AppUser{}).Table("db_AppUser_"+strconv.Itoa(info.卡号
详情.AppId)).Where("Id = ?", 临时软件用户 info.Id).
Updates(&update).Error
ka.go:698         if err != nil {
ka.go:699             return errors.Join(err, errors.New("修改
uid:"+strconv.Itoa(临时软件用户 info.Uid)+"软件用户信息失败"))
ka.go:700         }
ka.go:701         if info.卡号详情.VipTime != 0 {
ka.go:702             info.LogVipNumber = append(info.LogVipNumber,
DB.DB_LogVipNumber{
ka.go:703                 User: S 三元(info.is 卡号, 临时卡号 info.Name,
临时账号 info.User),
ka.go:704                 AppId: info.卡号详情.AppId,
ka.go:705                 Type: S 三元(info.is 计点, constant.Log_type_点
数, constant.Log_type_时间),
ka.go:706                 Time: time.Now().Unix(),
ka.go:707                 Ip: c.ClientIP(),
ka.go:708                 Count: Int64 到 Float64(info.卡号详情.VipTime),
ka.go:709                 Note: fmt.Sprintf(操作人+"操作追回 id:%d,卡
号:%s,扣除卡号充值"+info.vipTime 名称+", 用户类型 id:卡号%d->用户%d",
info.卡号详情.Id, info.卡号详情.Name, info.卡号详情.UserClassId, 临时软
件用户 info.UserClassId),
ka.go:710             })
ka.go:711         }
ka.go:712         if info.卡号详情.VipNumber != 0 {
ka.go:713

```

```

ka.go:714             info.LogVipNumber = append(info.LogVipNumber,
DB.DB_LogVipNumber{
ka.go:715                 User:  S 三元(info.is 卡号, 临时卡号 info.Name,
临时账号 info.User),
ka.go:716                 AppId: info.卡号详情.AppId,
ka.go:717                 Type:  constant.Log_type_积分,
ka.go:718                 Time:  time.Now().Unix(),
ka.go:719                 Ip:    c.ClientIP(),
ka.go:720                 Count: Float64 取负值(info.卡号详情.VipNumber),
ka.go:721                 Note:  fmt.Sprintf(操作人+"操作追回 id:%d,卡
号:%s,扣除卡号充值积分|新积分≈%s", info.卡号详情.Id, info.卡号详情.Name,
Float64 到文本(临时软件用户 info.VipNumber, 2)),
ka.go:722             })
ka.go:723         }
ka.go:724
ka.go:725         if info.is 卡号 {
ka.go:726             // 卡号没什么可变更的
ka.go:727         } else {
ka.go:728             //余额只需要改 RMB
ka.go:729             if info.卡号详情.Rmb != 0 {
ka.go:730                 临时账号 info.Rmb = Float64 减 float64(临时账号
info.Rmb, info.卡号详情.Rmb, 2)
ka.go:731                 tempTx = tx.Model(DB.DB_User{}).Where("Id
= ?", 临时账号 info.Id).
ka.go:732                     Update("Rmb", 临时账号 info.Rmb)
ka.go:733                 if tempTx.Error != nil {
ka.go:734                     return errors.Join(tempTx.Error,
errors.New("修改 id:"+strconv.Itoa(临时账号 info.Id)+"用户 Rmb 信息失败
"))
ka.go:735                 }
ka.go:736                 info.LogMoney = append(info.LogMoney,
DB.DB_LogMoney{
ka.go:737                     User:  临时账号 info.User,
第 58 页
ka.go:738                     Time:  time.Now().Unix(),
ka.go:739                     Ip:    c.ClientIP(),
ka.go:740                     Count: Float64 取负值(info.卡号详情.Rmb),
ka.go:741                     Note:  fmt.Sprintf(操作人+"操作追回 id:%d,卡
号:%s,扣除卡号充值余额≈%s|新余额≈%s", info.卡号详情.Id, info.卡号详
情.Name, Float64 到文本(info.卡号详情.Rmb, 2), Float64 到文本(临时账号
info.Rmb, 2)),
ka.go:742                 })
ka.go:743             }
ka.go:744         }
ka.go:745
ka.go:746         //开始处理推荐人增加的时间点数
ka.go:747         if 索引 >= len(已用推荐人数组) {

```

```

ka.go:748             continue
ka.go:749         }
ka.go:750         值 = 已用推荐人数组[索引]
ka.go:751         if 值 == "" {
ka.go:752             continue
ka.go:753         }
ka.go:754         // First 传入的如果带主键 id 会自动增加这个条件
ka.go:755         临时账号 info = DB.DB_User{}
ka.go:756         临时卡号 info = DB.DB_Ka{}
ka.go:757         临时软件用户 info = DB.DB_AppUser{}
ka.go:758         if info.is 卡号 {
ka.go:759             tempTx =
ka.go:760             tx.Model(DB.DB_Ka{}).Clauses(clause.Locking{Strength:
ka.go:761             "UPDATE"}).First(&临时卡号 info, "Name = ?", 值)
ka.go:762             if tempTx.Error != nil {
ka.go:763                 return errors.Join(tempTx.Error, errors.New("
ka.go:764                 推荐人["+值+"]卡号不存在"))
ka.go:765             }
ka.go:766             } else {
ka.go:767                 tempTx =
ka.go:768                 tx.Debug().Model(DB.DB_User{}).Clauses(clause.Locking{Strength:
ka.go:769                 "UPDATE"}).First(&临时账号 info, "User = ?", 值)
ka.go:770                 if tempTx.Error != nil {
ka.go:771                     return errors.Join(tempTx.Error, errors.New("
ka.go:772                     推荐人["+值+"]账号不存在"))
ka.go:773                 }
ka.go:774             }
ka.go:775             tempTx =
ka.go:776             tx.Model(DB.DB_AppUser{}).Table("db_AppUser_"+strconv.Itoa(info.卡号
ka.go:777             详情.AppId)).
ka.go:778             Clauses(clause.Locking{Strength: "UPDATE"}).
ka.go:779             First(&临时软件用户 info, "Uid = ?", S三元(info.is 卡
ka.go:780             号, 临时卡号 info.Id, 临时账号 info.Id))
ka.go:781             if tempTx.Error != nil {
ka.go:782                 return errors.Join(tempTx.Error, errors.New(值+"推
ka.go:783                 荐人软件用户 info 不存在"))
ka.go:784             }
ka.go:785             局_临时整数, err2 = userClass.L_userClass.J 计算权重值
ka.go:786             2(c, info.卡号详情.UserClassId, 临时软件用户 info.UserClassId, info.卡号
ka.go:787             详情.InviteCount)
ka.go:788             if err2 != nil {
ka.go:789                 return errors.Join(tempTx.Error, errors.New(值+"计
ka.go:790                 算推荐人权重值失败"))
ka.go:791             }
ka.go:792             临时软件用户 info.VipTime -= 局_临时整数
ka.go:793             update = map[string]interface{}{
ka.go:794                 "VipTime": 临时软件用户 info.VipTime,
ka.go:795             }

```

```

ka.go:783         err =
tx.Model(DB.DB_AppUser{}).Table("db_AppUser_"+strconv.Itoa(info.卡号
详情.AppId)).Where("Id = ?", 临时软件用户 info.Id).

ka.go:784             Updates(&update).Error
ka.go:785             if err != nil {
ka.go:786                 return errors.Join(err, errors.New("修改推荐人
uid:"+strconv.Itoa(临时软件用户 info.Uid)+"软件用户信息失败"))
ka.go:787             }

                                第 59 页

ka.go:788             if info.卡号详情.VipTime != 0 {
ka.go:789                 info.LogVipNumber = append(info.LogVipNumber,
DB.DB_LogVipNumber{
ka.go:790                     User:   S 三元(info.is 卡号, 临时卡号 info.Name,
临时账号 info.User),
ka.go:791                     AppId:  info.卡号详情.AppId,
ka.go:792                     Type:   S 三元(info.is 计点, constant.Log_type_点
数, constant.Log_type_时间),
ka.go:793                     Time:  time.Now().Unix(),
ka.go:794                     Ip:    c.ClientIP(),
ka.go:795                     Count:  Int64 到 Float64(info.卡号详情.VipTime),
ka.go:796                     Note:  fmt.Sprintf(操作人+"操作追回 id:%d,卡
号:%s,扣除卡号推荐人充值"+S 三元(info.is 计点, "点数", "时间"), info.卡号
详情.Id, info.卡号详情.Name),
ka.go:797                     })
ka.go:798             }
ka.go:799         }
ka.go:800     }
ka.go:801     //重置卡并冻结,删除信息
ka.go:802     err = tx.Model(DB.DB_Ka{}).Where("Id = ? ", info.卡号详
情.Id).Updates(
ka.go:803         map[string]interface{}{
ka.go:804             "Status":    2,
ka.go:805             "User":        "",
ka.go:806             "Num":           0,
ka.go:807             "InviteUser":     "",
ka.go:808             "UserTime":       "",
ka.go:809             "UseTime":        0,
ka.go:810             "AdminNote":     info.卡号详情.AdminNote + "已被追回,历
史充值用户:" + info.卡号详情.User + ",推荐人用户:" + info.卡号详
情.InviteUser,
ka.go:811         }).Error
ka.go:812     return err
ka.go:813 }
ka.go:814
ka.go:815 if err == nil {
ka.go:816     //如果是上层传递的,就返回日志,否则直接输出
ka.go:817     if _, ok := c.Get("tx"); ok {
ka.go:818         c.Set("info.LogVipNumber", info.LogVipNumber)

```

```

ka.go:819             c.Set("info.LogMoney", info.LogMoney)
ka.go:820         } else {
ka.go:821             if err = log.L_log.S 输出日志(c, info.LogMoney); err !=
nil {
ka.go:822                 global.GVA_LOG.Error("输出日志失败!", zap.Any("err",
err))
ka.go:823             }
ka.go:824             if err = log.L_log.S 输出日志(c, info.LogVipNumber);
err != nil {
ka.go:825                 global.GVA_LOG.Error("输出日志失败!", zap.Any("err",
err))
ka.go:826             }
ka.go:827         }
ka.go:828     }
ka.go:829
ka.go:830     return err
ka.go:831 }
ka.go:832
ka.go:833 // 删除已耗尽次数卡号
ka.go:834 func (j *ka) S 删除耗尽次数卡号(c *gin.Context, AppId int) (数量 int64,
err error) {
ka.go:835     var info struct {
ka.go:836         卡号详情 []DB.DB_Ka
ka.go:837         删除 id []int

```